HKU Summer Course

Introduction to Artificial Intelligence

Jintao Ke

Department of Civil Engineering
Faculty of Engineering
The University of Hong Kong, Pokfulam, Hong Kong
Email: kejintao@hku.hk

https://sites.google.com/view/kejintao

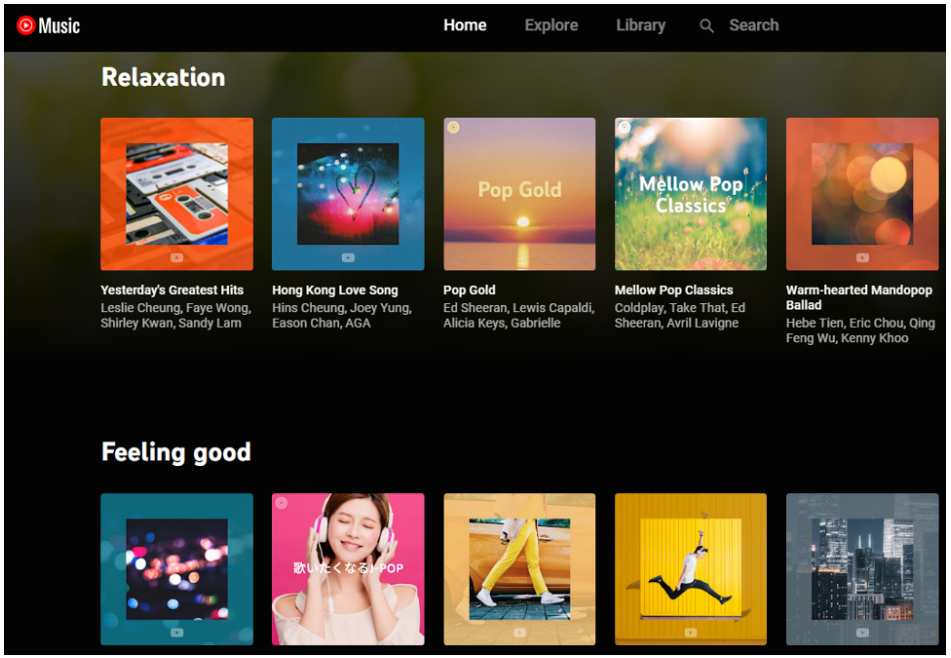
Outline

1. Introduction to AI
2. Useful Data Platforms
3. Fundamentals of AI

A Question to Start

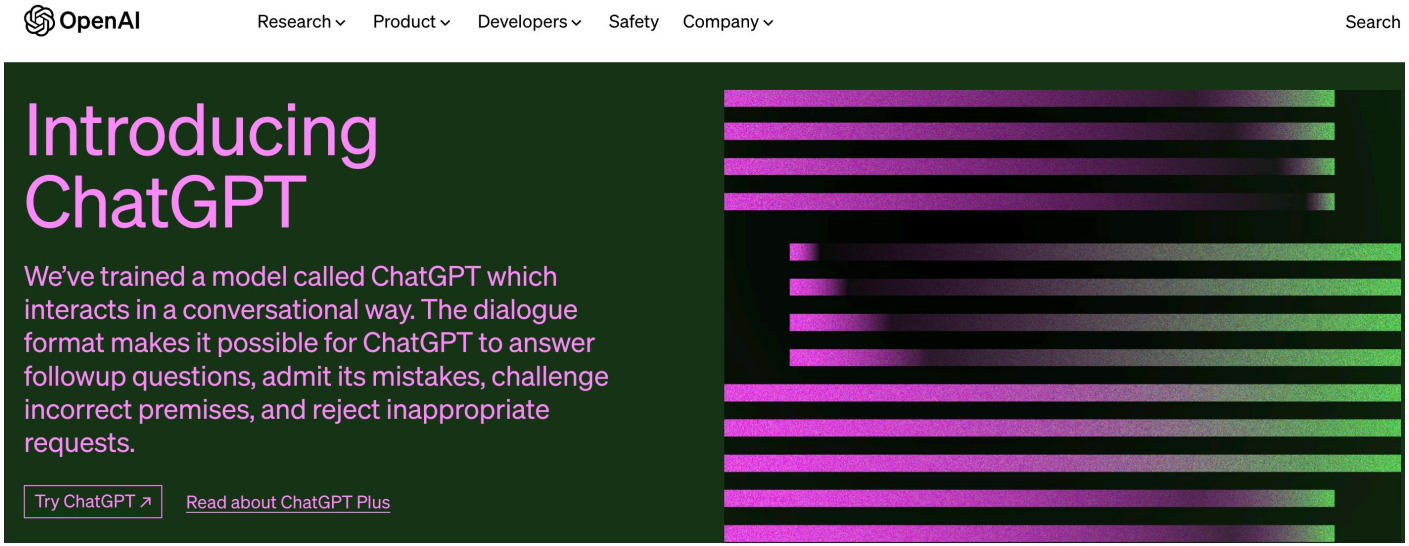
- AI may already have recommended videos, translated text, unlocked your phone, routed your trip, filtered spam, or helped write code.
- AI is no longer only a research topic. It is becoming part of everyday infrastructure.

Before this lecture, AI has already been around you.



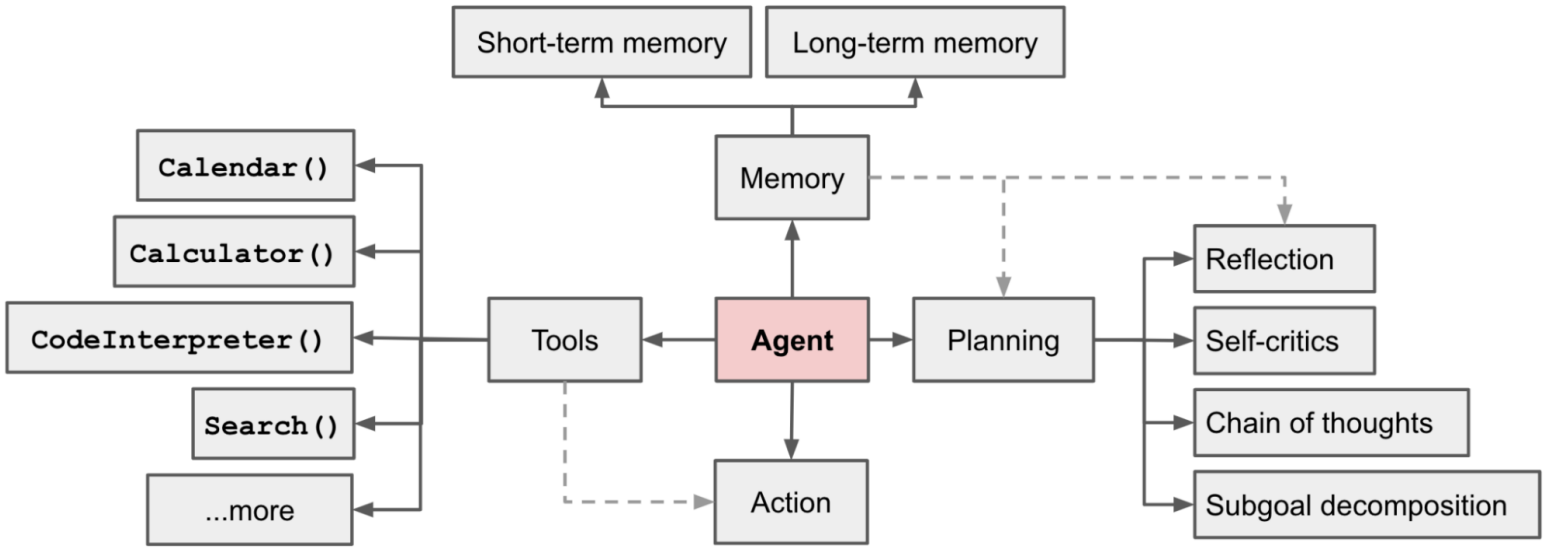
AI Is Becoming the New Interface

- ChatGPT (the first widely adopted AI chatbot), Gemini, Claude, and Copilot turn natural language into answers, code, summaries, and plans.
- The interface is changing from clicking menus to describing goals in plain language.
- The same chatbox can connect to files, images, code, search, and external tools.



From Search Engine to AI Agent

- A search engine returns links; a chatbot generates responses.
- An AI agent can plan steps, use tools, check outputs, and iterate toward a goal.
- This is why the same model can become a coding assistant, a research assistant, or a workflow automation tool.



Source: Lilian Weng, LLM Powered Autonomous Agents

Common AI Tools You May Already Know

Chatbots
ChatGPT, Gemini, Claude, Grok

Coding Agents
Cursor, Claude Code, Codex, Devin

Search / Research
Perplexity, Elicit, NotebookLM, Deep Research

Productivity
Microsoft Copilot, Notion AI, Canva AI

Creative AI
Midjourney, Firefly, Sora, Veo

General Agents
OpenAI Operator, Manus, WorkBuddy, OpenClaw

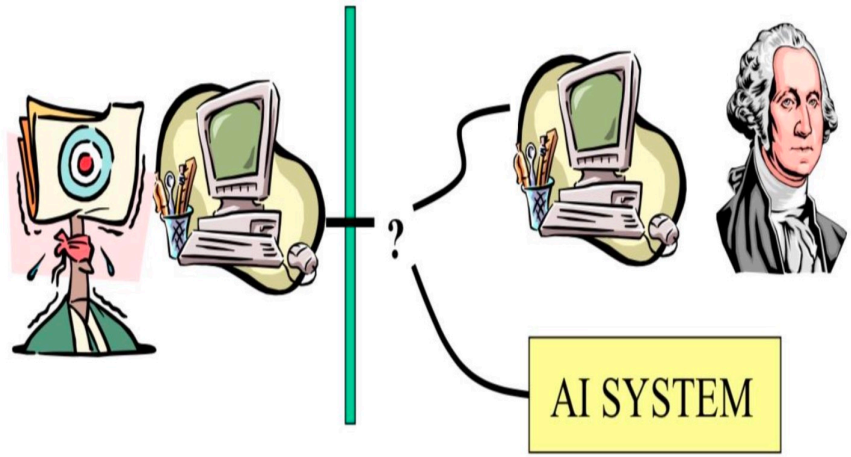
The important shift: AI tools are becoming interfaces to many digital tasks, not just another search box.

What Is Artificial Intelligence?

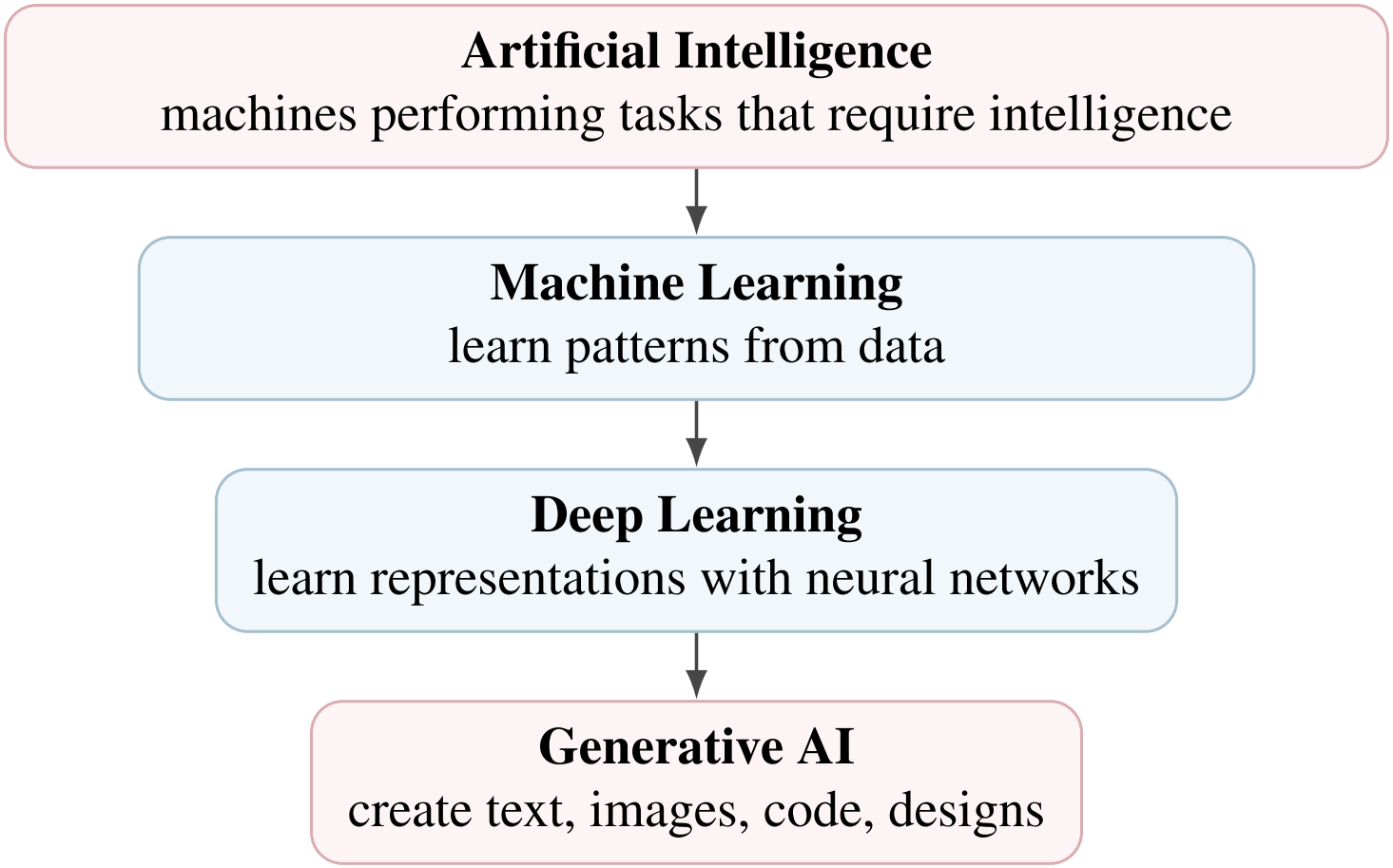
- John McCarthy described AI as the science and engineering of making intelligent machines.
- A practical view: AI helps machines perceive, learn, reason, generate, and act.
- Modern AI often learns from data rather than relying only on hand-written rules.

The Turing Test: Can Machines Think?

- In 1950, Alan Turing proposed an operational question: can a machine imitate human conversation well enough that a human judge cannot reliably tell the difference?
- The Turing Test is not the only definition of intelligence, but it shaped how people think about machine intelligence.
- Modern chatbots make this question feel much less abstract than it did in 1950.



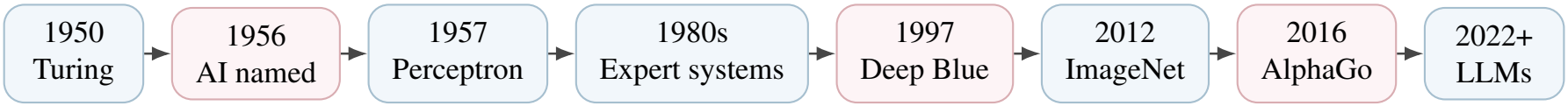
AI, Machine Learning, Deep Learning, Generative AI



How Does AI Work?

- AI is powerful when data, model, evaluation, and deployment all match the real task.
- It is risky when the model output is trusted without validation.

A Short History of AI



- **1950s–1960s**: AI was born around reasoning, search, symbolic logic, and early neural networks.
- **1970s–1980s**: expert systems encoded human rules, but were brittle and expensive to maintain.
- **1990s–2010s**: data, computing power, and machine learning made AI effective in games, vision, speech, and prediction.
- **2020s**: foundation models and agents make AI more general, interactive, and useful across tasks.

Milestone: IBM Watson on Jeopardy!

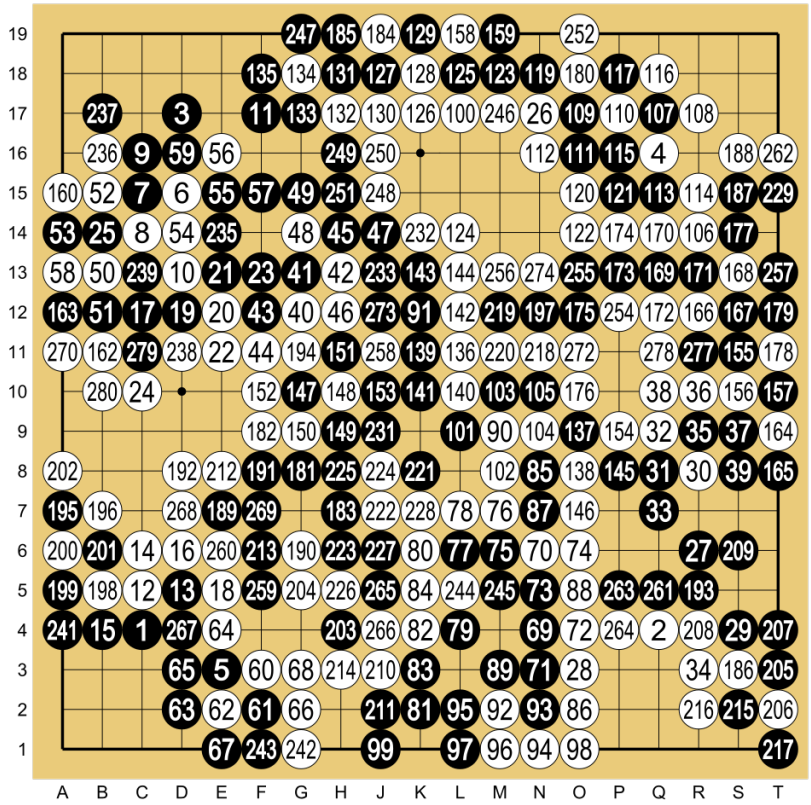
- **Jeopardy!** is a famous American quiz show where clues often involve wordplay, facts, and fast recall.
- In 2011, IBM Watson defeated two champion human players on Jeopardy!.
- This was a milestone for natural language processing, information retrieval, and question answering.

Source: Wikimedia Commons, IBM Watson w Jeopardy, CC BY-SA 2.0

Milestone: AlphaGo and Strategic Decision Making

- Games are useful AI testbeds because states, actions, rules, and rewards are clear.
- AlphaGo defeated Lee Sedol in 2016, showing the power of deep learning and reinforcement learning.
- The lesson is broader than Go: AI can learn strategies in very large decision spaces.

Source: Wikimedia Commons, Lee Sedol vs AlphaGo Game 5, CC BY-SA 4.0

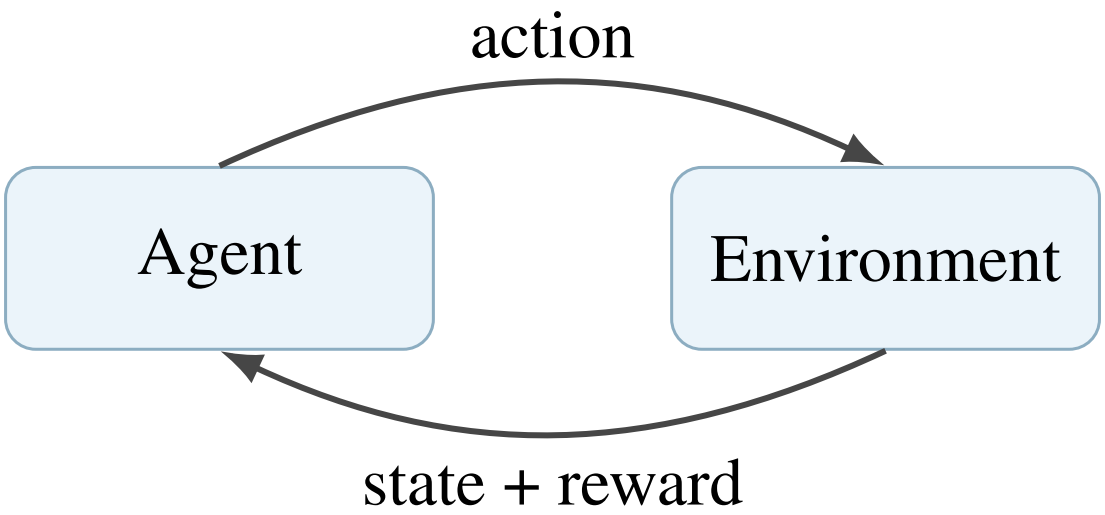


Lee Sedol (B) vs AlphaGo (W) - Game 5

(118) at 107 161 at 25 (230) at (148) (234) at 53 (240) at (200) 253 at (184) 271 at 25 275 at (168) (276) at 151

Reinforcement Learning: Learning by Trial and Error

- An agent observes the state of an environment, takes an action, and receives a reward.
- By repeatedly trying actions, it learns a policy: what action should I take in each situation?
- Engineering analogies include traffic signal control, robot control, fleet dispatching, and dynamic traffic management.



Computer Vision: Teaching Machines to See

- Classification: what is in the image?
- Detection: where are objects?
- Segmentation: which pixels belong to each object?
- Engineering use: inspection, mapping, monitoring, and autonomous driving.

Autonomous Driving: Real Systems on the Road

- Different companies take different technical paths: robotaxi fleets, advanced driver assistance, and purpose-built autonomous vehicles.
- The shared challenge is the same: perceive the world, predict other road users, plan safely, and control the vehicle.

Waymo robotaxi

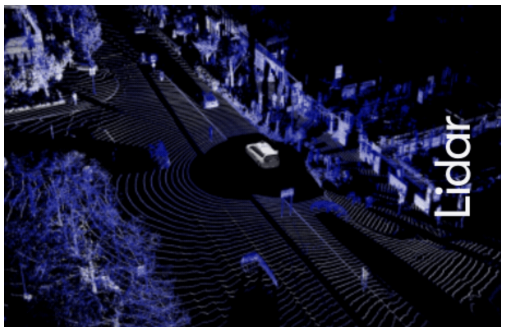
Tesla Autopilot / ADAS

Baidu Apollo Go

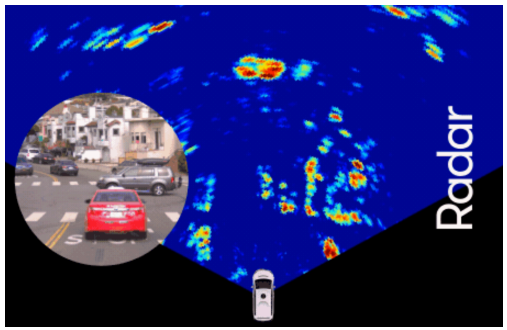
Source: Wikimedia Commons vehicle images

Autonomous Driving Needs Multiple Sensors

Cameras
lanes, signs, lights



LiDAR
3D shape and depth



Radar
speed and bad weather

Sensor fusion combines complementary information and provides redundancy for safety.

Source: Waymo Driver Handbook

Large Language Models: From Text to Reasoning Agents

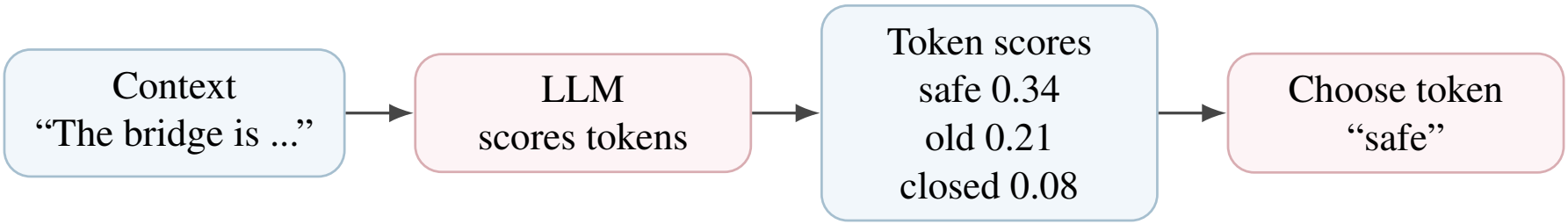
- LLMs started as text prediction models, but quickly became instruction-following, multimodal, and tool-using systems.
- The main change is clear: models are moving from "write a paragraph" to "reason, use tools, and complete tasks."

- New models can work with text, images, audio, code, long documents, and external tools.
- They are powerful, but we still need to check their answers because hallucinations and unstable reasoning remain real risks.

Source: OpenAI model releases; Stanford AI Index 2026

The Core Idea: Next Token Prediction

- An LLM reads the context and predicts the most likely **next token**.
- A token can be a word, part of a word, punctuation, or a code symbol.
- The model repeats this step many times, so a full answer is generated token by token.



It is not guaranteeing a fact at every step. It is choosing a likely continuation based on the context and patterns learned from data.

Source: See also: visual explanations of next-token prediction in LLM tutorials

Why Hallucinations Happen

- Because generation is based on likelihood, the model can produce text that **looks reasonable but is false**.
- It may fail at tasks that require exact counting, geometry, physical constraints, or verified facts.
- Examples: wrong citations, wrong numerical facts, counting letters in "strawberry", or image models drawing too many fingers.

Source: Why Language Models Hallucinate; strawberry counting discussions

A visual analogy: an output may look reasonable but still break simple rules.

How Intelligent Is AI Getting?

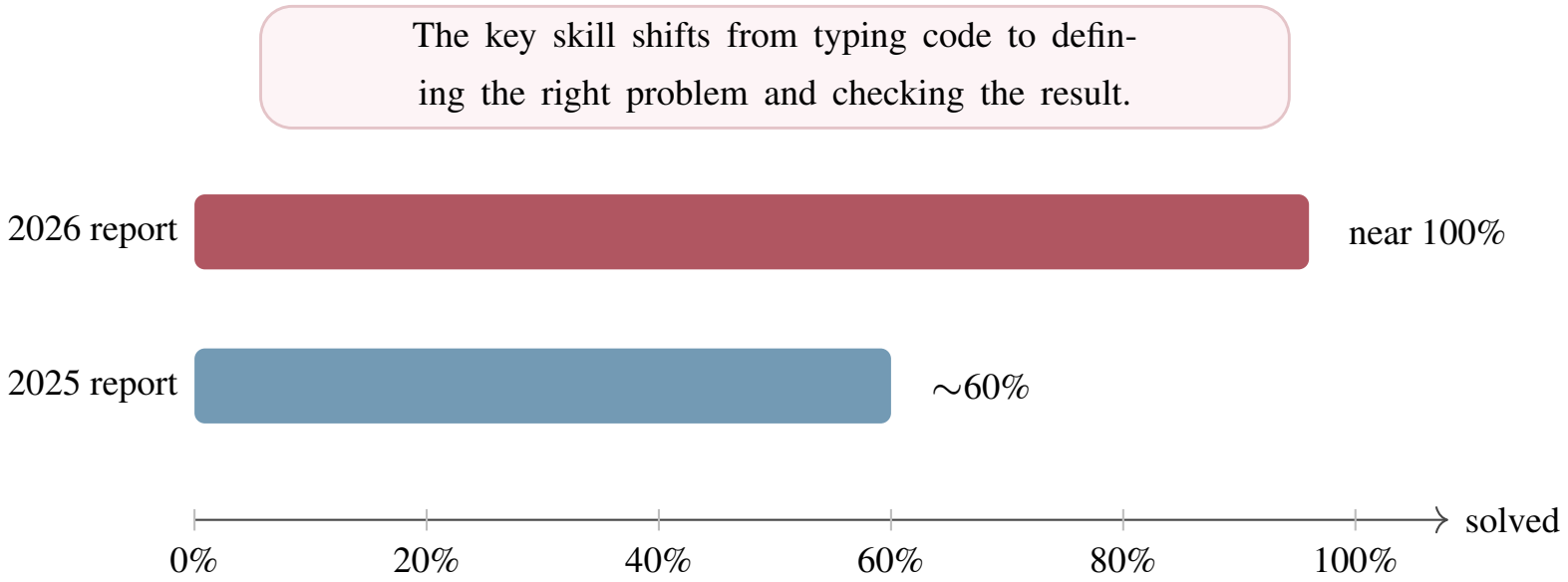
- The 2026 AI Index shows that AI capability is **still improving quickly** and is reaching more people.
- Several top models now reach or pass human-level scores on some science, multimodal, and competition mathematics tasks.
- AI ability is uneven: a model may solve hard math problems but still fail a simple everyday task.

The right question is not "Does AI have IQ?" but "Which parts of human work can AI already do well?"

Source: Stanford AI Index 2026

Coding: AI Can Help Write and Fix Code

- SWE-bench Verified tests whether AI can fix real software issues.
- According to the 2026 AI Index, performance rose from about **60%** to **near 100%** in one year.
- A useful coding workflow is now: describe the task, let AI draft a solution, test it, review it, and then integrate it.

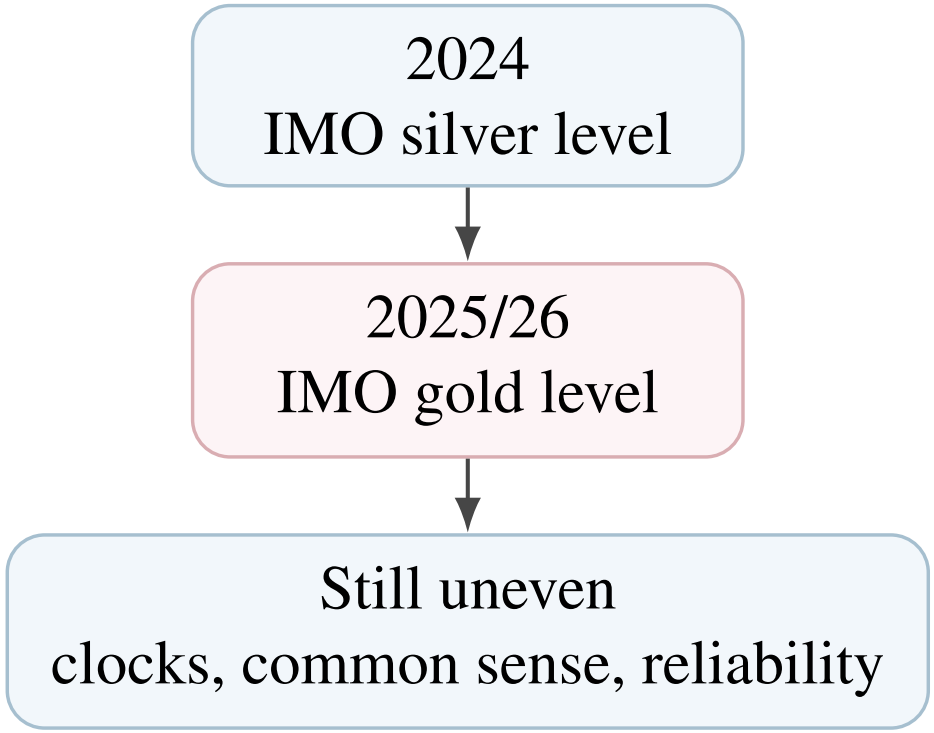


Source: Stanford AI Index 2026

Math: From Silver Medal to Gold Medal

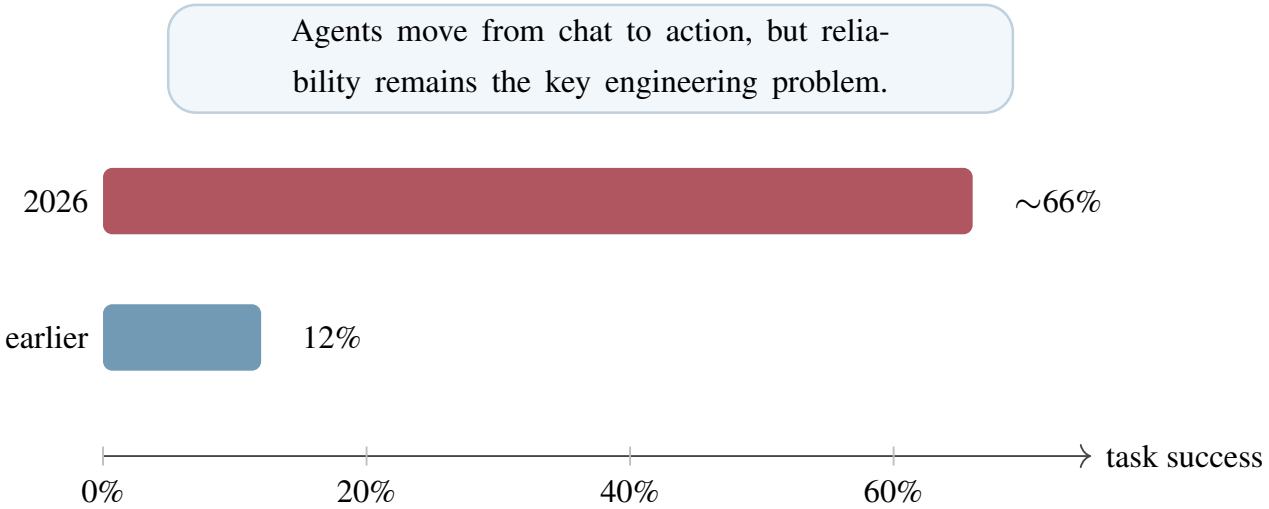
- In 2024, AlphaProof and AlphaGeometry 2 reached silver-medal level at IMO.
- By the 2026 AI Index summary, Gemini Deep Think earned a **gold medal** at IMO.
- Yet the same report also shows uneven ability: the top model reads analog clocks correctly only about **50.1%** of the time.

Source: Google DeepMind 2024; Stanford AI Index 2026



Agents: AI Can Use Software Tools

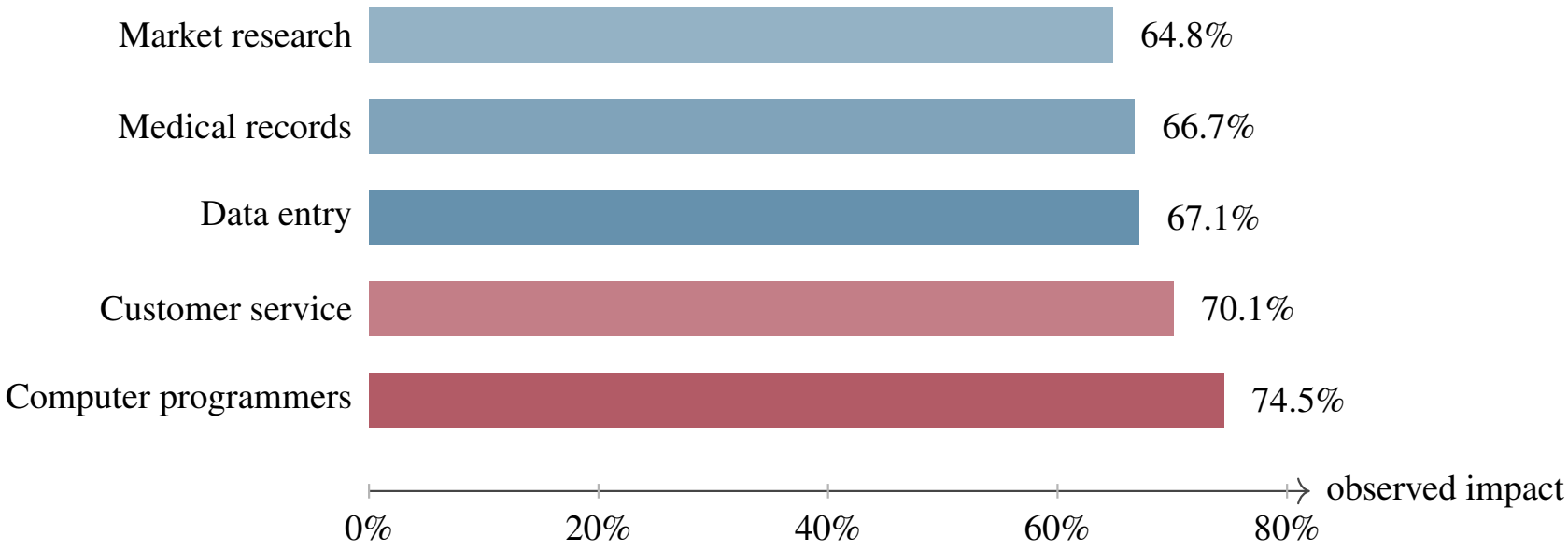
- Agent benchmarks test whether AI can use software, call tools, and finish multi-step computer tasks.
- The 2026 AI Index reports OSWorld agent success rising from **12%** to about **66%**.
- This is a big improvement, but agents still fail about one in three structured attempts.



Source: Stanford AI Index 2026

Which Jobs Are More Affected by AI?

- Studies of real AI usage find strong impact in knowledge and communication work: coding, customer support, data entry, analysis, and documentation.
- Being affected by AI does not mean immediate replacement. It means many tasks can be assisted or partly automated.



Source: Anthropic Economic Index / PayScope summary, 2026; observed AI use in real work

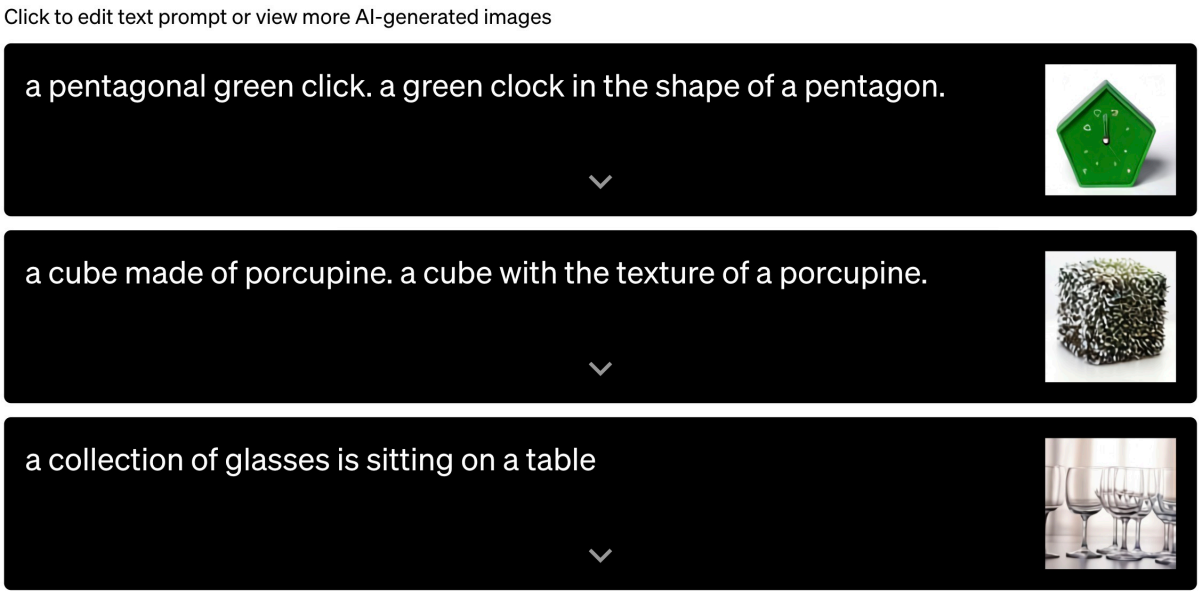
The Takeaway: AI Is Becoming a Work Partner

- AI can already help with writing, coding, debugging, summarizing, mathematical reasoning, data analysis, and design exploration.
- The human role shifts toward asking the right question, judging the answer, checking the result, using domain knowledge, and taking responsibility.
- For engineers, the practical question is not whether AI is "smart", but how to use it safely and effectively on real engineering problems.

This course opens the black box so you can understand, evaluate, and use AI rather than only consume it.

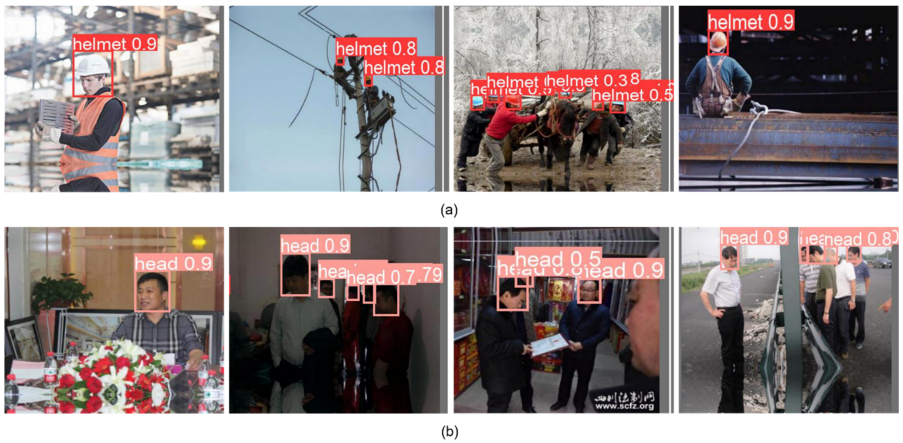
Generative AI: Text, Images, Code, Design

- Generate design concepts and visual examples.
- Draft code, documentation, reports, and emails.
- Summarize drawings, papers, project documents, and meeting notes.
- Support brainstorming, but not replace engineering validation.



AI in Civil Engineering

- AI can turn inspection images, sensor data, and engineering documents into useful evidence for decisions.
- Typical applications include construction safety monitoring, bridge defect detection, crack segmentation, structural health monitoring, and material design.



Safety helmet detection results with bounding boxes

Defect annotation for model training

Source: Hayat and Morgado-Dias, Applied Sciences 2022; Ruggieri et al., Sensors 2026

AI in Transportation Engineering

- Transportation systems generate streaming data from detectors, cameras, GPS traces, transit cards, and connected vehicles.
- AI supports traffic prediction, incident detection, signal control, transit operations, logistics, and autonomous driving.

Source: Wikimedia Commons, NHP and NDOT Traffic Dispatch Center

What Can Go Wrong?

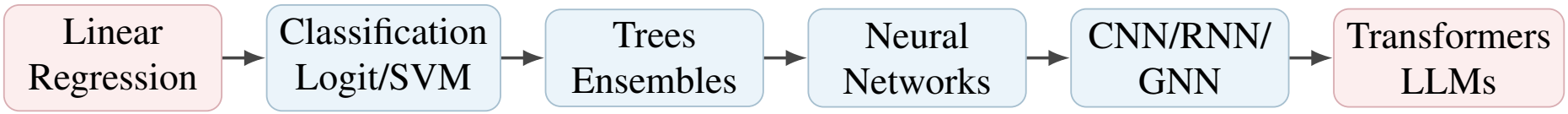
- Hallucination: confident but wrong outputs.
- Bias: training data may encode historical bias.
- Data leakage: future information enters training.
- Poor transfer: a model trained in one city may fail in another.
- Black box: predictions may be difficult to explain.
- Overtrust: automation can hide uncertainty and responsibility.

AI Looks Advanced, But It Is Not Mysterious

Modern AI can be powerful, but the basic ideas are learnable.

data $\rightarrow$ features $\rightarrow$ model $\rightarrow$ prediction $\rightarrow$ decision

Our Learning Path



We start simple so that advanced AI becomes understandable, not mysterious.

What You Should Be Able to Do

- Understand AI papers: identify the problem, data, model, metric, and insight.
- Build models: train and evaluate regression, classification, and neural network models.
- Use AI tools: use GPT/Gemini/Copilot productively while checking the logic.
- Apply to engineering: connect model outputs to civil and transportation decisions.

Takeaway Before We Move to Data

AI is everywhere, but it is not magic.

This course will help you understand AI step by step,

then use it to solve real engineering problems.

Outline

1 Introduction to AI

2 Useful Data Platforms

3 Fundamentals of AI
- What Is Machine Learning
- Regression and Classification
- Common AI Models
- Overfitting
- Optimization Algorithms

Kaggle

- Kaggle[1] offers a no-setup, customizable, programming environment and various datasets for you

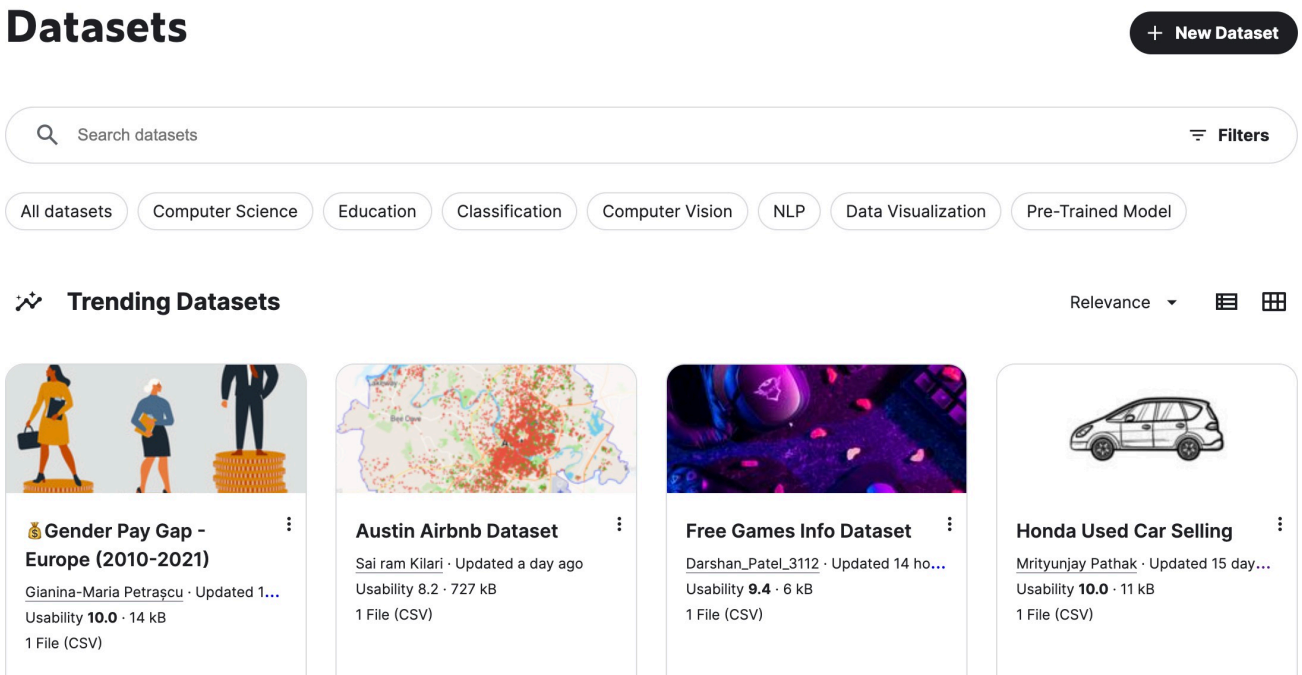


Figure: Kaggle

[1] In this slideshow, you can click the figure name to get access to the targeted website

Machine Learning Dataset from UC Irvine

- UCI machine learning repository is a well-known database for standard testing, where you can easily get data that fits your specific requirements

Figure: UCI Machine Learning Repository

Registry of Open Data on AWS

- Amazon Web Services (AWS) is the most comprehensive and widely used cloud platform in the world, and it also offers a public registry to help people discover and share datasets that are available via AWS resources

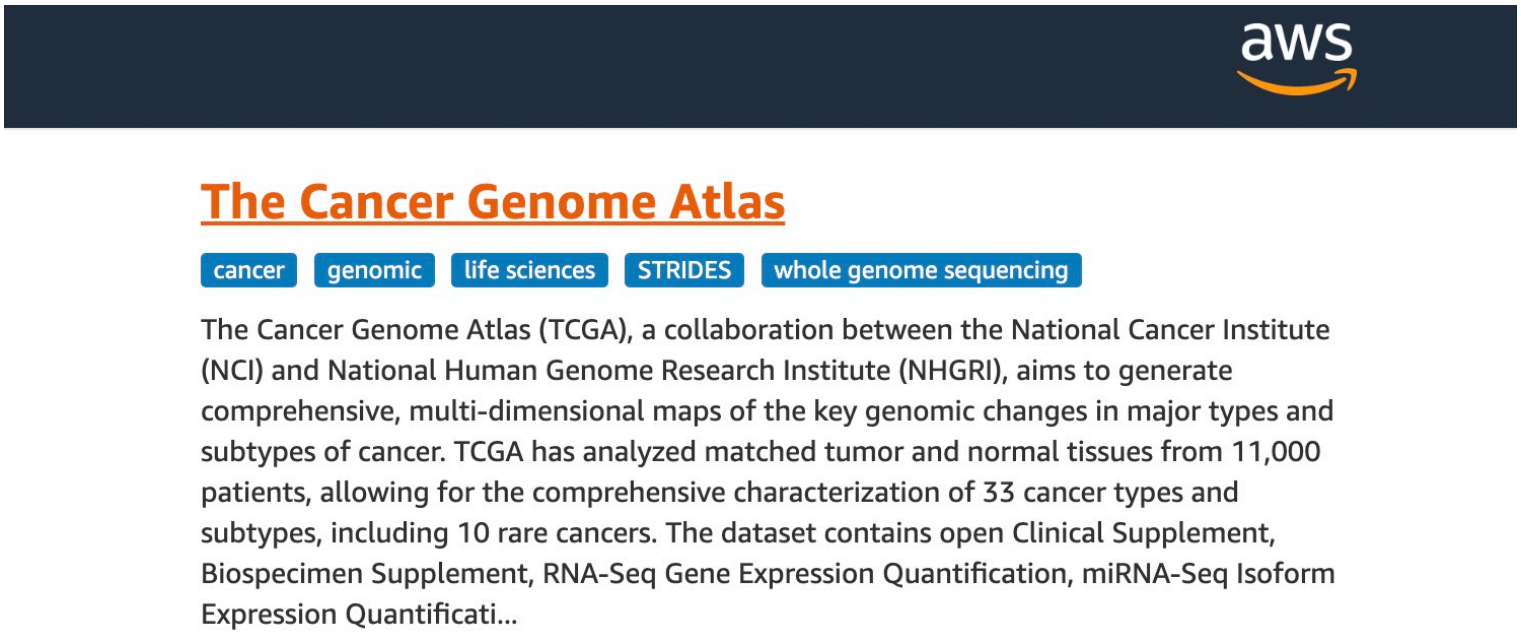


Figure: Registry of Open Data on AWS

Microsoft Research Open Data

- Microsoft research team provides a series of open datasets related to cutting-edge scientific research

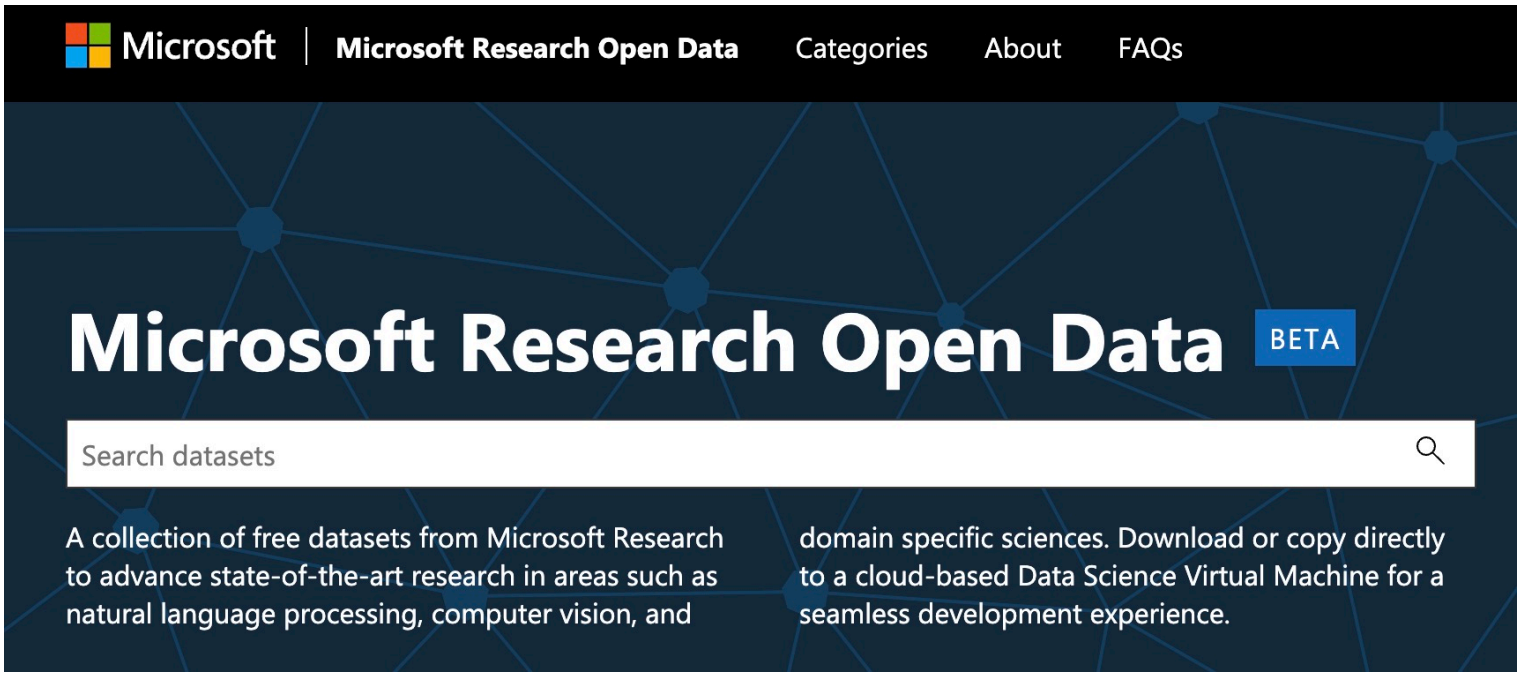


Figure: Microsoft Research Open Data

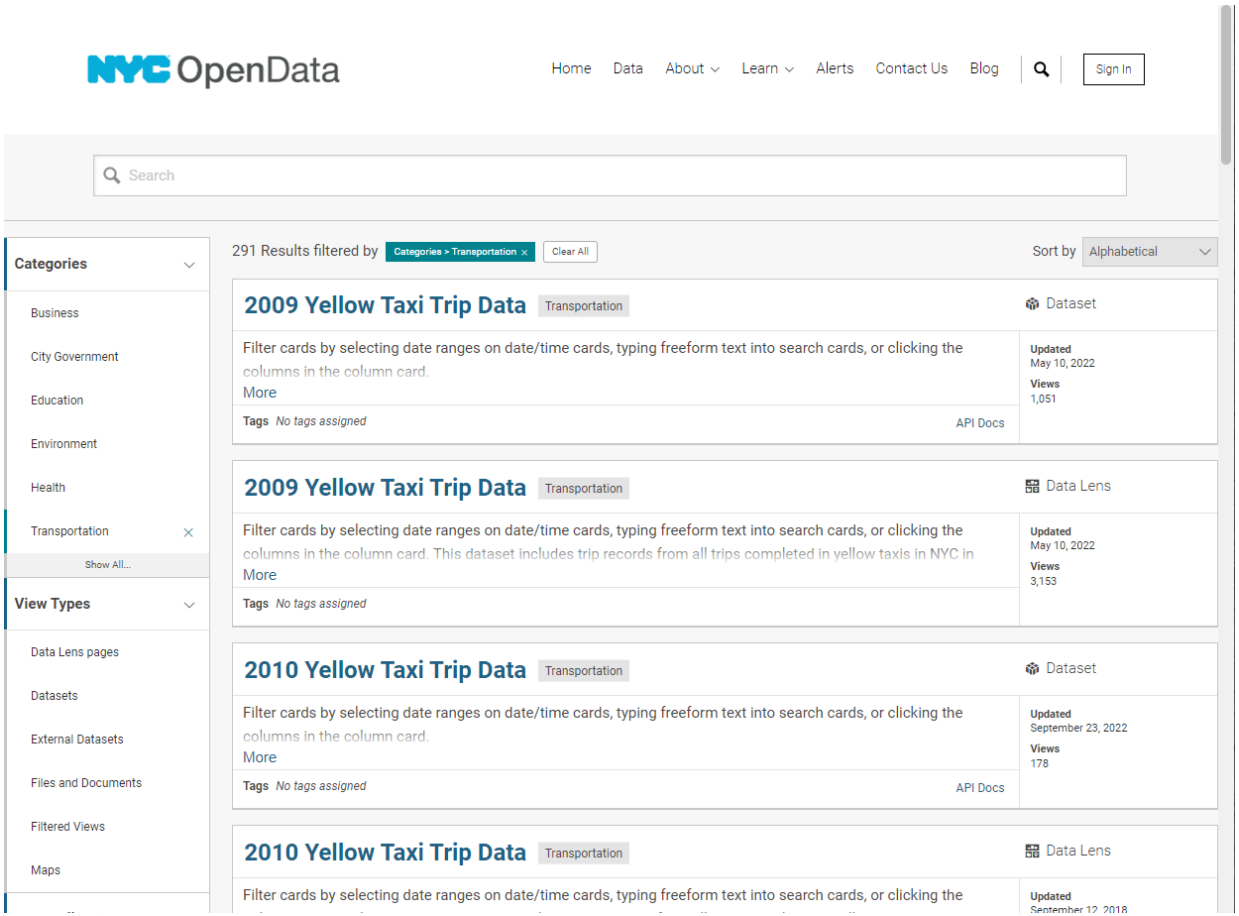
Other open-sourced Data

Researchers can retreat data from a list of open-sourced data platforms, including but not limited to:

- NYC open data (https://opendata.cityofnewyork.us/)
- Hong Kong open data (https://data.gov.hk/sc/)
- Caltrans PeMs (https://pems.dot.ca.gov/)
- Didi GAIA Open Dataset
- RITIS (https://ritis.org/intro)
- TIMS (https://tims.berkeley.edu/)
- SafeGraph (https://www.safegraph.com/)
- Highways England Traffic Flow Data (https://webtris.highwaysengland.co.uk/)
- BART Ridership Data (https://www.bart.gov/about/reports/ridership)
- Bureau of Transportation Statistics (https://www.bts.gov/)

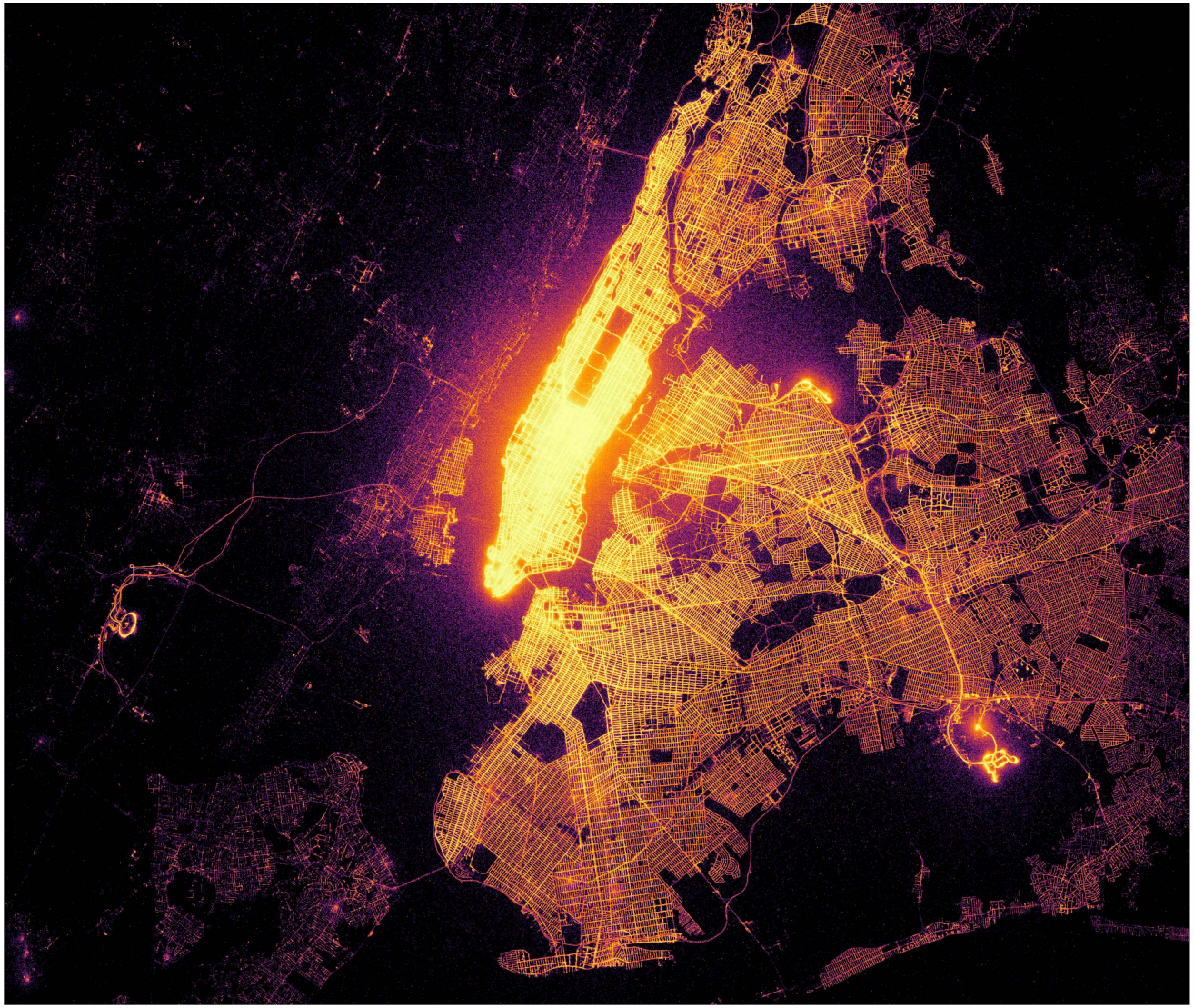
NYC Open Data

New York City open data platform (https://opendata.cityofnewyork.us/) offers many types of transportation data, including Real-Time Traffic Speed Data, Traffic Volume Counts, Bicycle Routes, etc., to the public.

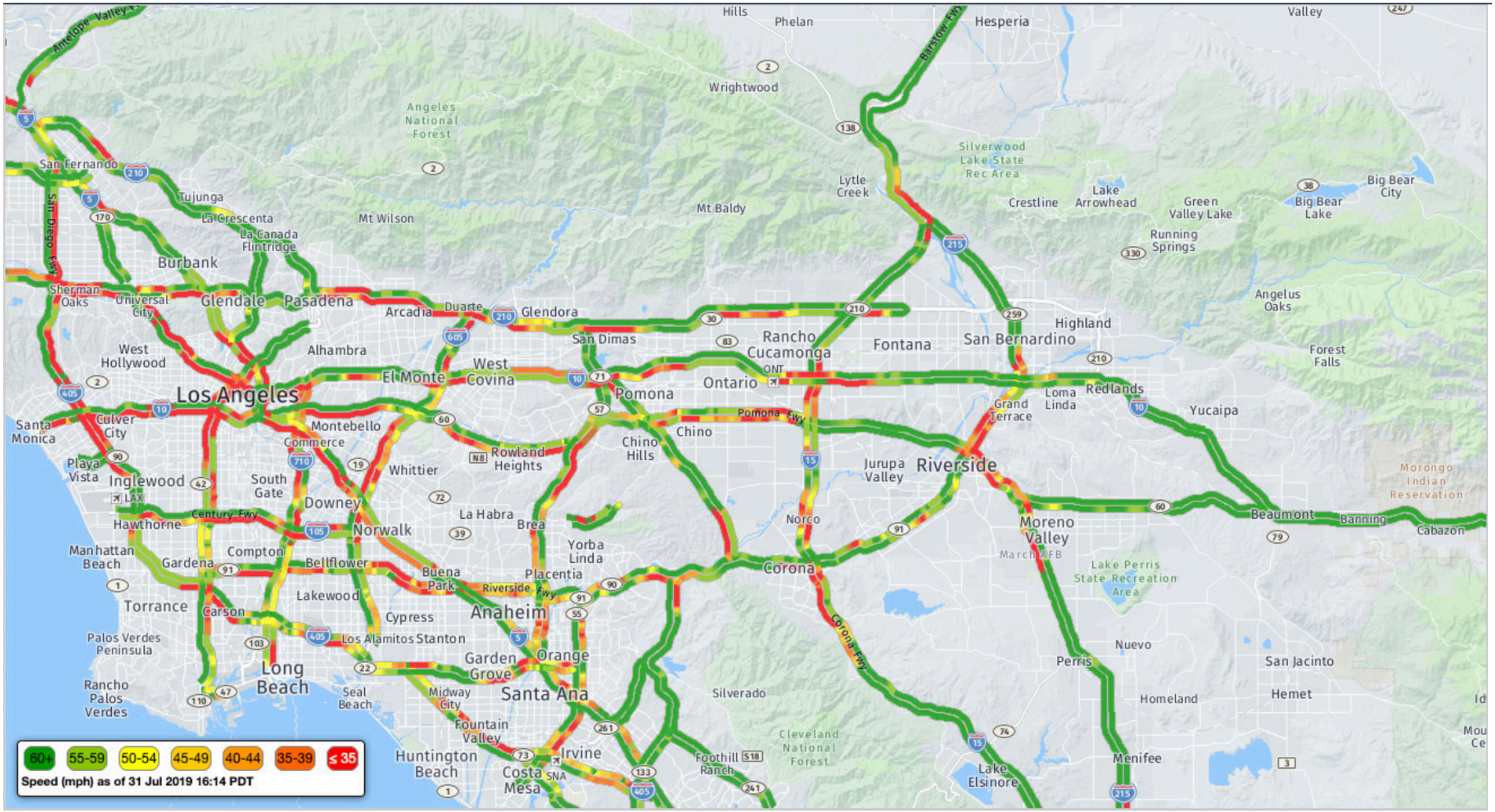


NYC Network

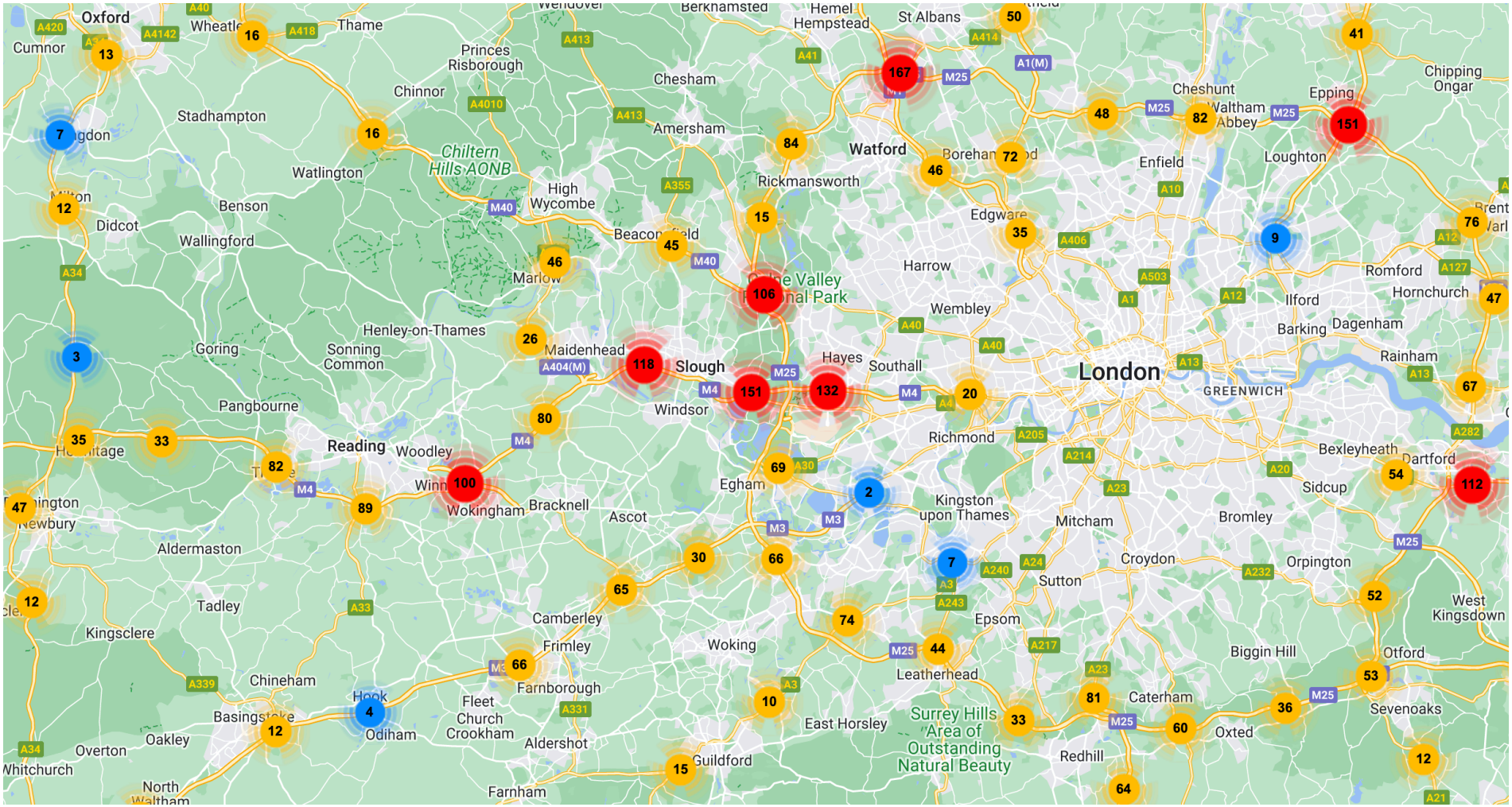
NYC Taxi Demand Intensity



Caltrans PeMS Data

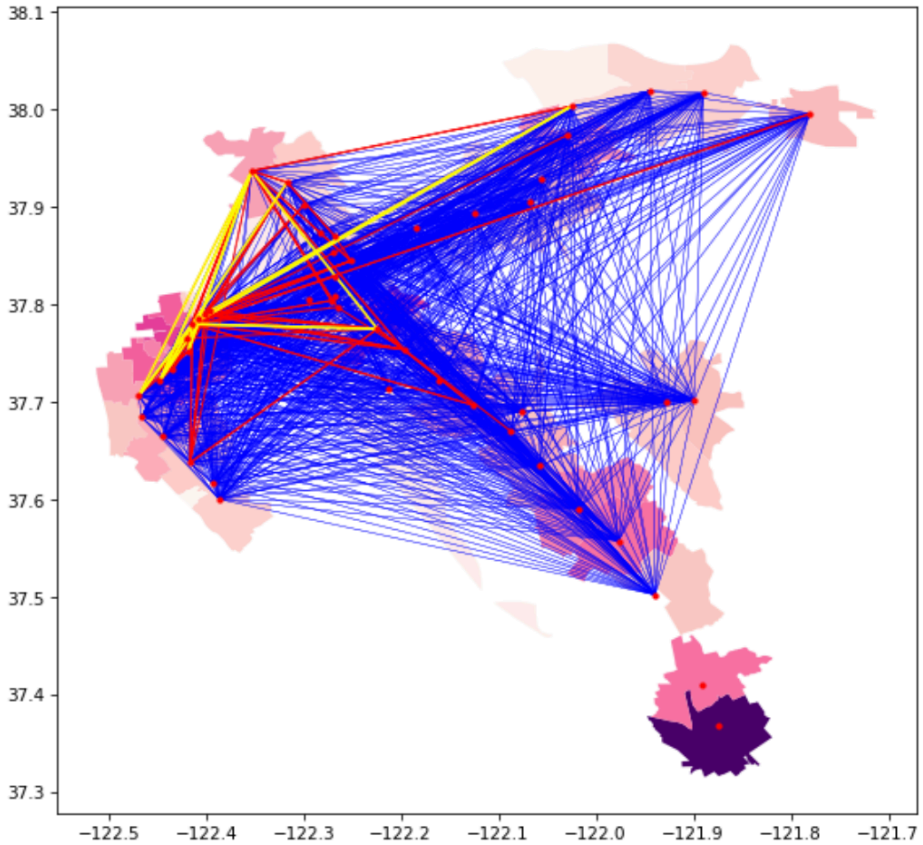


Highways England Traffic Flow Data



BART Ridership Data

The Bay Area Rapid Transit system provides ridership reports and data. Yellow lines represent high ridership (OD pairs), red lines represent medium ridership, and blue lines represent low ridership.



Other Useful Data I

The Weather Data Environment (WxDE)

- `https://wxde.fhwa.dot.gov/`
- The Weather Data Environment (WxDE) is a research project that collects and shares transportation-related weather data with a particular focus on weather data related to connected vehicle applications.

ITS Knowledge Resources

- `http://www.itsknowledgeresources.its.dot.gov/`
- ITS provide a proven set of strategies for advancing transportation safety, mobility, and environmental sustainability by integrating communication and information technology applications into the management and operation of the transportation system across all modes.

Other Useful Data II

Archived Data User Service (ADUS)

- `https://www.fhwa.dot.gov/policyinformation/travel/adus.cfm`
- ADUS provides the National ITS Architecture with the requirements for archiving and re-use of data collected for ITS operations. This FHWA website provides a wealth of information related to ITS and traffic related data.

Data.gov

- `https://www.data.gov/`
- The purpose of Data.gov is to increase public access to high value, machine readable datasets generated by the Executive Branch of the Federal Government.

Other Useful Data III

US DOT ITS JPO

- `http://www.its.dot.gov/`
- The US Department of Transportation (US DOT) ITS Joint Program Office (JPO) home page.

National Transit Database (NTD)

- `http://www.ntdprogram.gov/`
- NTD is the Federal Transit Administration's (FTA) national database of statistics for the transit industry. The NTD is composed of data reported by more than 600 transit agencies across the US. The data are then analyzed and compiled into reports published by FTA and made available to the public on the NTD Program website.

Other Useful Data IV

Traffic Volume Trends

- `http://www.fhwa.dot.gov/policyinformation/travel_monitoring/tvt.cfm`
- Traffic Volume Trends is a monthly report based on hourly traffic count data reported by the States. These data are collected at approximately 4,000 continuous traffic counting locations nationwide in US.

TranStats - Multimodal Transportation Database

- `http://www.transtats.bts.gov/`
- TranStats provides one-stop shopping for intermodal transportation data. It has a searchable index of more than 100 databases covering every mode of transportation, plus social and demographic data commonly used in transportation research.

Outline

1 Introduction to AI

2 Useful Data Platforms

3 Fundamentals of AI
- What Is Machine Learning
- Regression and Classification
- Common AI Models
- Overfitting
- Optimization Algorithms

Outline

1 Introduction to AI

2 Useful Data Platforms

3 Fundamentals of AI
- What Is Machine Learning
- Regression and Classification
- Common AI Models
- Overfitting
- Optimization Algorithms

What is Machine Learning?

- Artificial Intelligence (AI) is intelligence demonstrated by machines, as opposed to natural intelligence displayed by animals including humans.
- Machine Learning is study of computer algorithms that can improve automatically through experience and by the use of data. It is regarded as a part of AI
- Deep Learning is a broader family of machine learning methods based on artificial neural networks with deeper structure and stronger ability to learn from big data
- Data Mining is the application of machine learning methods to large databases.

Applications of Machine Learning

Machine learning

- Supervised learning: Learn a general rule that maps inputs (features) to outputs (label) for making predictions.
  - Classification
  - Regression
- Unsupervised learning: Discover some hidden patterns from the input data itself. No label is given for learning
- Reinforcement learning: A computer program interacts with a dynamic environment in which it must perform a certain goal
- Supervised and unsupervised learning is more related to predictive causal analytics and can also support prescriptive analytics, while reinforcement learning directly provide solutions to prescriptive analytics

Data Samples

Features and labels

- Features (Independent variables): we have a set of N samples $\{\mathbf{x}_1, \mathbf{x}_2, ..., \mathbf{x}_N\}$, where $\mathbf{x}_n, \forall n \in 1, 2, ..., N$ is a vector with length D, indicating there is D features in each sample. The nth sample $\mathbf{x}_n = [x_{n1}, x_{n2}, ..., x_{nD}]$, while all samples $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, ..., \mathbf{x}_N]^T$
- Labels (Dependent variables): for each sample $n \in 1, 2, ..., N$, we have a target value or label y_n associated with the sample vector $\mathbf{x}_n$. All samples $\mathbf{Y} = [y_1, y_2, ..., y_n]^T$
- As for supervised learning, the objective is to find out a function $g(\mathbf{x})$ that takes a sample of features $\mathbf{x}_n$ ($\mathbf{x}$ for short) as input and generates an output $\hat{y} = g(\mathbf{x})$ for predicting the corresponding label y. The function $g(\mathbf{x})$ can also be expressed as $g(\mathbf{x}|\mathbf{w})$ parameterized by a set of parameters $\mathbf{w}$.

Data Samples

Features and labels example

An example dataset with four samples is shown below. Each sample in this dataset has three features and a label.

Sample	Feature 1	Feature 2	Feature 3	Label
1	11	12	13	32
2	21	22	23	64
3	31	32	33	128
4	41	42	43	256

Take the first sample as an example, $n = 1$, from the table above we know that the three features $x_{n1} = 11, x_{n2} = 12, x_{n3} = 13$ and label $y_1 = 32$.
In a vector form $\mathbf{x}_n = [x_{n1}, x_{n2}, x_{n3}] = [11, 12, 13]$.

Data Samples

Features and labels example

Sample	Feature 1	Feature 2	Feature 3	Label
1	11	12	13	32
2	21	22	23	64
3	31	32	33	128
4	41	42	43	256

All samples' features $\mathbf{X}$ and labels $\mathbf{Y}$ can be expressed as

$$\begin{matrix} x_{n1} & x_{n2} & x_{n3} \\ \downarrow & \downarrow & \downarrow \end{matrix}$$

$$\mathbf{X} = \begin{bmatrix} 11 & 12 & 13 \\ 21 & 22 & 23 \\ 31 & 32 & 33 \\ 41 & 42 & 43 \end{bmatrix} = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \mathbf{x}_3 \\ \mathbf{x}_4 \end{bmatrix}, \quad \mathbf{Y} = \begin{bmatrix} 32 \\ 64 \\ 128 \\ 256 \end{bmatrix} \begin{matrix} \rightarrow & y_1 \\ \rightarrow & y_2 \\ \rightarrow & y_3 \\ \rightarrow & y_4 \end{matrix}$$

Data Samples

Training and Testing

- Training data (set): a sample of data with features and labels $(\mathbf{x}_n, y_n), \forall n \in 1, 2, ..., N$ available to learn parameters $\boldsymbol{w}$ that can help $g(\mathbf{x}|\mathbf{w})$ to predict y as accurately as possible.
- Test data (set): once the model is trained (i.e., parameters $\mathbf{w}$ are determined) on the basis of training data, the model can predict y based on the features of new samples, which are said to comprise a *test set*. The ability to correctly predict new samples' features is known as generalization.
- Validation set: a part of training data extracted to tune the hyperparameters (i.e. the architecture) of a model and used for early stopping in the training of neural networks.

Three Types of Machine Learning

Supervised learning	Unsupervised learning	Reinforcement learning
Learn from examples with labels: $\mathbf{x} \rightarrow y$. Typical tasks are regression and classification.	Find hidden patterns without labels, such as clusters, lower-dimensional structure, or anomalies.	Learn actions by trial and error through rewards, useful for games, robots, and traffic control.

Key message

Most models in this course can be understood as learning a mapping from inputs to outputs, or learning useful patterns from data.

Unsupervised Learning

- No label is given, namely, only features $\mathbf{x}$ are available.
- Learning "what normally happens" and "interesting patterns" in the data.
- Much less well-defined than supervised learning with no obvious error measure.
- Some applications:
  - Density estimation
  - Clustering: grouping similar samples, e.g., K-means clustering, expectation-maximization algorithm.
  - Dimensionality reduction/data compression: to discover latent factors that can represent the majority of the information of the original data, e.g., principal component analysis (PCA).

Reinforcement Learning

- Finding suitable actions to take in a given situation in order to maximize a reward in long run.
- There is a sequence of states and actions in which the learning algorithm is interacting with its environment.
- The current action not only affects the immediate reward but also has an impact on the reward at all subsequent time steps.
- The algorithm learn a policy (a sequence of actions) by a process of trail and error.
- Some applications:
  - Playing games, playing chess (e.g, Alpha Go)
  - Robot navigation in search of goal location
  - Traffic signal control
  - Order dispatching for ride-hailing services (Uber, Didi)

Outline

1 Introduction to AI

2 Useful Data Platforms

3 Fundamentals of AI
- What Is Machine Learning
- Regression and Classification
- Common AI Models
- Overfitting
- Optimization Algorithms

Regression

Training Phase

- A sample of data $\mathcal{X}$ (training data) with features and labels $(\mathbf{x}_n, y_n), \forall n \in 1, 2, ..., N$ are collected, where y is a continuous value (e.g., any real value in the range of $[-\infty, \infty]$).
- Use the training set to find out the set of parameters or weights $\hat{\mathbf{w}}$ that makes $g(\mathbf{x}|\mathbf{w})$ predict y as accurately as possible.

Testing Phase

- For any new testing samples with features $\mathbf{x}_n$, we can use the trained model to estimate the corresponding $\hat{y}_n = g(\mathbf{x}_n|\hat{\mathbf{w}})$, which is denoted as predicted label.
- The actual label y_n corresponding to the sample $\mathbf{x}_n$ is called the true label. Prediction errors can be measured by the difference between predicted label $\hat{y}_n$ and true label y_n.

Error Minimization I

In training phase, how to find out the optimal parameters?

- This can be done by minimizing an error function that measures the misfit between the function $g(\mathbf{x}|\mathbf{w})$, for any given value of $\mathbf{w}$, and the true labels y. One simple error function is given by the sum of the squares of the errors between the predictions $g(\mathbf{x}_n|\mathbf{w})$ for each data point $\mathbf{x}_n$ and the corresponding true labels y_n

$$E(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N}\left[\hat{y}_n - y_n\right]^2 = \frac{1}{2}\sum_{n=1}^{N}\left[g(\mathbf{x}_n|\mathbf{w}) - y_n\right]^2 \tag{1}$$

- where the factor of 1/2 is included for later convenience. So the optimal parameters can be obtained by

$$\hat{\mathbf{w}} = \arg\min_{\mathbf{w}} E(\mathbf{w}) \tag{2}$$

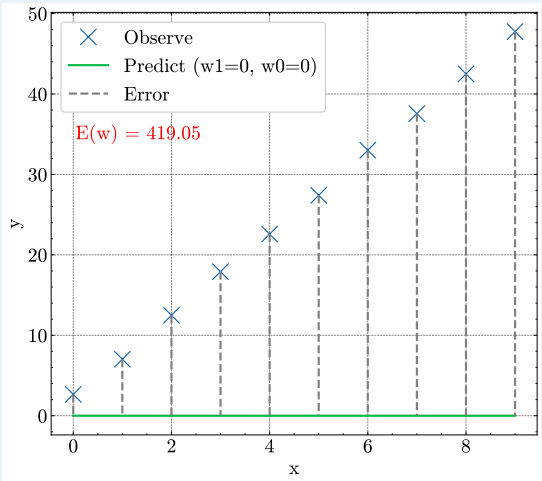
Error Minimization II

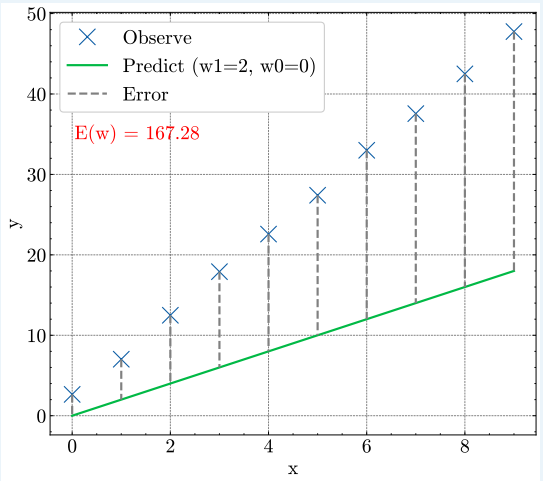
An example

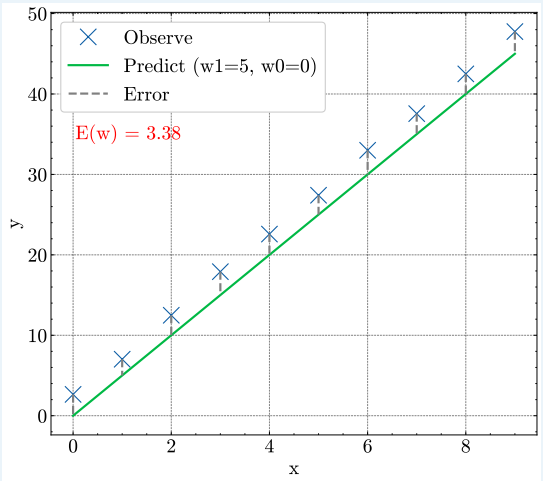
Taking a linear regression with one variable $\hat{y} = w_0 + w_1 x$ as an example, we need to change the values of coefficients w_0 and w_1 to find the lowest error function value $E = \frac{1}{2} \sum_{n=1}^{N} [\hat{y}_n - y_n]^2$.

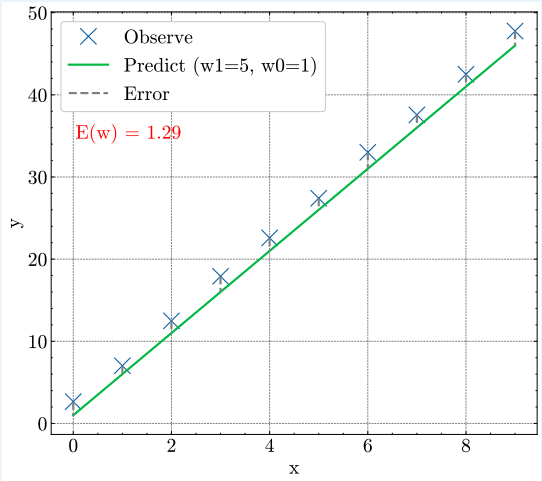
Error Minimization III

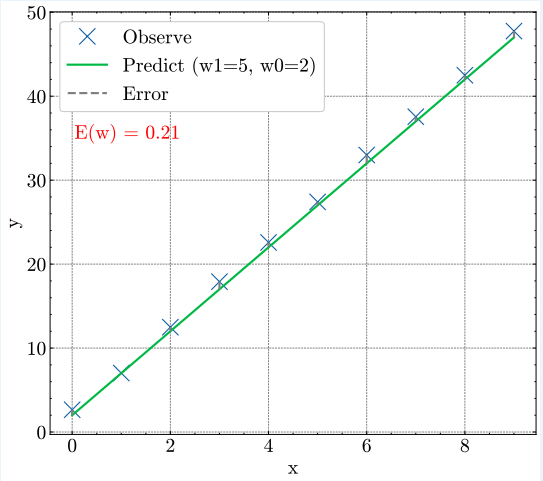
An example

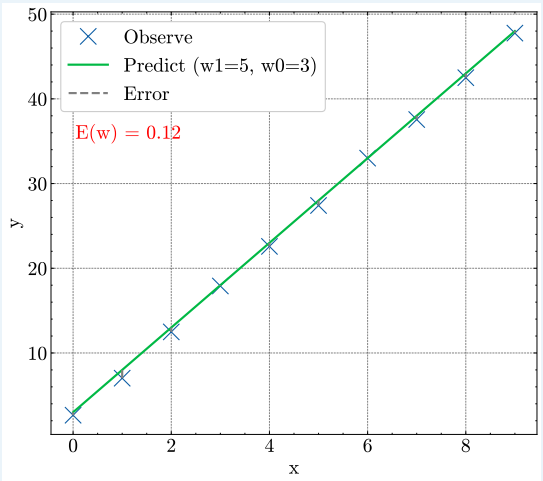












Performance Evaluation for Regression

In testing phase, how to measure prediction accuracy?

Given N samples in the test set, use the model $g(\mathbf{x}_n|\hat{\mathbf{w}})$ to estimate $\hat{y}_n$ for each testing sample with features $\mathbf{x}_n$. Then the performance measures can be:

- Mean squared error (MSE): $\frac{1}{N}\sum_{n=1}^{N}(\hat{y}_n - y_n)^2$
- Root mean squared error (RMSE): $\sqrt{\frac{1}{N}\sum_{n=1}^{N}(\hat{y}_n - y_n)^2}$
- Mean absolute error (MAE): $\frac{1}{N}\sum_{n=1}^{N}|\hat{y}_n - y_n|$
- Mean absolute percentage error (MAPE): $\frac{1}{N}\sum_{n=1}^{N}\left|\frac{\hat{y}_n - y_n}{y_n}\right|$
- Coefficient of determination (R^2): $1 - \frac{\sum_{n=1}^{N}(\hat{y}_n - y_n)^2}{\sum_{n=1}^{N}(y_n - \bar{y})^2}$, where $\bar{y}$ is the mean value of y_n in the test set. $R^2 = 1$ when predicted label values exactly match the observed true labels.

Classification I

- Training set with N samples: $\mathcal{X} = \{(\mathbf{x}_n, y_n)\}_{n=1}^{N}$
- Each sample (e.g., a car) contains a vector of features $\mathbf{x}_n$ and a label y_n which represents the class of the sample (e.g., family cars for positive samples or other cars for negative samples).
  - Feature vector: $\mathbf{x}_n = (x_{n1}, x_{n2})^T$ (only 2 features for simplicity, e.g., x_{n1} = price, x_{n2} = engine power)
  - Class label: $y_n = \begin{cases} 1, & \text{if } \mathbf{x}_n \text{ is a positive sample} \\ 0, & \text{if } \mathbf{x}_n \text{ is a negative sample} \end{cases}$
- Use $\mathcal{X}$ to train a decision rule to classify a sample $\mathbf{x}$ into either a positive or negative sample, based on the discriminant functions $g(\mathbf{x}_n|\mathbf{w})$, for example,

$$\hat{y}_n = \begin{cases} 1, & \text{if } g(\mathbf{x}_n|\mathbf{w}) > 0.5 \\ 0, & \text{if } g(\mathbf{x}_n|\mathbf{w}) < 0.5 \end{cases} \tag{3}$$

Classification II

Testing Phase

- For any new testing samples with features $\mathbf{x}_n$, we can use the trained model to estimate the discriminant function $g(\mathbf{x}_n|\hat{\mathbf{w}})$.
- Given a threshold τ, we can classify a sample $\mathbf{x}_n$ into a positive sample, i.e., $\hat{y}_n = 1$, if $g(\mathbf{x}_n|\hat{\mathbf{w}}) > \tau$, and into a negative sample , i.e., $\hat{y}_n = 0$, if $g(\mathbf{x}_n|\hat{\mathbf{w}}) < \tau$.
- The actual label y_n corresponding to the sample $\mathbf{x}_n$ is called the true label, while $\hat{y}_n$ is the predicted label. Prediction errors can be measured by the difference between predicted label $\hat{y}_n$ and true label y_n.

How to validate accuracy?

- How can we check whether the predicted labels are accurate and reliable on unseen test data?

Performance Evaluation for Classification I

- True positive (TP): true label is positive $y_n = 1$, and estimated label is correct $\hat{y}_n = 1$. This case is referred to as "hit".
- True negative (TN): true label is negative $y_n = 0$ (sometimes we use $y_n = -1$), and estimated label is correct $\hat{y}_n = 0$.
- False positive (FP): true label is negative $y_n = 0$, and estimated label is wrong (positive) $\hat{y}_n = 1$. This case is referred to as "type I error".
- False negative (FN): true label is positive $y_n = 1$, and estimated label is wrong (negative) $\hat{y}_n = 0$. This case is referred to as "type II error".
- True positive rate (TPR), sensitivity, recall, or hit rate, indicating the accuracy when the true label is positive:

$$\text{TPR} = \frac{TP}{P} = \frac{TP}{TP + FN} \tag{4}$$

- False positive rate (FPR) indicating the inaccuracy when the true label is negative:

$$\text{FPR} = \frac{FP}{N} = \frac{FP}{TN + FP} \tag{5}$$

Performance Evaluation for Classification II

- Specificity or true negative rate (TNR), indicating the accuracy when the true label is negative:

$$\text{Specificity} = \frac{TN}{N} = \frac{TN}{TN + FP} = 1 - \text{FPR} \tag{6}$$

- Precision, indicating the accuracy when the predicted label is positive:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{7}$$

- Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{8}$$

Confusion Matrix

Sources: [20][21][

		Predicted condition		
	Total population = P + N	**Positive (PP)**	**Negative (PN)**	Informedness, bookmaker informedness (BM) = TPR + TNR − 1
Actual condition	**Positive (P)**	**True positive (TP),** hit	**False negative (FN),** type II error, miss, underestimation	True positive rate (TPR), recall, sensitivity (SEN), probability of detection, hit rate, power = $\frac{TP}{P}$ = 1 − FNR
	Negative (N)	**False positive (FP),** type I error, false alarm, overestimation	**True negative (TN),** correct rejection	False positive rate (FPR), probability of false alarm, fall-out = $\frac{FP}{N}$ = 1 − TNR
	Prevalence = $\frac{P}{P+N}$	Positive predictive value (PPV), precision = $\frac{TP}{PP}$ = 1 − FDR	False omission rate (FOR) = $\frac{FN}{PN}$ = 1 − NPV	Positive likelihood ratio (LR+) = $\frac{TPR}{FPR}$

ROC curve

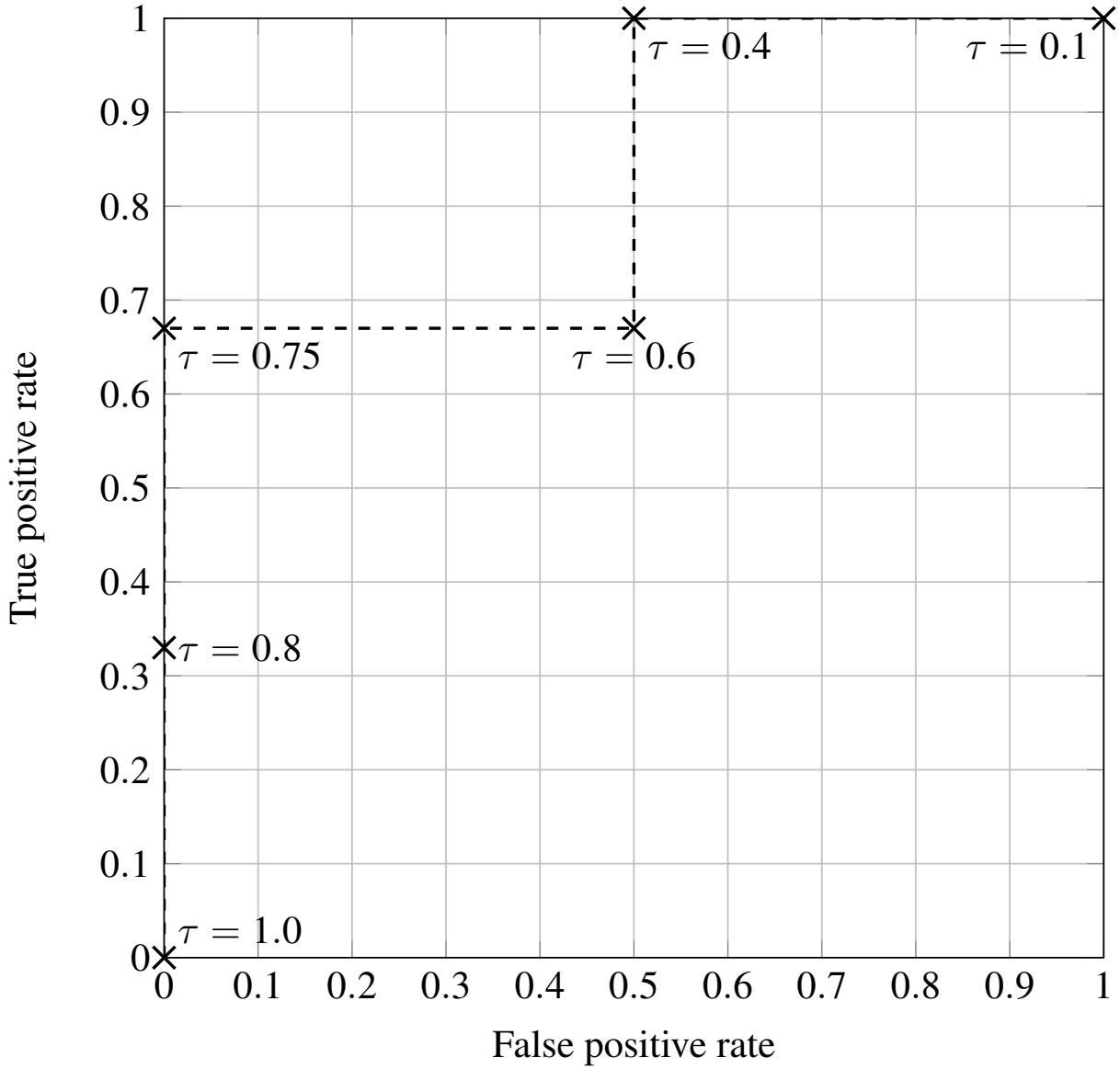
- A receiver operating characteristic curve, or ROC curve, is a graphical plot that illustrates the diagnostic ability of a binary classifier system as its discrimination threshold is varied.
- The ROC curve is created by plotting the true positive rate (TPR) against the false positive rate (FPR) at various threshold settings.

						TPR	FPR
True label	1	1	0	1	0		
Estimated probability	0.8	0.75	0.6	0.4	0.1		
Estimated label ($\tau = 0.5$)	1	1	1	0	0	$2/3$	$1/2$
Estimated label ($\tau = 0.7$)	1	1	0	0	0	$2/3$	0

ROC curve example I

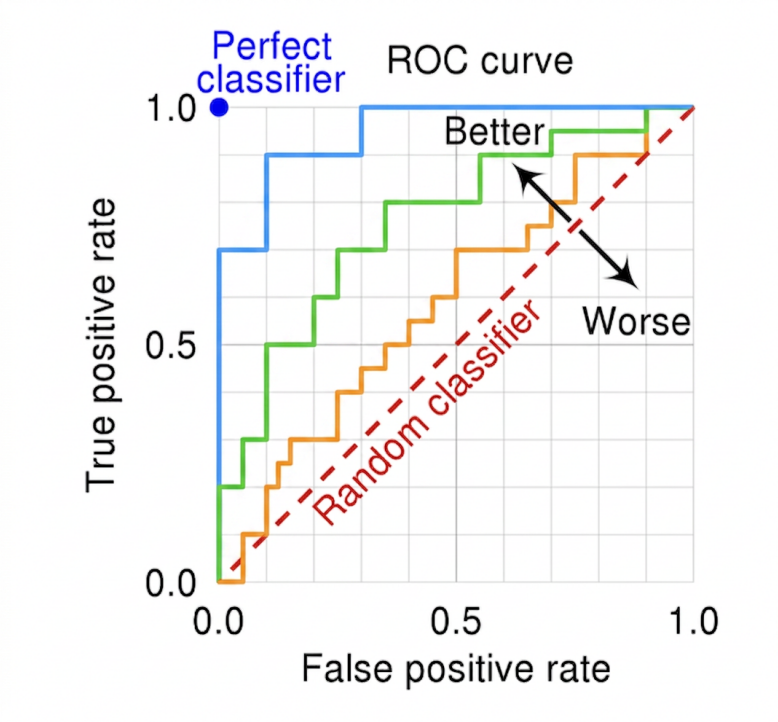
						TPR	FPR
True label	1	1	0	1	0		
Estimated probability	0.8	0.75	0.6	0.4	0.1		
Estimated label ($\tau = 1.0$)	0	0	0	0	0	0	0
Estimated label ($\tau = 0.8$)	1	0	0	0	0	$1/3$	0
Estimated label ($\tau = 0.75$)	1	1	0	0	0	$2/3$	0
Estimated label ($\tau = 0.6$)	1	1	1	0	0	$2/3$	0.5
Estimated label ($\tau = 0.4$)	1	1	1	1	0	1	0.5
Estimated label ($\tau = 0.1$)	1	1	1	1	1	1	1

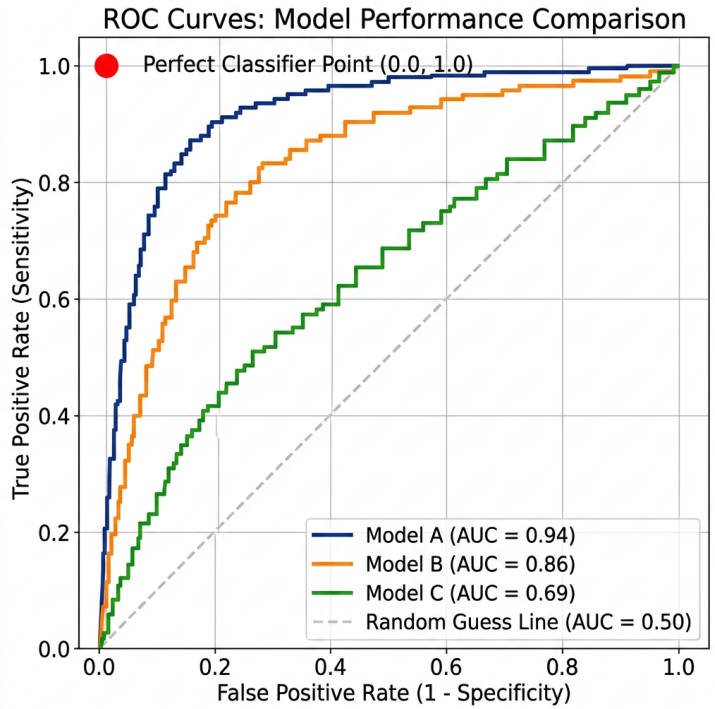
ROC curve example II



ROC Space, ROC Curve, and AUC

- ROC plots true positive rate against false positive rate under different thresholds.
- The closer the curve is to the upper-left corner, the better the classifier.
- AUC summarizes the whole curve: larger AUC means better classification ability.





Dimensions of Supervised Learning

- Thee dimensions:
  - **Model**:

$$g(\mathbf{x}|\mathbf{w}) \tag{9}$$

  - **Loss function (Error function)**:

$$E(\mathbf{w}|\mathcal{X}) = \sum_{n=1}^{N} L(y_n, g(\mathbf{x}_n|\mathbf{w})) \tag{10}$$

  - **Optimization procedure/algorithm**:

$$\hat{\mathbf{w}} = \arg\min_{\mathbf{w}} E(\mathbf{w}|\mathcal{X}) \tag{11}$$

- No free lunch theorem:
  - There is no universally best model.
  - Different types of models have to be developed to suit the nature of the data in real applications.

Outline

1 Introduction to AI

2 Useful Data Platforms

3 Fundamentals of AI
- What Is Machine Learning
- Regression and Classification
- Common AI Models
- Overfitting
- Optimization Algorithms

One Common View of Many AI Models

- A model takes input features $\mathbf{x}$ and predicts an output $\hat{y}$.
- The model has parameters $\mathbf{w}$ that control how inputs are transformed into outputs.
- Linear regression is the simplest example, but the same skeleton appears again and again:

$$\mathbf{x} \quad \longrightarrow \quad g(\mathbf{x}|\mathbf{w}) \quad \longrightarrow \quad \hat{y}$$

Key message

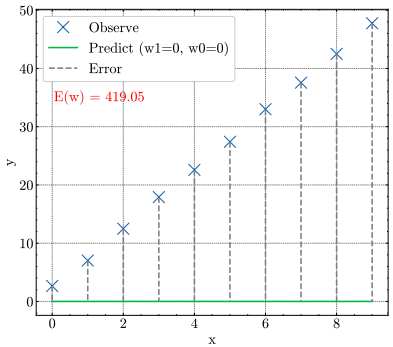
Different models mainly differ in the shape of $g(\mathbf{x}|\mathbf{w})$: line, curve, boundary, tree, ensemble, neural network, or Transformer.

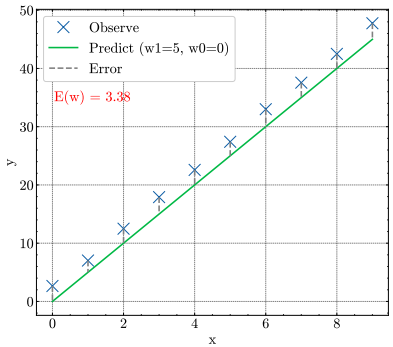
Linear Regression: A Weighted Sum

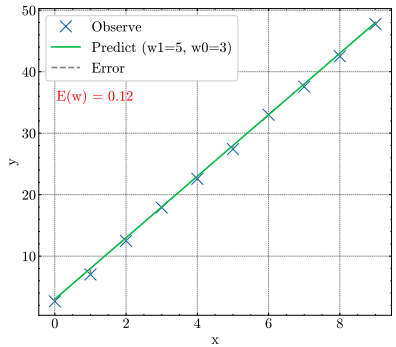
- Output is continuous: travel time, traffic demand, bridge response, construction cost.
- The model form is simple and interpretable:

$$\hat{y} = w_0 + w_1 x_1 + \cdots + w_D x_D$$

- Each coefficient tells us how one feature changes the prediction, holding other features fixed.





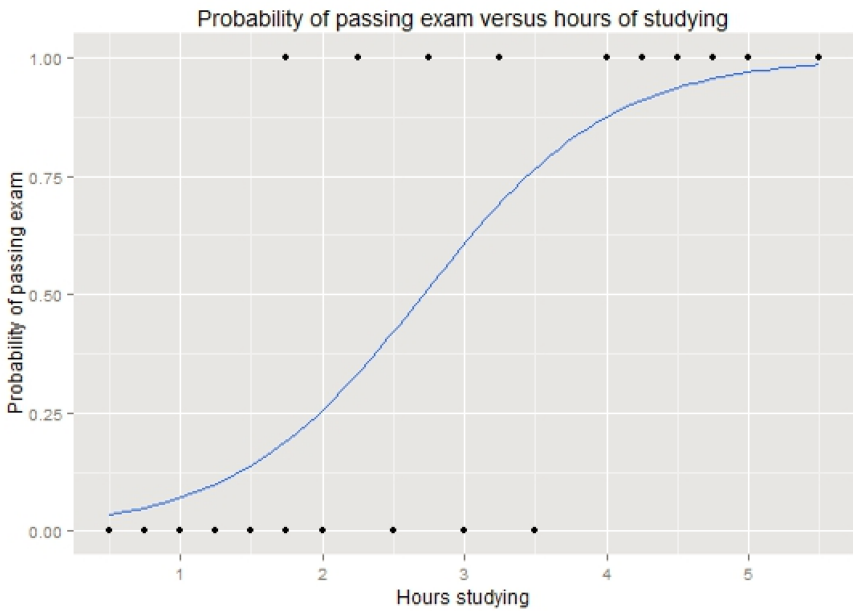


Key message

Linear regression is the baseline idea: choose a model form, learn parameters, then predict.

Logistic Regression: A Probability Model

- Output is a probability, such as crash/no crash or congested/not congested.
- Logistic regression first computes a linear score from the features.
- The score is then transformed into a probability:
  - Linear score: $z = \mathbf{w}^T\mathbf{x} + b$
  - Probability: $P(y = 1|\mathbf{x}) = \sigma(z)$

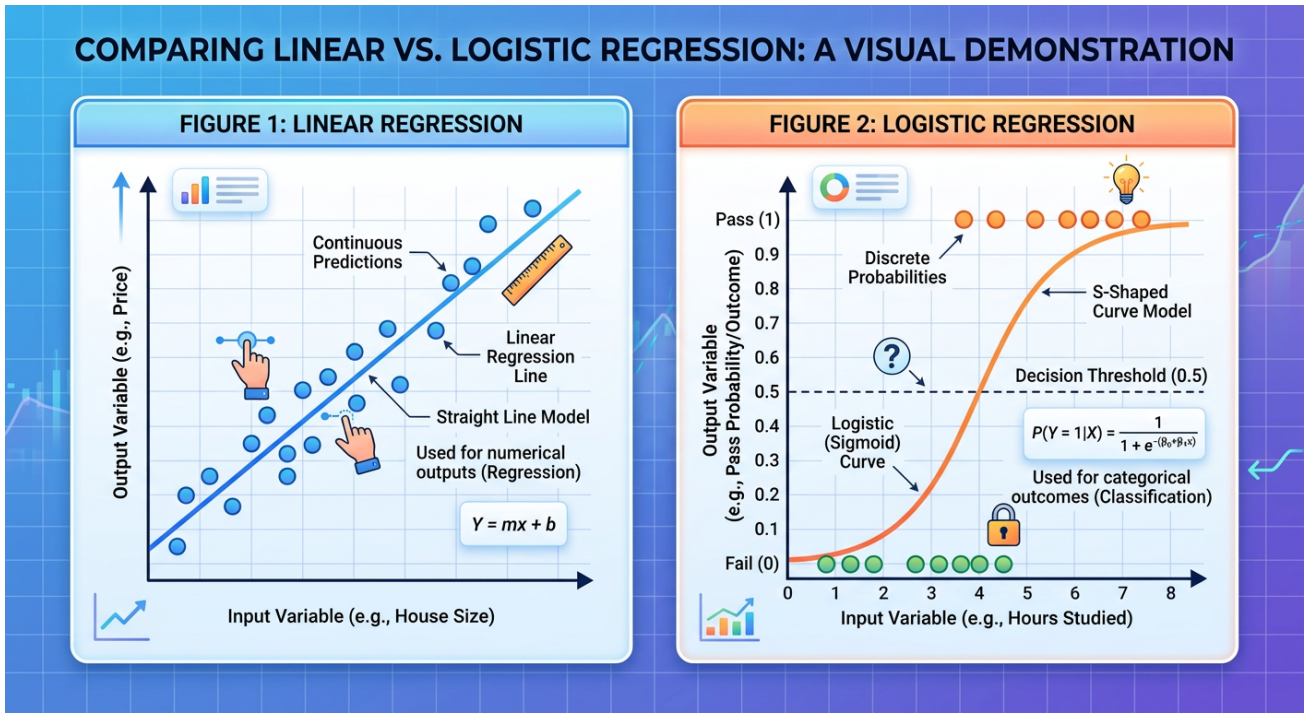


Key message

Classification follows the same skeleton: input features, model transformation, output class probability.

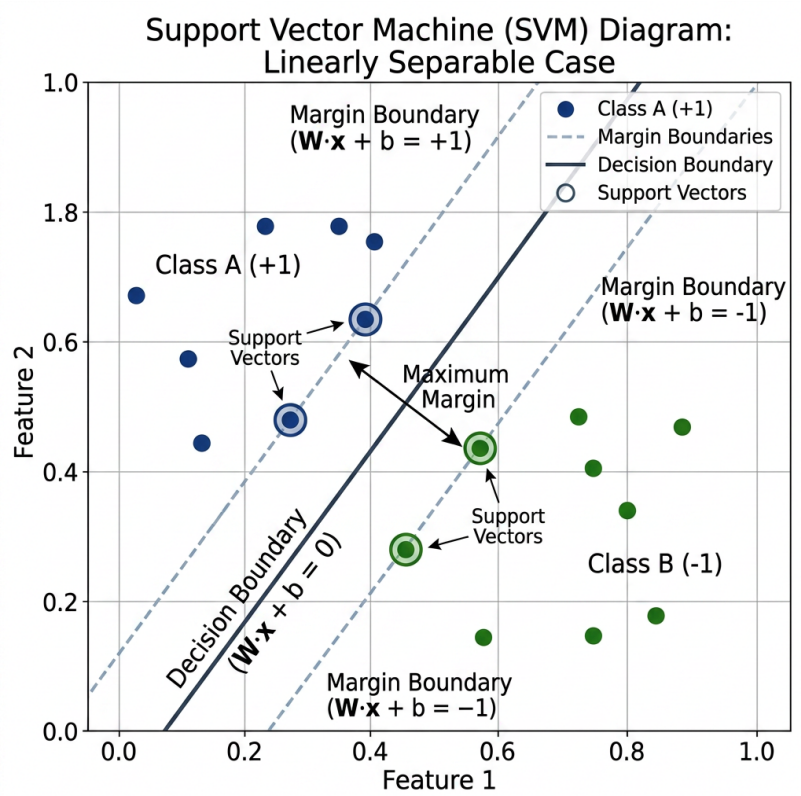
Logistic Regression: The Sigmoid Function

- Linear score: $z = \mathbf{w}^T\mathbf{x} + b = w_0 + w_1x_1 + w_2x_2 + \cdots + w_Dx_D,\ w_0 = b$
- Sigmoid probability: $P(y = 1|\mathbf{x}) = \sigma(z) = \dfrac{e^z}{e^z + 1}$
- Decision rule: $\hat{y} = 1$ if $P(y = 1|\mathbf{x}) \geq 0.5$, and $\hat{y} = 0$ otherwise.



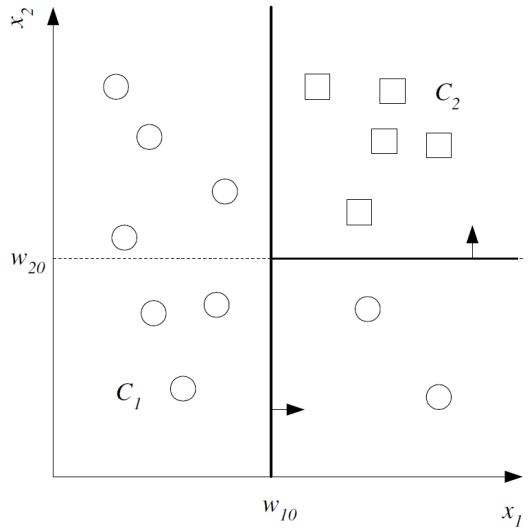
Support Vector Machines: A Large-Margin Boundary

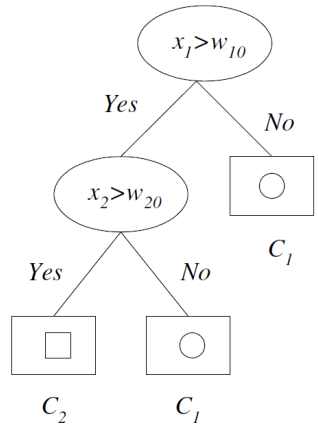
- SVM finds a boundary that separates classes with a large margin.
- Points near the boundary are the support vectors; they define the boundary.
- Kernel functions allow nonlinear boundaries while keeping the model idea compact.
- SVR adapts the same principle for regression.



Decision Trees: Rules from Data

- A tree predicts by asking a sequence of simple questions about the features.
- Each split divides the data into more homogeneous groups.
- A leaf node gives the final prediction: class label or continuous value.
- Trees are easy to explain, but a single tree can overfit.





Ensemble Learning: Why Combine Models?

- A single model may be unstable, biased, or too sensitive to one training sample.
- Ensemble learning trains multiple models and combines their predictions.
- The goal is not to make every individual model perfect, but to make the combined prediction more reliable.

Bagging

Train models in parallel on resampled data; combine by average or majority vote.

Boosting

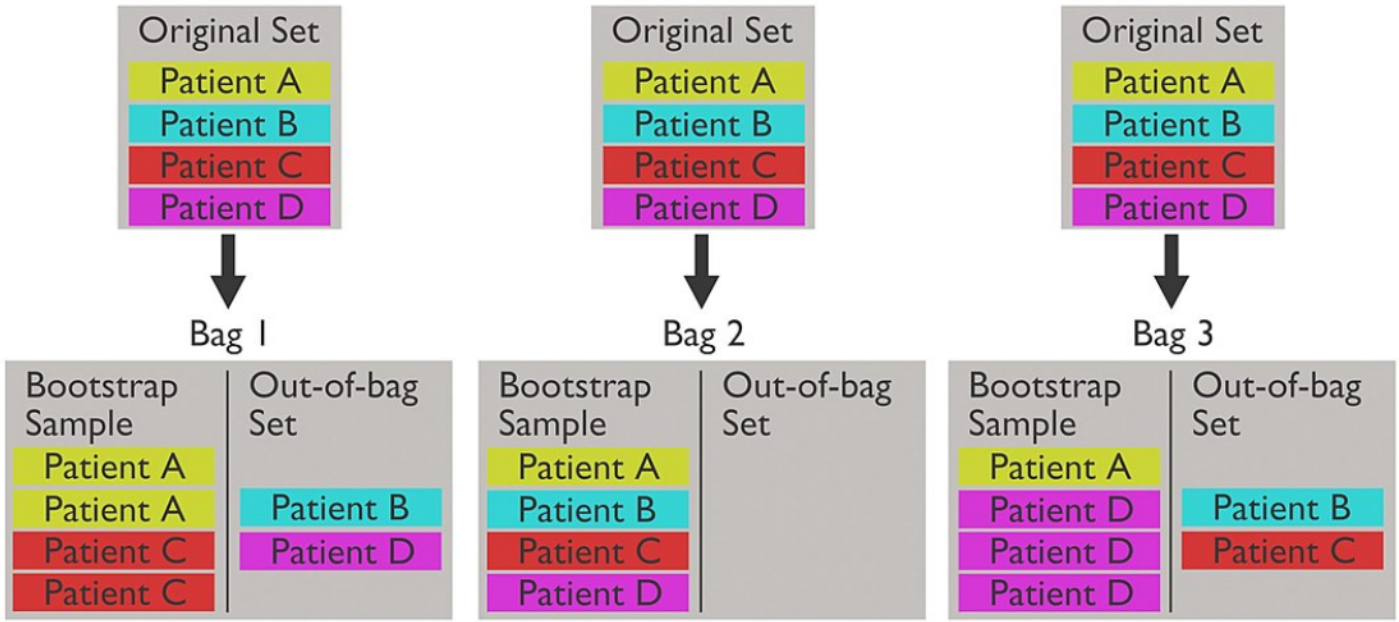
Train models sequentially; each new model focuses more on previous mistakes.

Key message

Bagging reduces variance; boosting reduces errors step by step by focusing on difficult samples.

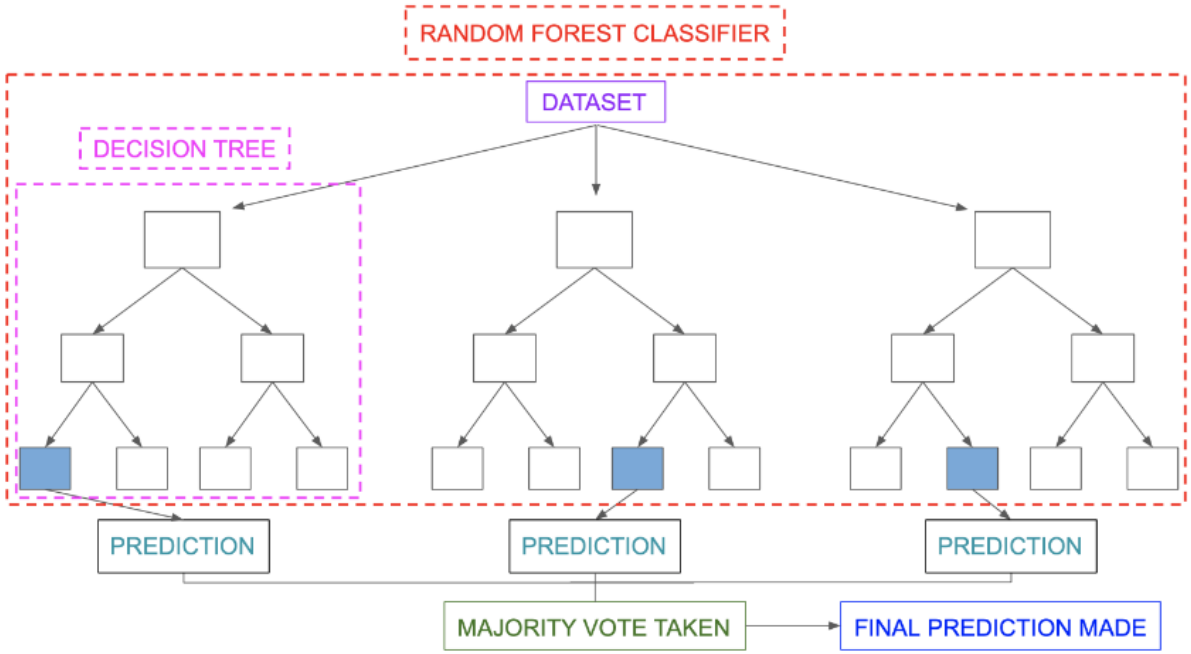
Bagging: Bootstrap Samples

- Bagging starts by creating many bootstrap samples from the original training set.
- Each sample is slightly different because some data points are repeated and some are left out.
- This makes the individual trees different from each other.



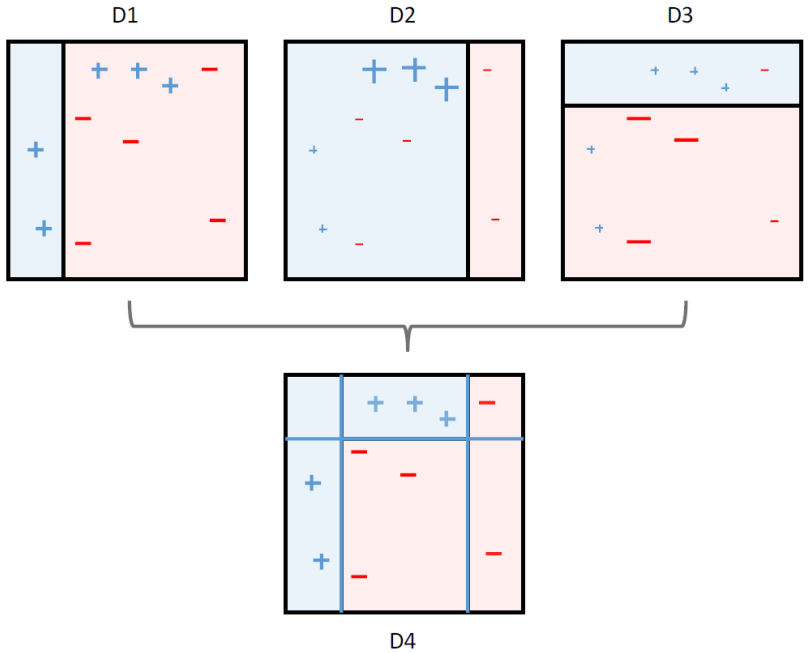
Bagging: Many Trees Vote Together

- Random forest is a typical bagging method.
- Each tree is trained on a different bootstrap sample.
- For classification, trees vote; for regression, predictions are averaged.



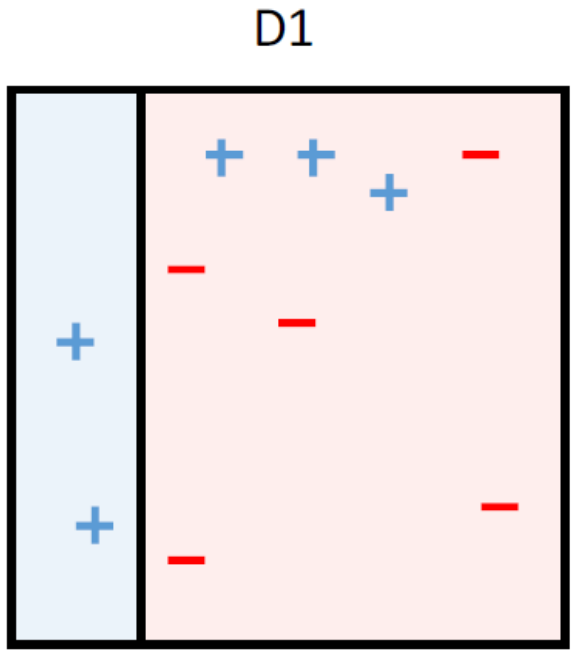
Boosting: A Simple Example

- Each weak learner is simple, such as a one-level decision tree.
- Boosting trains weak learners sequentially and gives more attention to previous mistakes.
- The final classifier combines D1, D2, and D3 into a stronger rule D4.



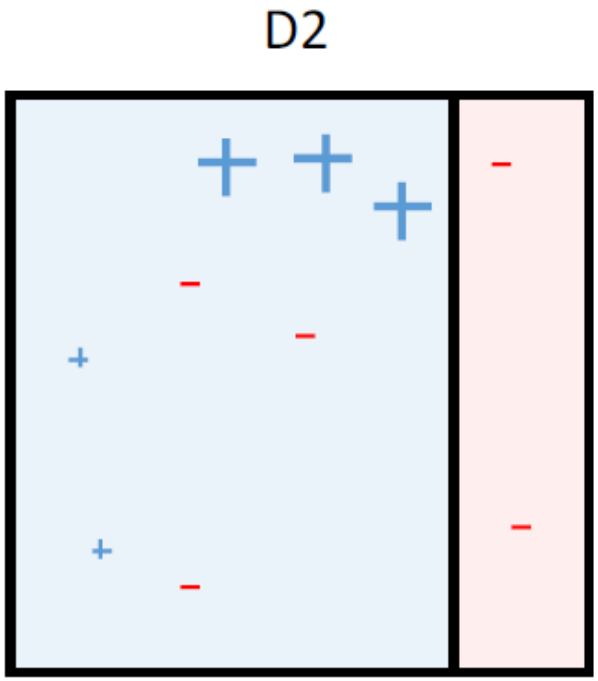
Boosting: D1

- At the beginning, all data points have equal weights.
- D1 is a very simple decision stump, so it misclassifies several positive samples.



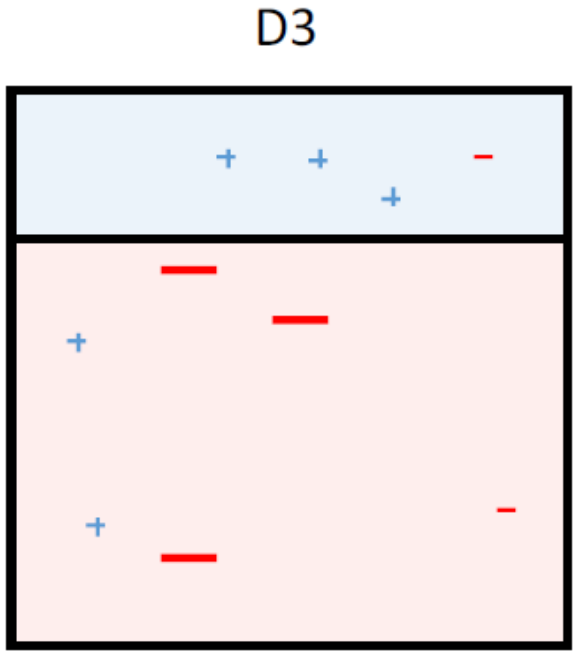
Boosting: D2

- Samples misclassified by D1 receive larger weights.
- D2 therefore tries harder to classify those previously difficult samples correctly.



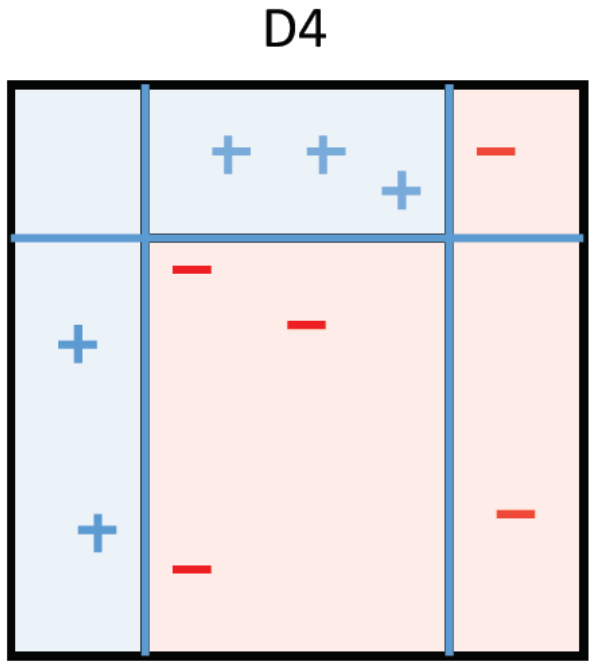
Boosting: D3

- After D2, another group of mistakes receives larger weights.
- D3 adds a new simple boundary to handle these remaining difficult samples.



Boosting: D4

- D1, D2, and D3 are weak learners, but their weighted combination becomes D4.
- The combined boundary is much more flexible than any single weak learner.

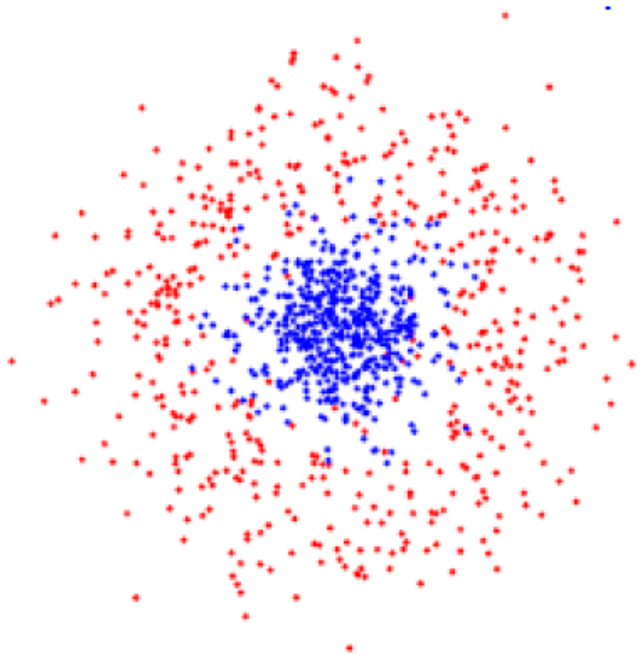


Key message

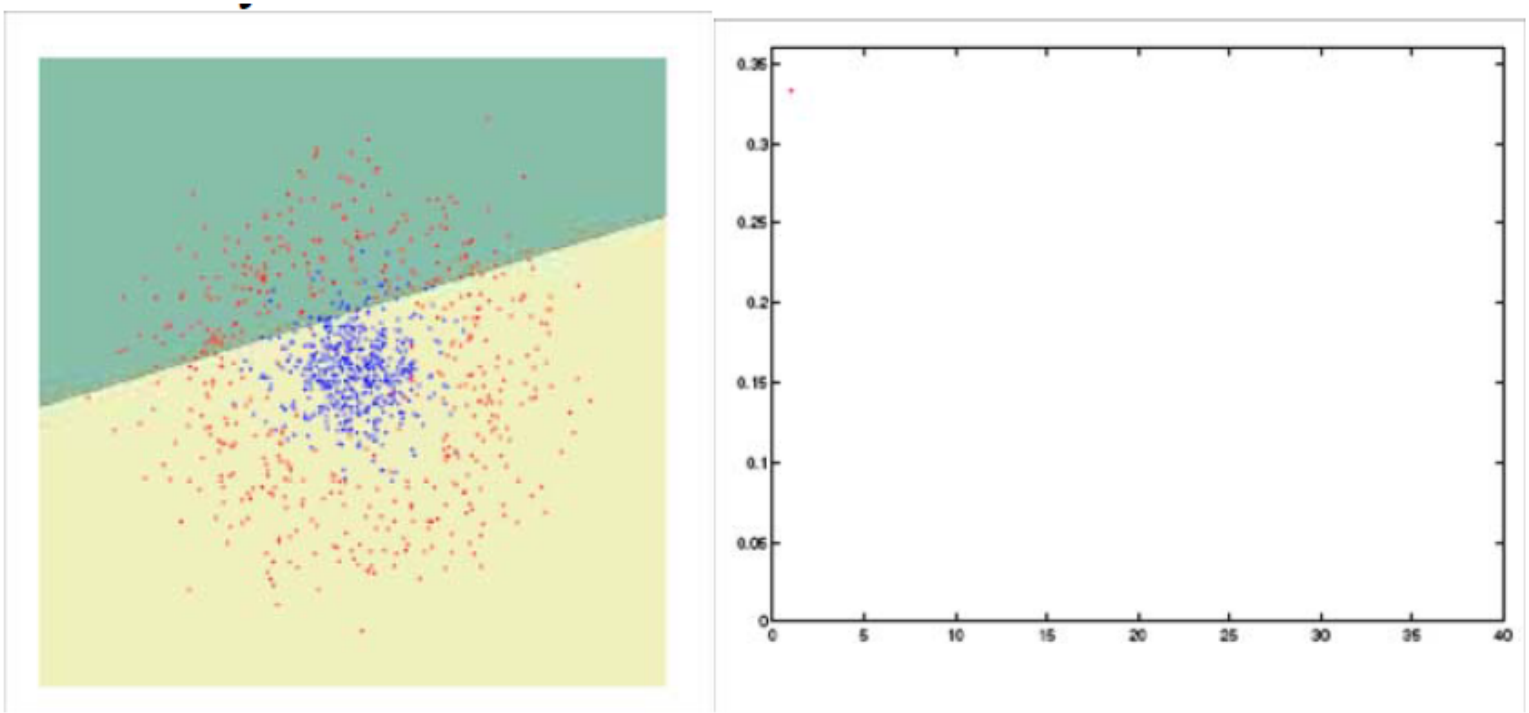
Boosting builds a strong model by adding simple models one by one.

Boosting: A More Realistic Example

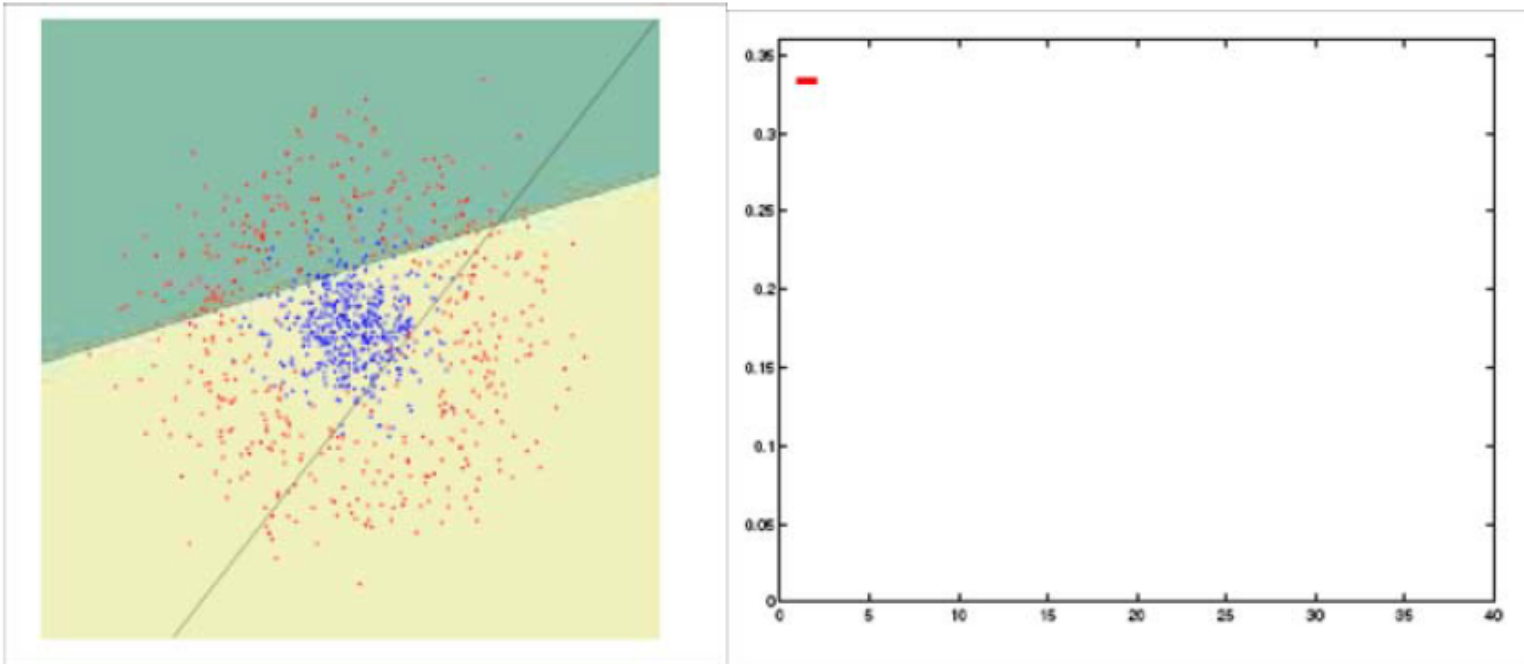
- The two classes form a nonlinear pattern.
- Linear weak learners are added sequentially to approximate a nonlinear boundary.



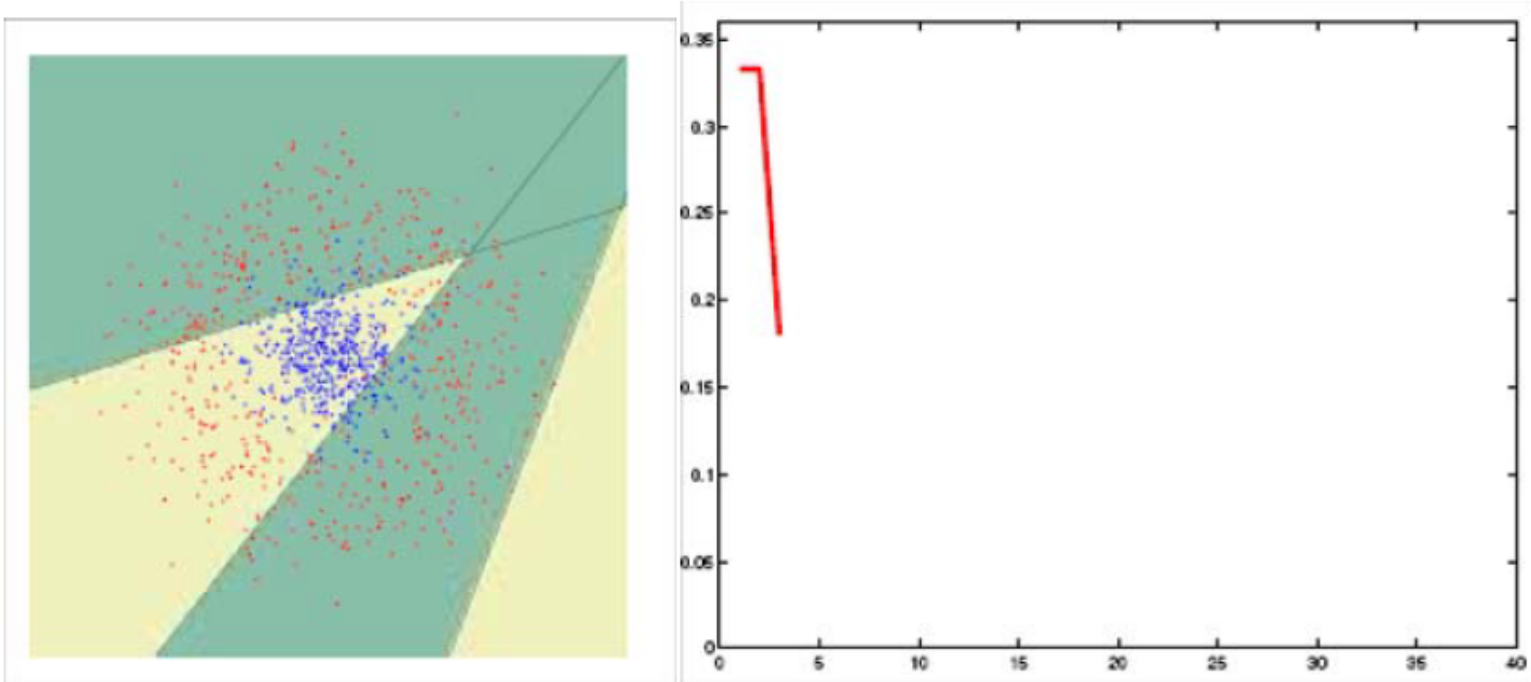
Boosting: Iteration 1



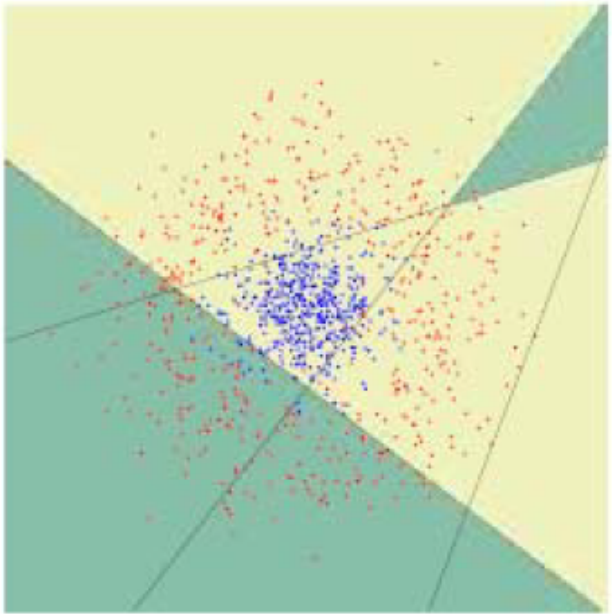
Boosting: Iteration 2

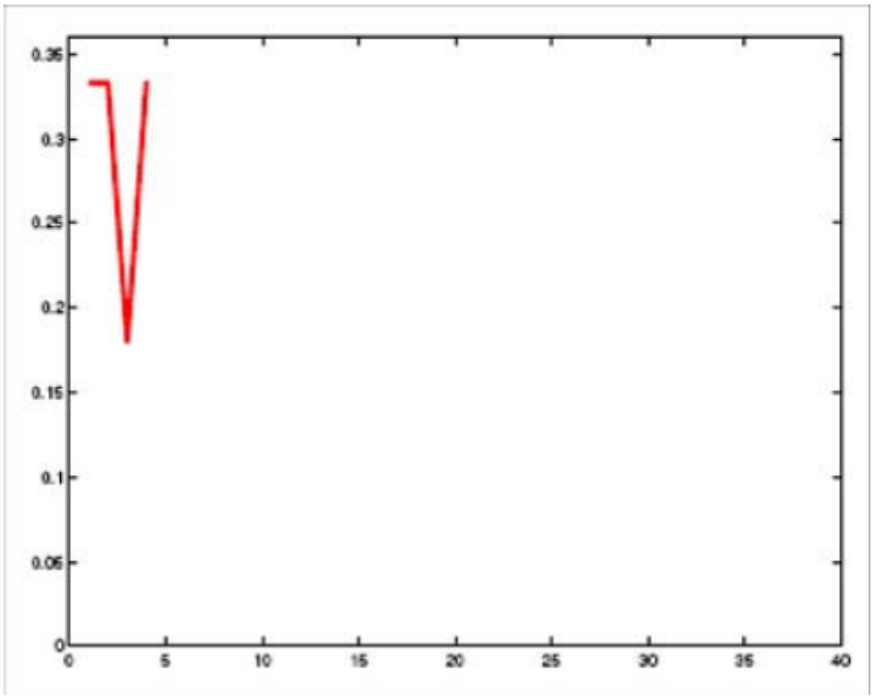


Boosting: Iteration 3

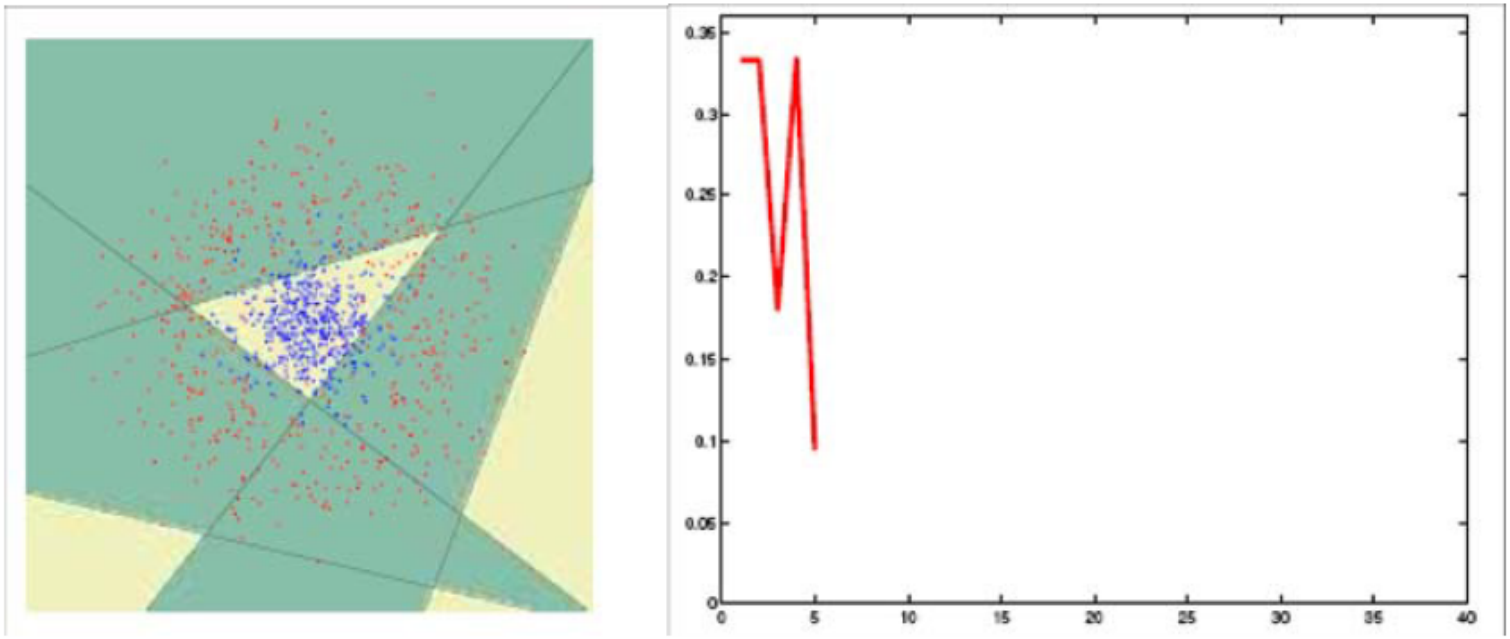


Boosting: Iteration 4

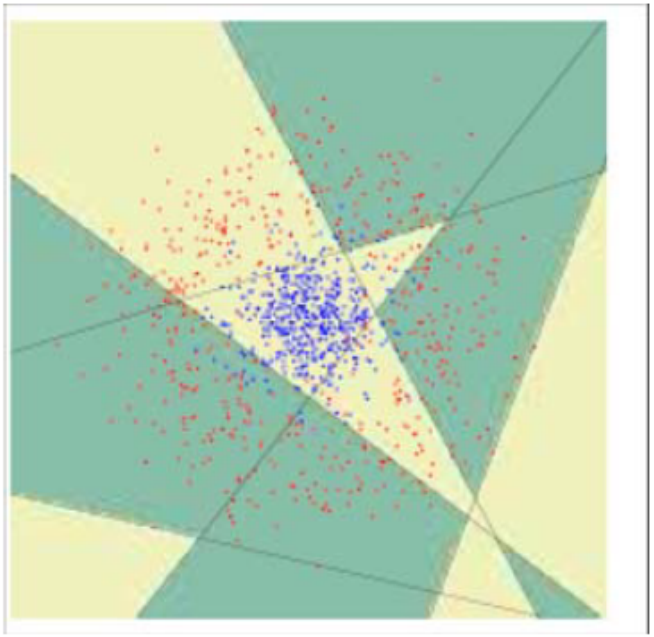


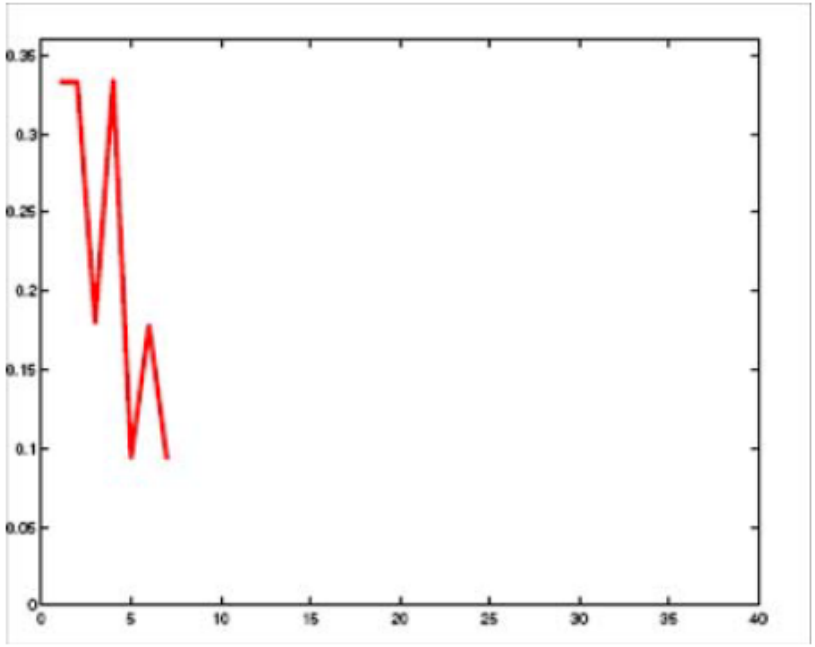


Boosting: Iteration 5

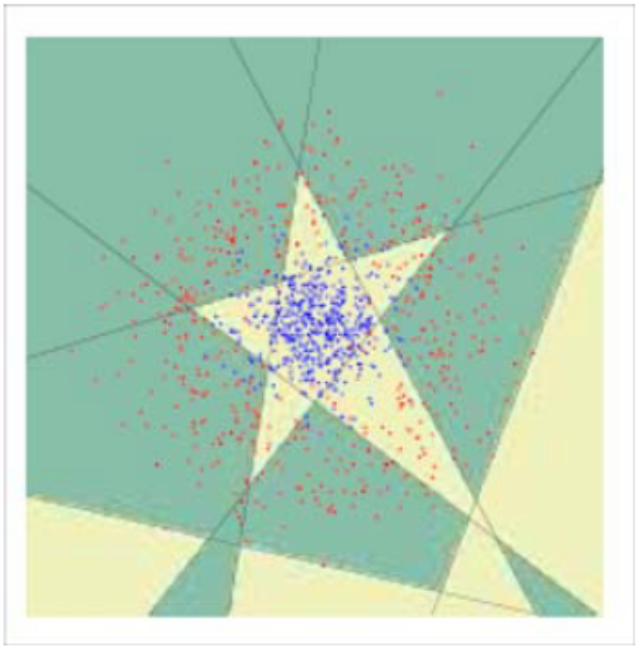


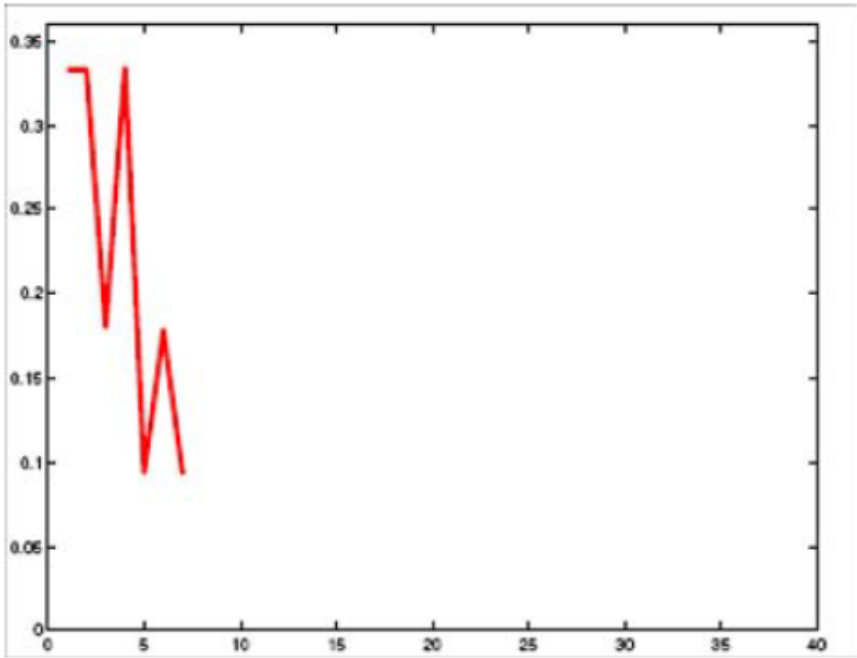
Boosting: Iteration 6





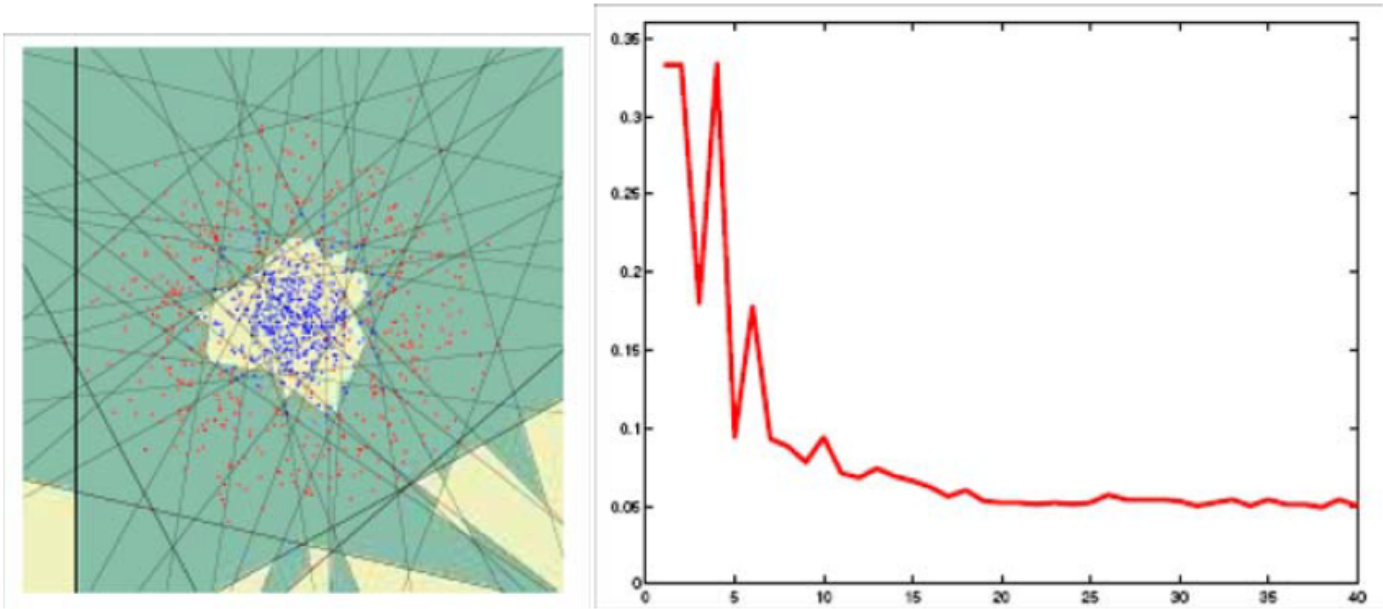
Boosting: Iteration 7





Boosting: Iteration 40

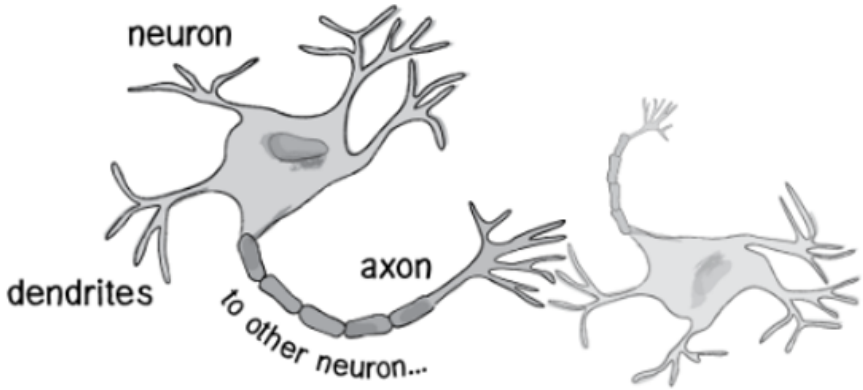
- As more weak learners are added, the boundary becomes more flexible and the error usually decreases.
- Too many iterations can still overfit, so validation remains important.

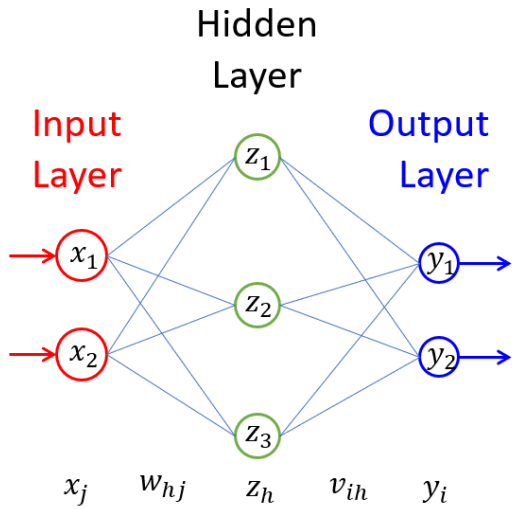


Neural Networks: From Neurons to Computation

- A biological neuron receives signals, combines them, and sends an output.
- An artificial neuron does something similar mathematically: weighted inputs, bias, activation.

$$a = \phi(w_1 x_1 + \cdots + w_D x_D + b)$$

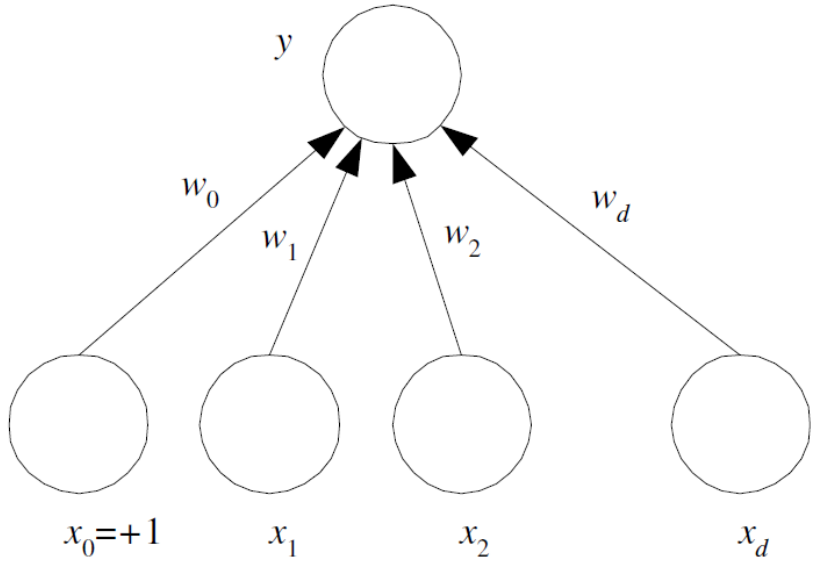




Single-Layer Perceptron: One Function

- A perceptron is the simplest neural network.
- It first computes a weighted sum of inputs.
- Then an activation function turns the score into an output.

$$y = \phi(w_0 + w_1 x_1 + \cdots + w_D x_D)$$



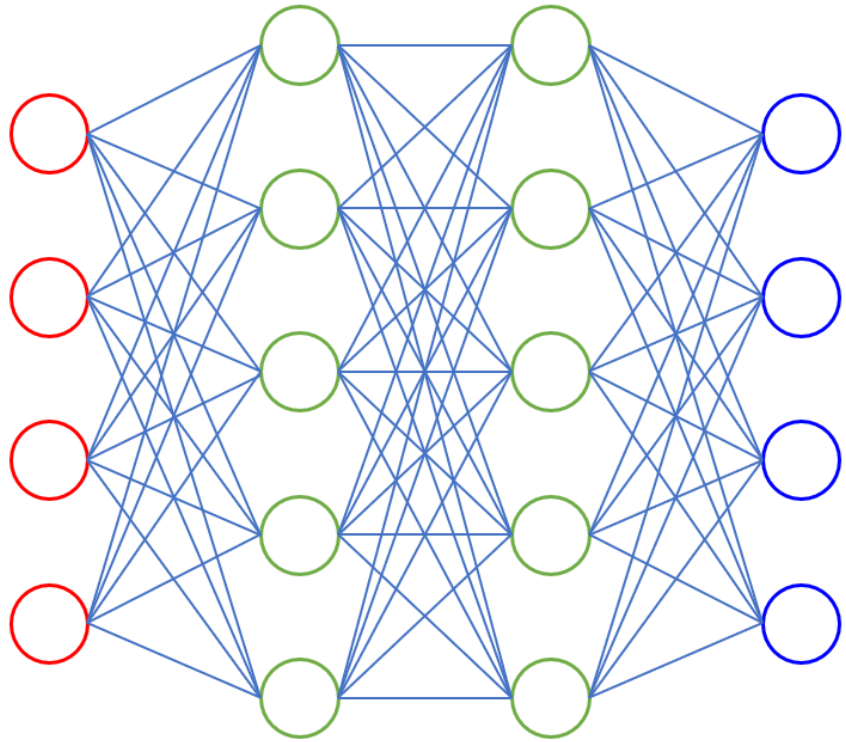
Key message

A perceptron is a function: input features go through weights and an activation to produce an output.

From Perceptron to MLP: Function Composition

- A multilayer perceptron stacks several functions together.
- Each layer transforms the previous layer's output.
- Nonlinear activations make the model more flexible than one linear model.

$$\mathbf{h}_1 = \phi_1(W_1\mathbf{x} + \mathbf{b}_1),$$
$$\mathbf{h}_2 = \phi_2(W_2\mathbf{h}_1 + \mathbf{b}_2),$$
$$\hat{\mathbf{y}} = \phi_3(W_3\mathbf{h}_2 + \mathbf{b}_3).$$
$$g(\mathbf{x}) = g_3(g_2(g_1(\mathbf{x})))$$

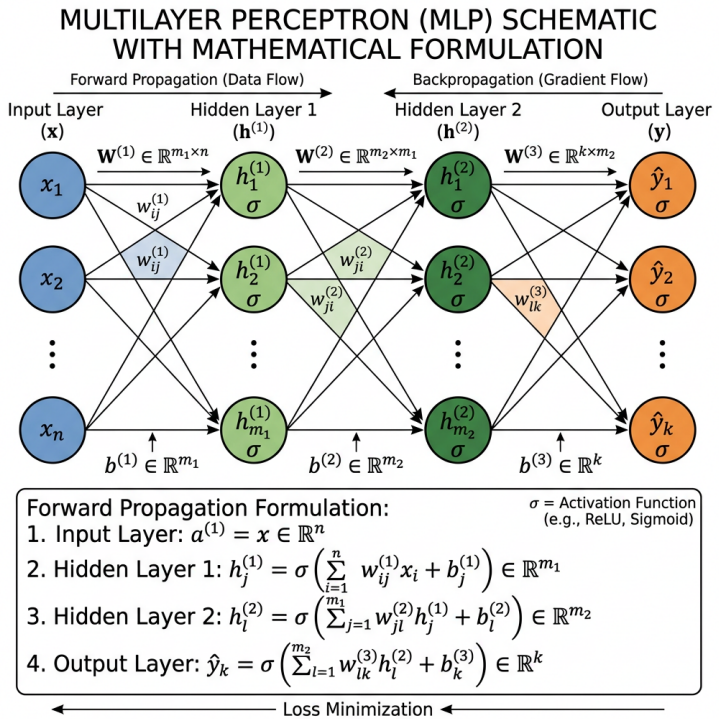


Key message

An MLP is essentially function composition: function inside function, layer after layer.

Neural Networks: Layers Learn Representations

- A neural network stacks many simple transformations.
- Hidden layers transform raw inputs into more useful representations.
- The parameters are weights and biases across all connections.
- Deeper networks can learn more complex nonlinear functions.



Key message: A neural network is still $g(\mathbf{x}|\mathbf{w})$, but g is built from many connected layers.

Neural Networks: A Transportation Engineering Example

- Example task: predict BART station-to-station ridership from station and catchment-area features.
- Inputs can include population, car ownership, employment, and road network density near stations.
- The neural network maps these features to ridership predictions.

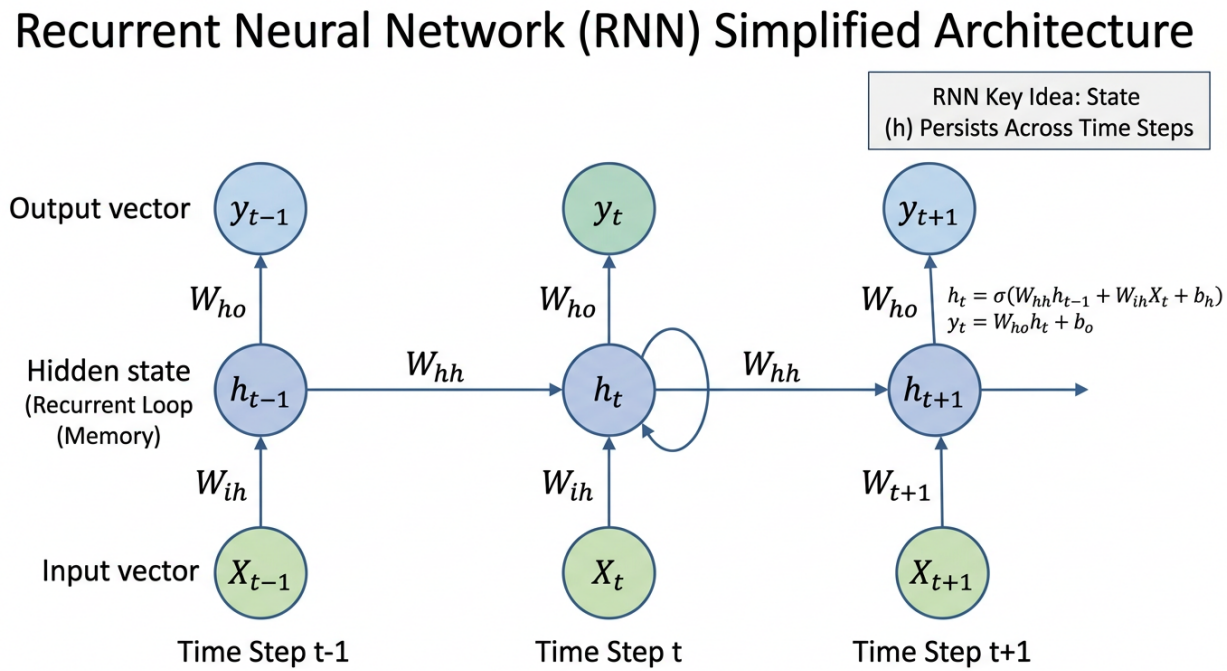
Neural Networks: Features for the BART Example

- Each station becomes one data sample with several engineered features.
- The model still follows the same supervised learning form: $\mathbf{x}$ is the feature vector and y is ridership.

	A	B	C	D	E
1	Station	TotPop	AutoOwn0	TotEmp	Tot Rd Dens
2	12TH	11084	1664	33120	42.232657
3	16TH	142973	20505	103479	32.296762
4	19TH	36177	4704	47570	31.822631
5	24TH	104191	8018	39663	32.894203
6	ANTC	70630	1142	13924	20.531431
7	ASHB	47031	2693	19304	30.456888
8	BALB	123382	3871	16384	29.802149
9	BAYF	92905	1949	21128	24.243937
10	CAST	61232	1276	11796	20.000183
11	CIVC	125375	32395	113812	32.944622

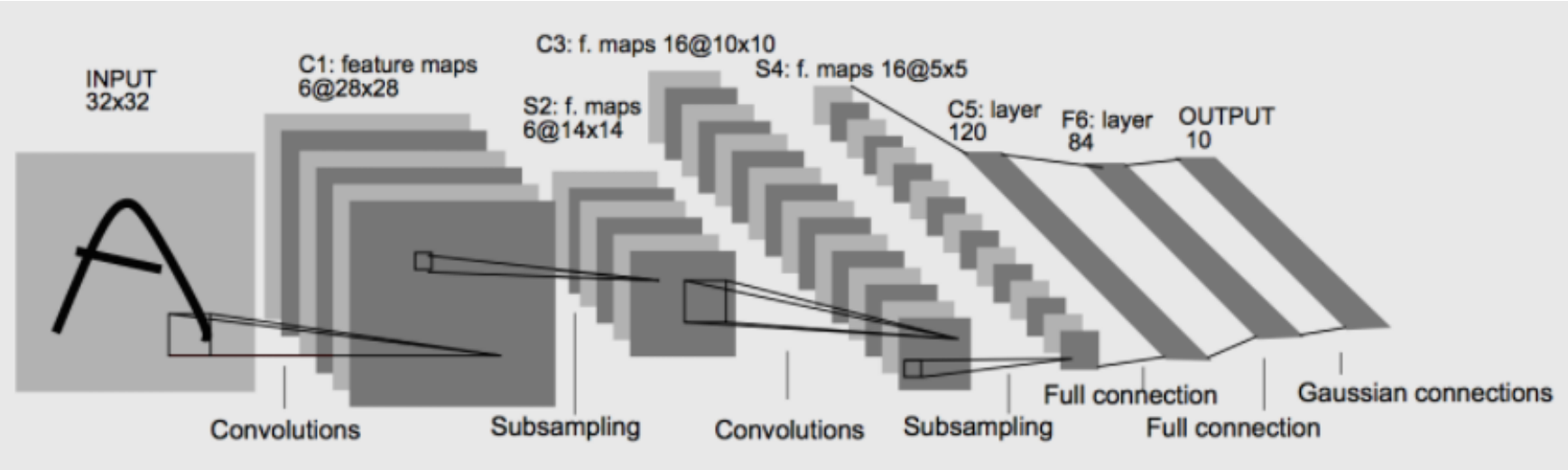
RNN: Models for Sequences

- RNNs are designed for ordered data: time series, text, trajectories, and sensor streams.
- The model keeps a hidden state that carries information from previous steps.
- Useful when yesterday's or last second's information matters for the current prediction.



CNN: Models for Images and Grids

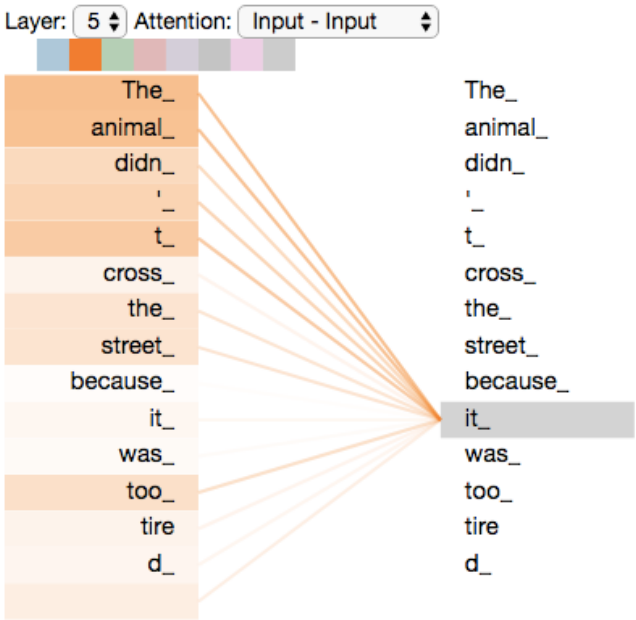
- CNNs are designed for grid-like data: images, maps, and rasterized spatial data.
- Convolutional filters scan local regions and detect patterns; pooling reduces spatial size.
- Later layers combine local patterns into high-level representations for prediction.



Transformer: Models Based on Attention

- Attention lets each token look at other relevant tokens before making a prediction.
- A Transformer stacks many attention-based blocks, so information can flow across the whole sequence.

Intuition: which words should "it" look at?



Architecture: attention blocks are stacked deeply

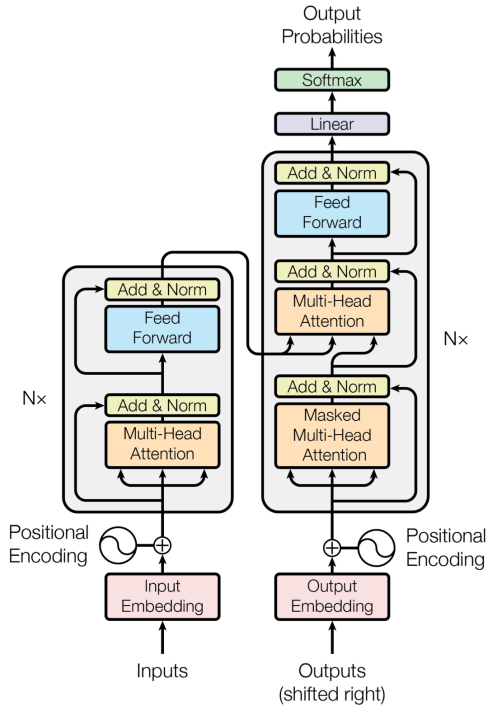
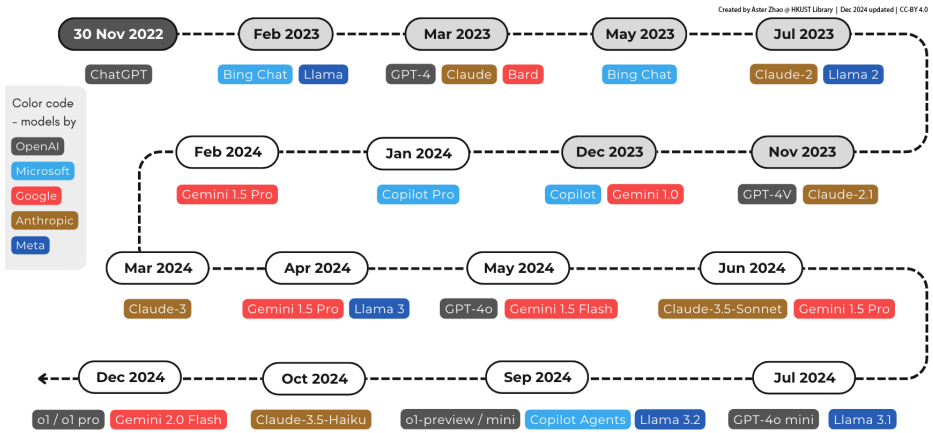


Figure 1: The Transformer - model architecture.

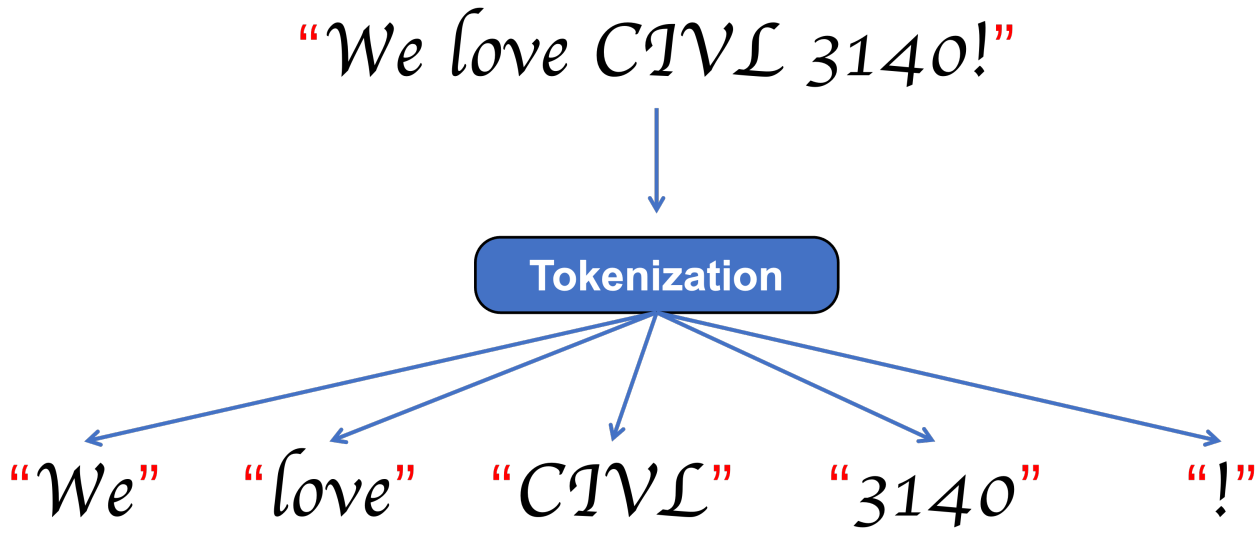
LLM: A Large Transformer Trained on Massive Data

- Large language models are usually Transformer-based models trained on massive text and code data.
- Their basic training task is next-token prediction.
- After instruction tuning and tool use, they can become assistants, coding agents, and research helpers.



LLM: Text Becomes Tokens

- An LLM does not directly see a paragraph as human language.
- The text is first split into tokens: words, subwords, punctuation, or symbols.
- These tokens become the input sequence $\mathbf{x}$ for the model.



LLM: Next-Token Prediction Is Still $g(\mathbf{x}|\mathbf{w})$

- For an LLM, $\mathbf{x}$ is the current context: all tokens seen so far.
- The model output is a vector of probabilities over the whole vocabulary.
- Each entry of this vector is the probability of one possible next token.

$$\hat{\mathbf{p}} = g(\text{context tokens} \mid \mathbf{w}) = [\hat{p}_1, \hat{p}_2, \ldots, \hat{p}_V],$$
$$\hat{p}_j = P(\text{next token} = j \mid \text{context}).$$

Vocabulary size V: one probability for each candidate token, and the probabilities sum to 1.

Current context	Probabilities for the next token			
"The bridge is"	safe: 0.42	long: 0.18	old: 0.11	closed: 0.08

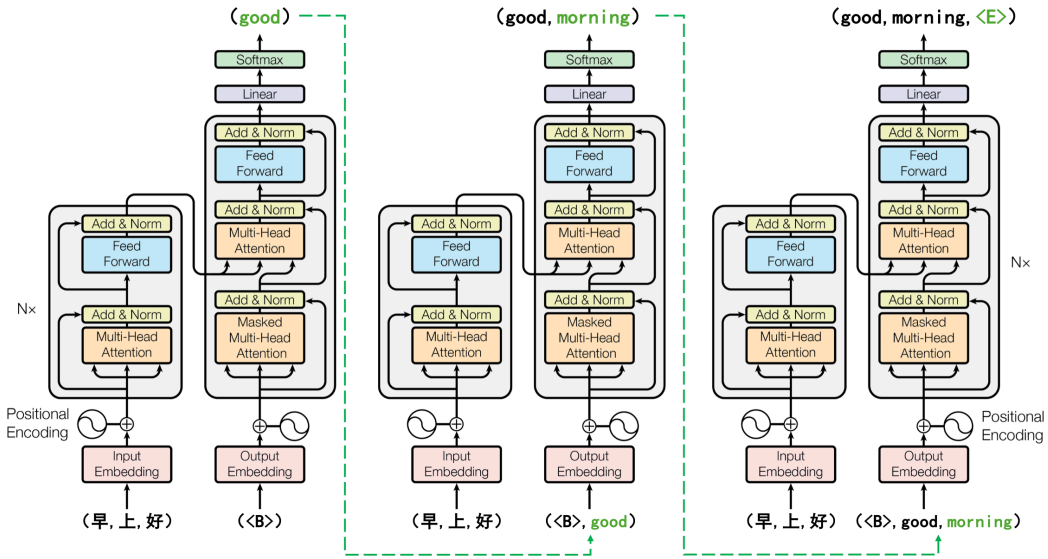
Key message

The label y is the next token in the training text; the model learns to assign high probability to the correct next token.

LLM: Generate One Token, Then Repeat

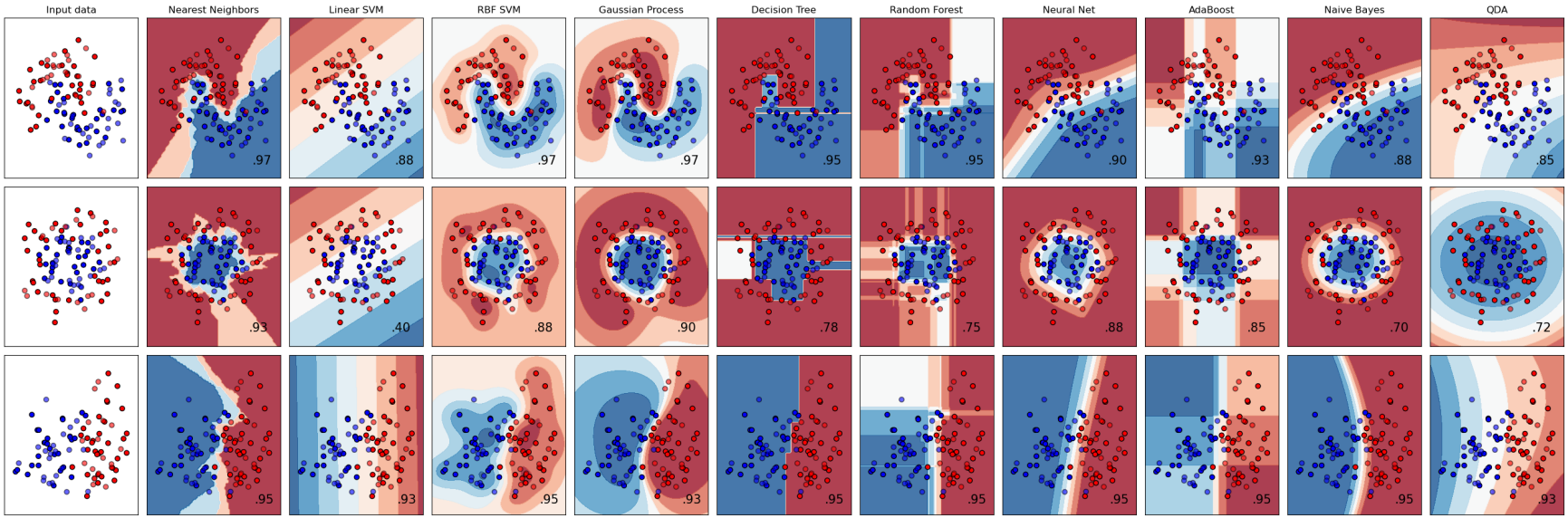
1. Input sentence: the source-language sequence shown in the figure, wrapped by <BOS> and <EOS>.
2. Step 1: given the source sentence and the decoder start token <BOS>, the model predicts "Good".
3. Step 2: given the same source sentence plus the generated token "Good", the model predicts "Morning".
4. The process repeats until the model predicts <EOS>.

$$\text{source sentence} + \texttt{<BOS>} \rightarrow \text{Good},$$
$$\text{source sentence} + \texttt{<BOS>} + \text{Good} \rightarrow \text{Morning},$$
$$\text{source sentence} + \texttt{<BOS>} + \text{Good Morning} \rightarrow \texttt{<EOS>}$$



The Main Message: Different Boundaries, Same Logic

- Different algorithms make different assumptions about the shape of the decision boundary.
- Simple models are easier to explain; flexible models can capture more complex patterns.
- There is no universally best model: the right choice depends on data, task, and decision need.



Source: scikit-learn, Classifier comparison example

The Main Message: Same Skeleton, Richer Model Forms

- Linear regression: weighted sum for continuous outputs.
- Logistic regression and SVM: decision boundaries for classes.
- Trees and ensembles: rule-based prediction and model combinations.
- Neural networks: layered transformations for complex nonlinear functions.
- RNN, CNN, Transformer, and LLM: architectures specialized for sequences, images, attention, and language.

Key message

They look different, but they are all models: choose a form $g(\mathbf{x}|\mathbf{w})$, learn parameters $\mathbf{w}$, then use it to predict or decide.

Outline

1 Introduction to AI

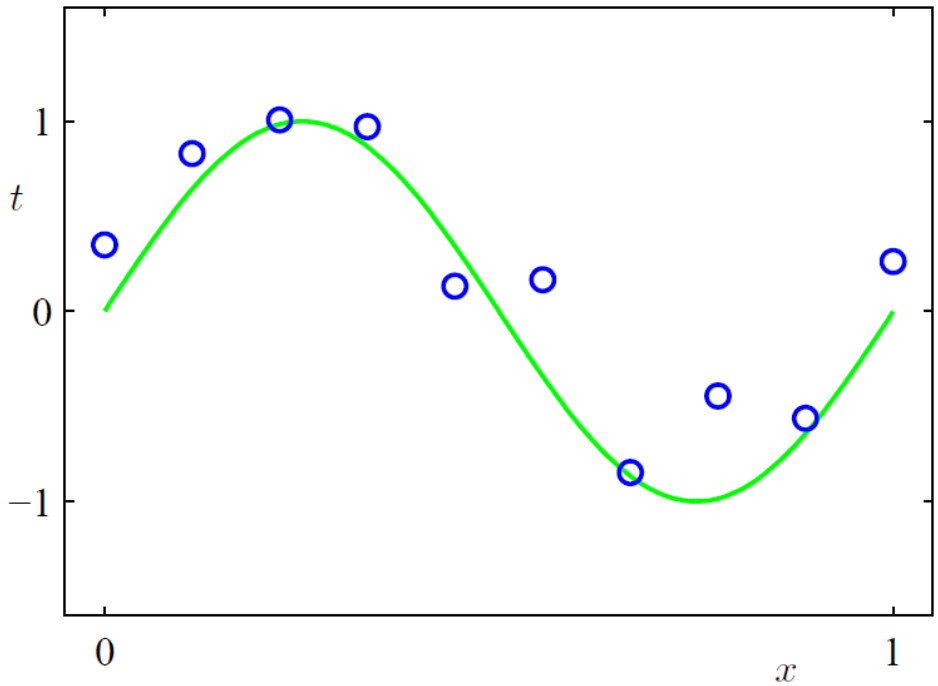
2 Useful Data Platforms

3 Fundamentals of AI
- What Is Machine Learning
- Regression and Classification
- Common AI Models
- Overfitting
- Optimization Algorithms

Case: Polynomial Curve Fitting

- Let us look at a regression problem:
  - Features: x (only one feature)
  - Label: t
- Data generation process (the same for training and test set):

$$t = \sin(2\pi x) + \text{noise} \tag{12}$$



Polynomial Function as Linear Model

- Then we use the following polynomial function to fit the training data:

$$y(x, \mathbf{w}) = w_0 + w_1 x + w_2 x^2 + \ldots + w_M x^M = \sum_{j=0}^{M} w_j x^j \tag{13}$$

  where $\mathbf{w} = (w_0, w_1, w_2, \ldots, w_M)^T$ and M is the order of the polynomial.
- Linear model: the function $y(x, \mathbf{w})$ is nonlinear in x, but linear in $\mathbf{w}$.

Parameters and Hyperparameters

Parameters

- Parameters are internal variables automatically updated by the optimization algorithm during training to minimize the model's prediction error.
- Parameters are usually initialized to random values or zeros at the beginning of training.
- Examples: Coefficients of regression models; Weights and biases in neural networks.

Hyperparameters

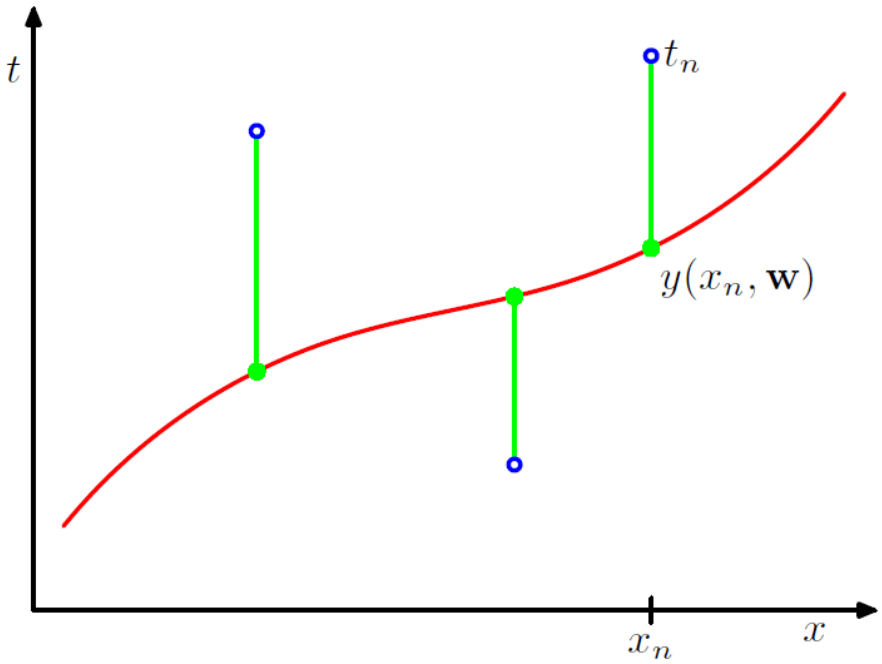
- Hyperparameters are external configurations set before the learning process begins to govern the training process.
- Examples: The order of polynomial; Learning rate in optimization algorithms.

Error Minimization

- Clearly, this is a regression problem, thus the error function is given by:

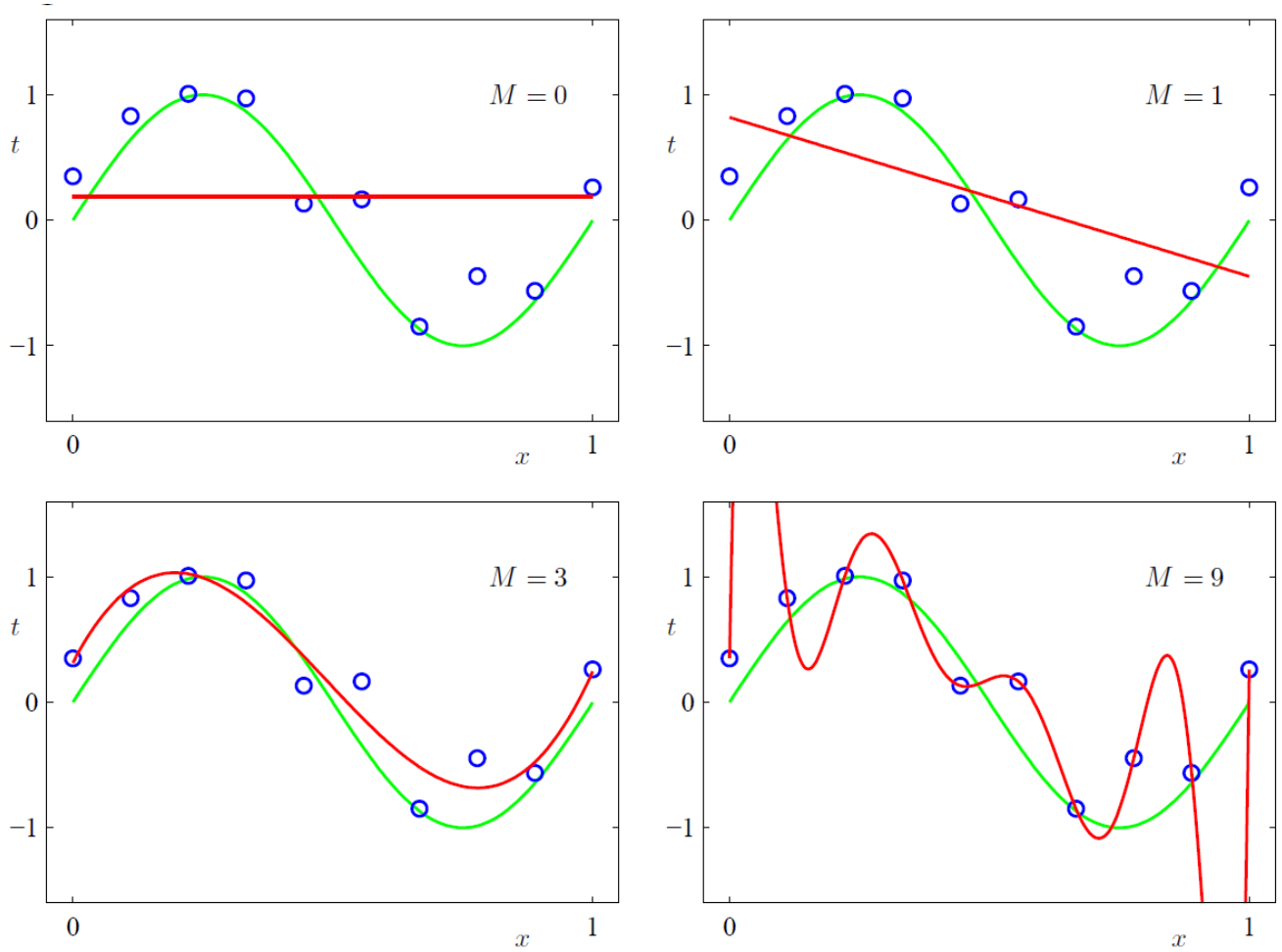
$$E(\mathbf{w}|\mathcal{X}) = \frac{1}{2}\sum_{n=1}^{N}[y(x_n, \mathbf{w}) - t_n]^2 \tag{14}$$

- The optimal parameters $\hat{\mathbf{w}}$ can be obtained by minimizing the error function $E(\mathbf{w}|\mathcal{X})$.



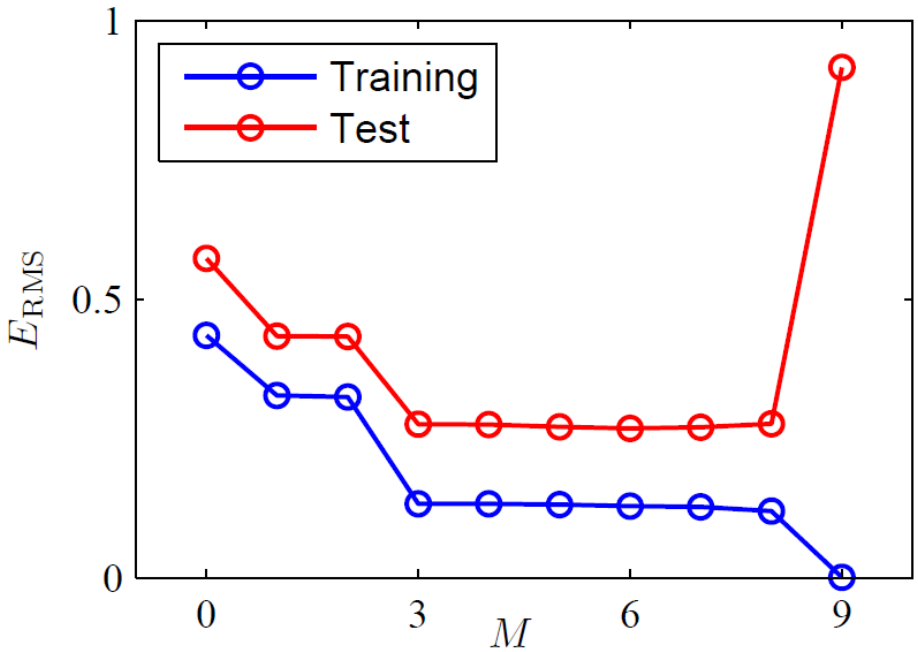
Model Selection

- Choosing the order of polynomial is an example of model selection. In this case, which model is better?



Over-fitting Issue

- Although the training set and test set are drawn from the same distribution, they have different sample points.
- A very complex model may over-fit the curve of the training set, and thus may not well fit the test set. (indicating a low generalization ability)
- Calculate $E_{RMS} = \sqrt{2E(\hat{\mathbf{w}}|\mathcal{X})/N}$ for training and test set for different models (with different orders of polynomial)



Over-fitting Issue

- The optimal parameters $\mathbf{w}^*$ ($\hat{\mathbf{w}}$) in different models:

	$M=0$	$M=1$	$M=3$	$M=9$
w_0^*	0.19	0.82	0.31	0.35
w_1^*		−1.27	7.99	232.37
w_2^*			−25.43	−5321.83
w_3^*			17.37	48568.31
w_4^*				−231639.30
w_5^*				640042.26
w_6^*				−1061800.52
w_7^*				1042400.18
w_8^*				−557682.99
w_9^*				125201.43

How to Reduce Over-fitting

- Method 1: increase the number of samples.

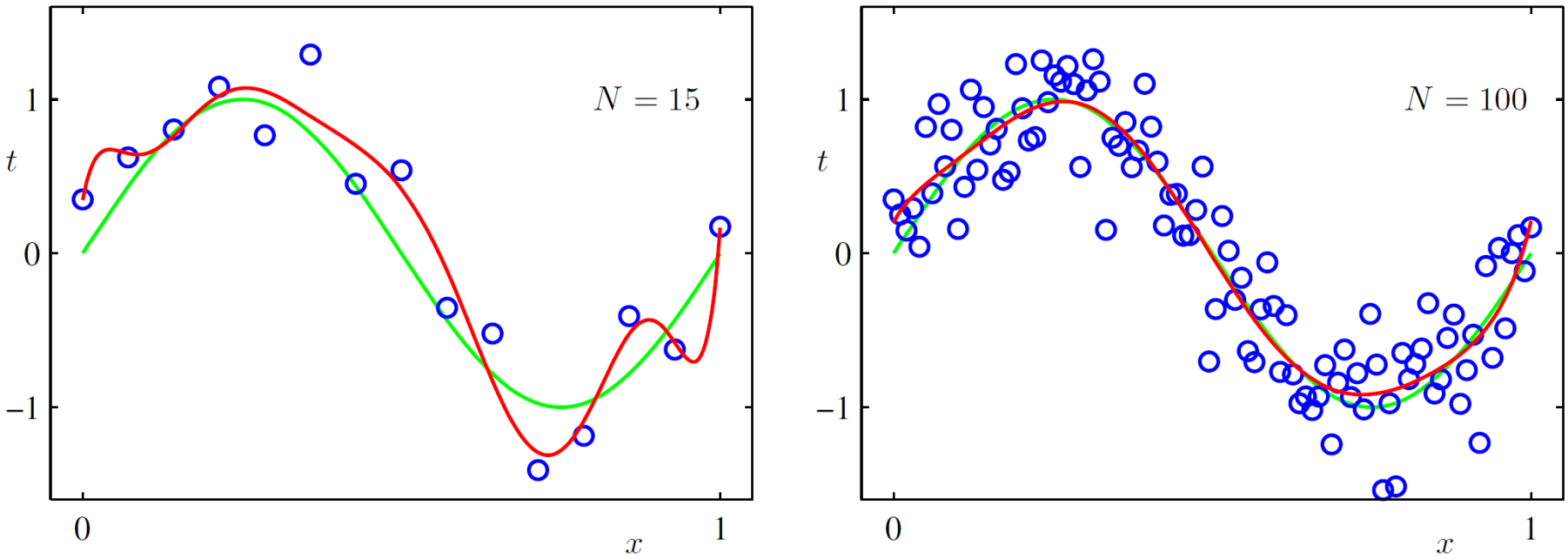


Figure 1.6 Plots of the solutions obtained by minimizing the sum-of-squares error function using the $M = 9$ polynomial for $N = 15$ data points (left plot) and $N = 100$ data points (right plot). We see that increasing the size of the data set reduces the over-fitting problem.

How to Reduce Over-fitting

- Method 2: Adding a regularization term to the error function:

$$\tilde{E}(\mathbf{w}|\mathcal{X}) = \frac{1}{2}\sum_{n=1}^{N}[y(x_n, \mathbf{w}) - t_n]^2 + \frac{\lambda}{2}\|\mathbf{w}\|^2 \tag{15}$$

- where $\|\mathbf{w}\|^2 = \mathbf{w}^T\mathbf{w} = w_0^2 + w_1^2 + ... + w_M^2$, and the coefficient λ governs the relative importance of the regularization term compared with the sum-of-squares error term.
- The larger the λ, the stronger the penalty on the size of parameters. This will force the model to select parameters with reasonable values.

How to Reduce Over-fitting

- No regularization: $\lambda = 0, \log \lambda = -\infty$
- Medium regularization coefficient: $\lambda = e^{-18}, \log \lambda = -18$
- Large regularization coefficient: $\lambda = 1, \log \lambda = 0$

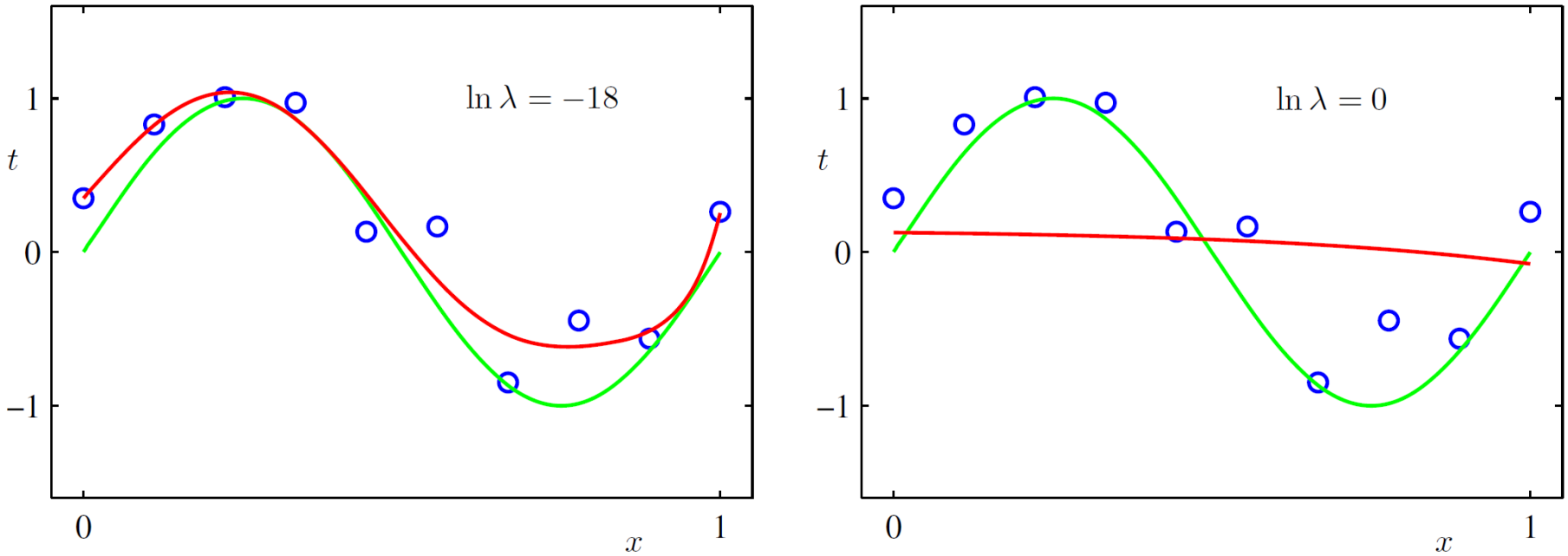


Figure 1.7 Plots of $M = 9$ polynomials fitted to the data set shown in Figure 1.2 using the regularized error function (1.4) for two values of the regularization parameter λ corresponding to $\ln \lambda = -18$ and $\ln \lambda = 0$. The case of no regularizer, i.e., $\lambda = 0$, corresponding to $\ln \lambda = -\infty$, is shown at the bottom right of Figure 1.4.

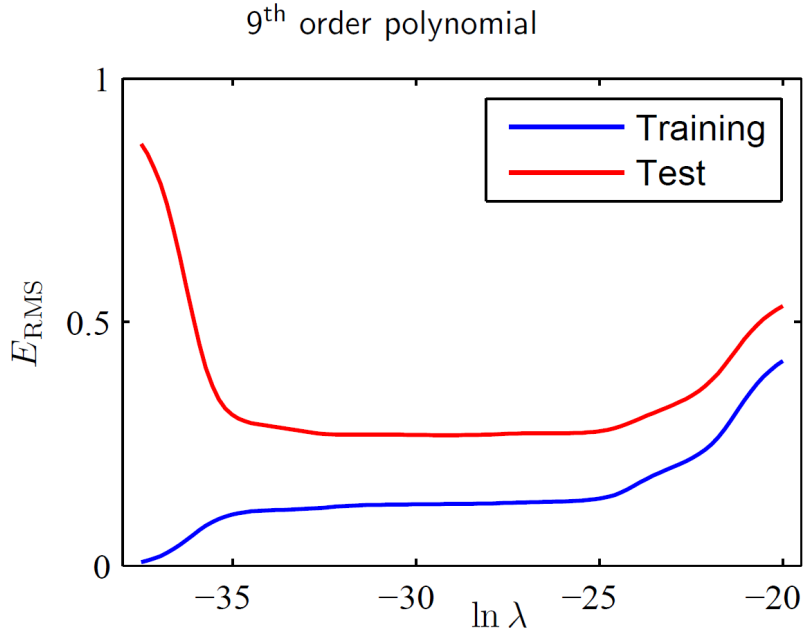
How to Reduce Over-fitting

- We see that, as the value of λ increases, the absolute magnitude of the coefficients gets smaller.

	$\ln \lambda = -\infty$	$\ln \lambda = -18$	$\ln \lambda = 0$
$w_0^\star$	0.35	0.35	0.13
$w_1^\star$	232.37	4.74	-0.05
$w_2^\star$	-5321.83	-0.77	-0.06
$w_3^\star$	48568.31	-31.97	-0.05
$w_4^\star$	-231639.30	-3.89	-0.03
$w_5^\star$	640042.26	55.28	-0.02
$w_6^\star$	-1061800.52	41.32	-0.01
$w_7^\star$	1042400.18	-45.95	-0.00
$w_8^\star$	-557682.99	-91.53	0.00
$w_9^\star$	125201.43	72.68	0.01

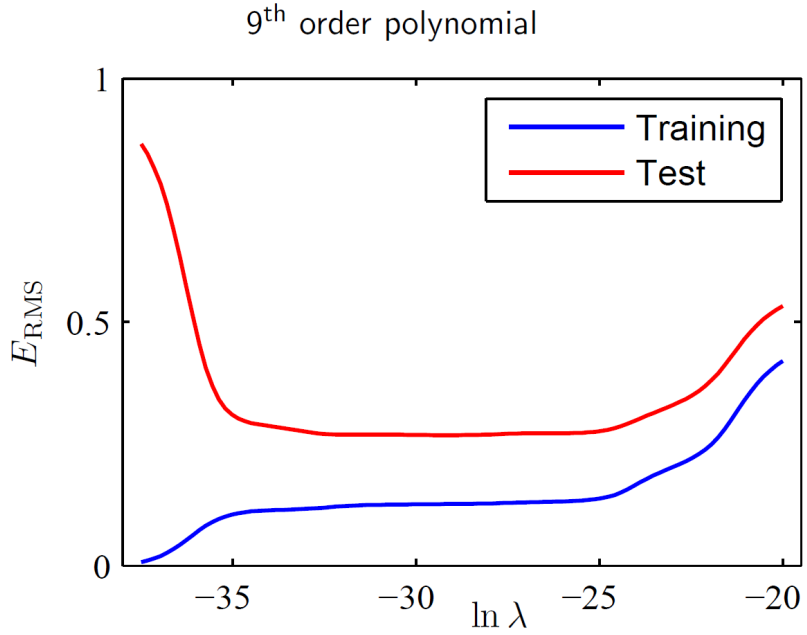
How to Reduce Over-fitting

- As the value of λ increases, the gap between the RMSE in the training and test set becomes more narrow.
- The generalization error (test error) first decreases and then increases with λ, indicating that there exists a trade-off between bias and variance.
  - Bias: how much the expected value of the label diverge from the true label
  - Variance: how much, on the average, the estimate varies from the expected value of the label



How to Reduce Over-fitting

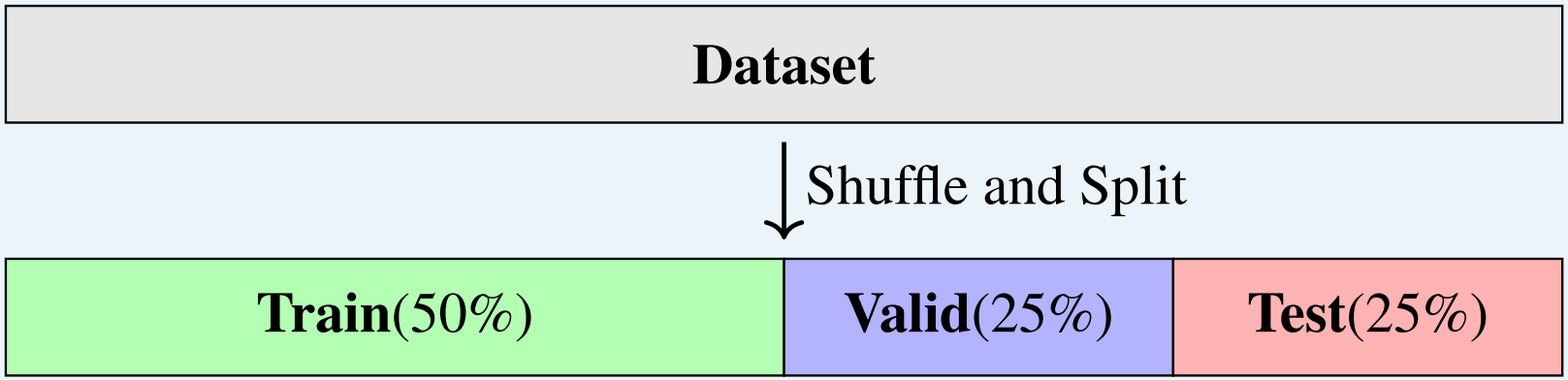
- Under-fitting: when $\log \lambda$ is very large, the training error itself is large, leading to a large bias. Thus the test error is also large.
- Over-fitting: when $\log \lambda$ is very small, although the training error is very small, the model has very large variance, leading to a bad generalization ability or a large test error.
- Good fitting: when $\log \lambda$ is medium, the model has medium training error and low test error.



Cross-Validation I

Hold-out

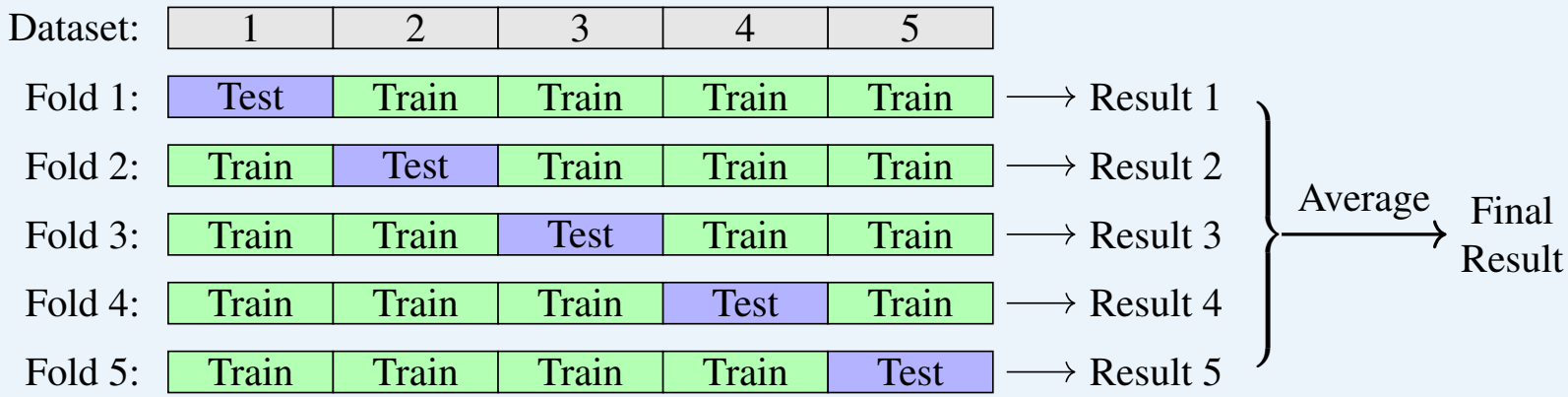
- To estimate the generalization error, we need data unseen during model training.
- As usual, we split the data into:
  - Training set (e.g., 50%)
  - Validation set (e.g., 25%)
  - Test set (e.g., 25%)



Cross-Validation II

K-fold cross-validation

- When data is limited, we may use k-fold cross-validation
  - Shuffle the dataset randomly.
  - Split the dataset into k groups
  - For each unique group, do:
    - take the group as a test set and the remaining groups as a training set,
    - fit the data with the training set and evaluate it on the test set,
    - record the test error.
  - Calculate the average test error of all groups.



Outline

1 Introduction to AI

2 Useful Data Platforms

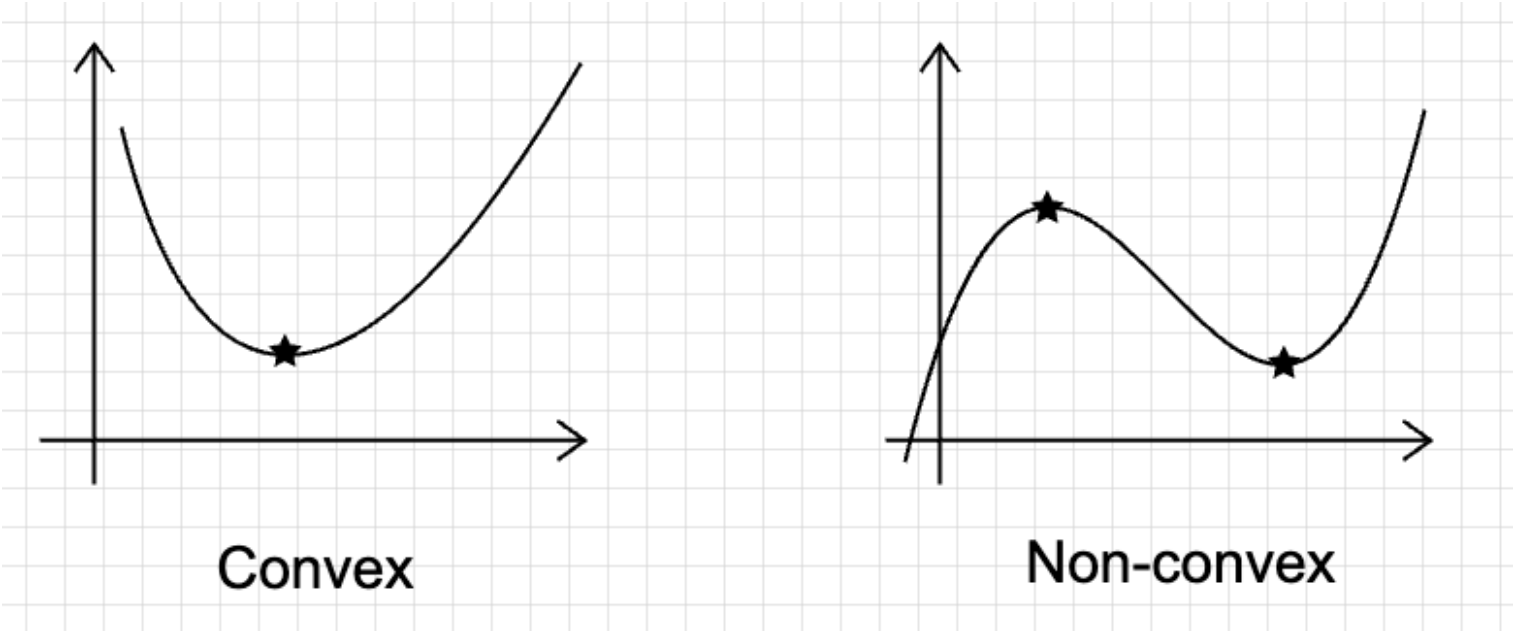
3 Fundamentals of AI
- What Is Machine Learning
- Regression and Classification
- Common AI Models
- Overfitting
- Optimization Algorithms

Optimization Problems

To obtain the minimum value of a problem is to find $\mathbf{x}^*$, s.t. $f(\mathbf{x}) > f(\mathbf{x}^*), \forall \mathbf{x} \in \Omega^n$.

Gradient Descent

- For solving global minima/maxima for convex functions[2]
- For solving local minima/maxima for non-convex functions



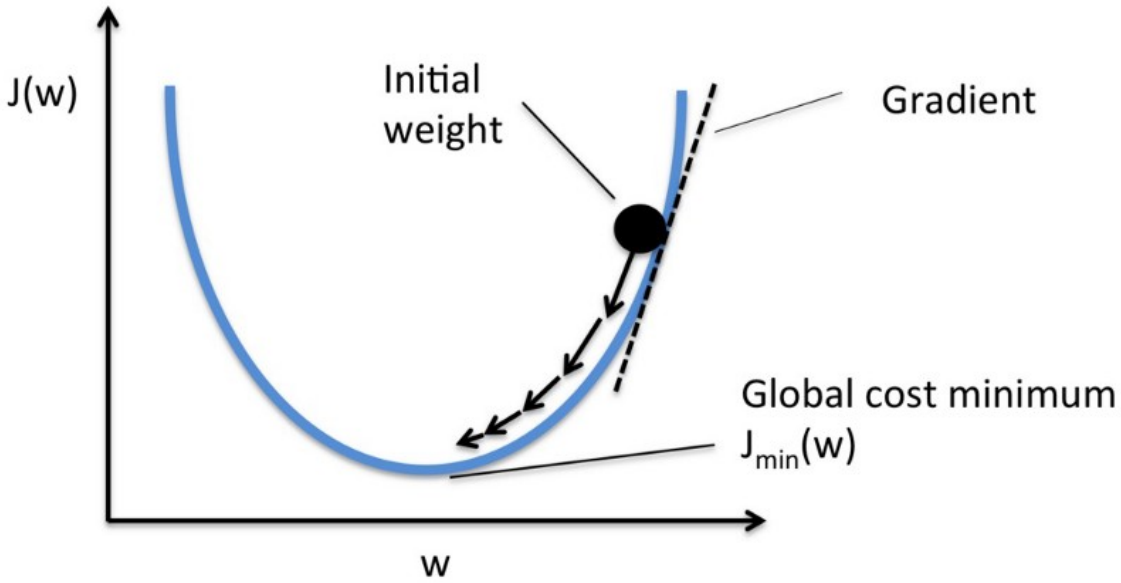
[2] A convex function $f(x)$ is that $\frac{f(x_1)+f(x_2)}{2} > f(\frac{x_1+x_2}{2}), \forall x_1, x_2 \in \Omega$.

Gradient Descent

- Update rules

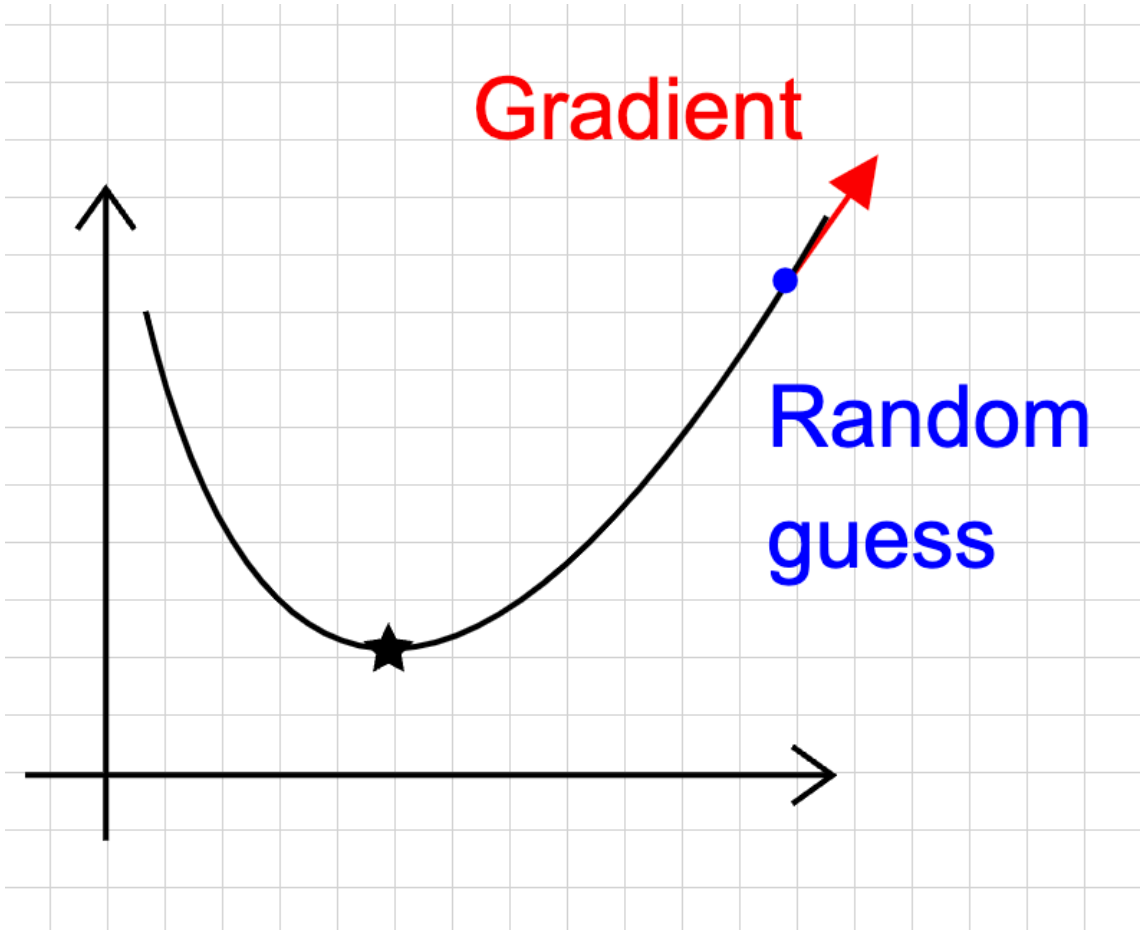
$$\mathbf{x}_{i+1} \leftarrow \mathbf{x}_i - \eta \nabla f(\mathbf{x}_i), i = 1, \cdots, n \tag{16}$$

- where η is the learning rate to control the speed of gradient descent.
- ∇ denote the gradient operator. Note that gradient always points to the direction that the function value increases.



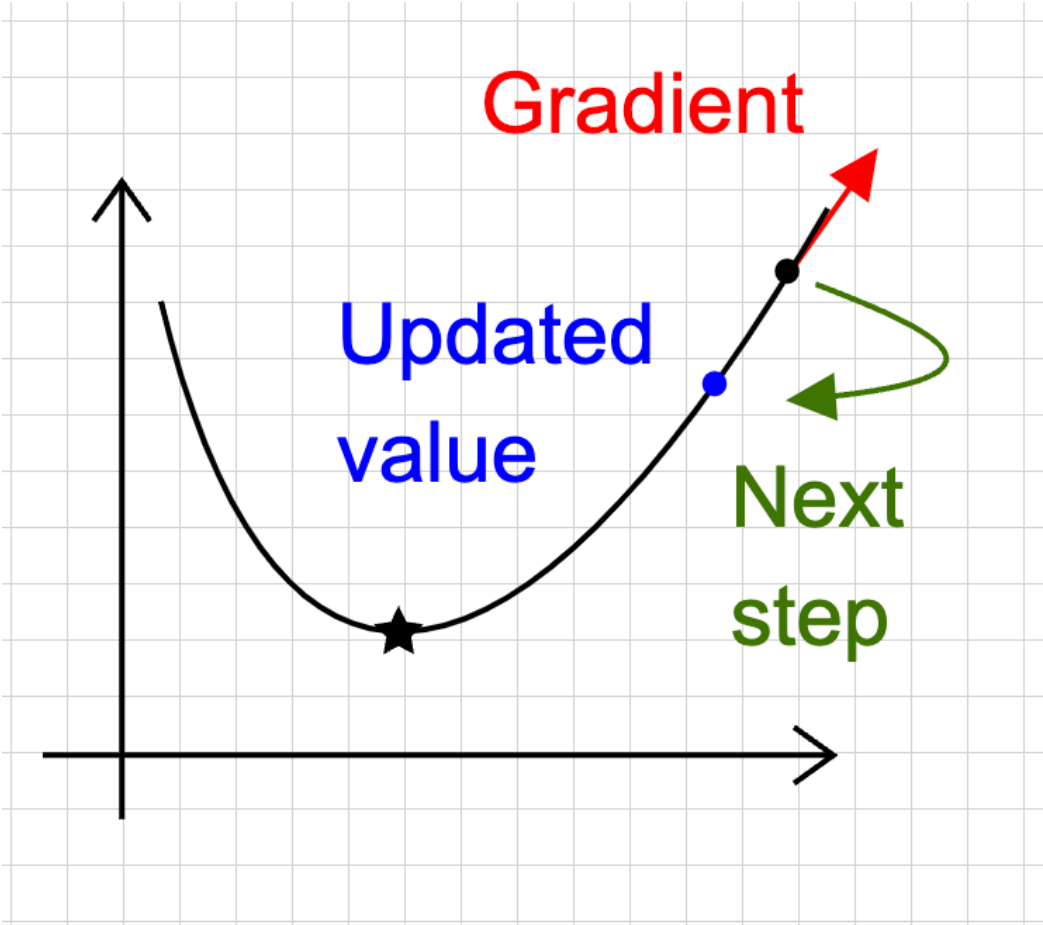
Gradient Descent

- Step 1, initialization



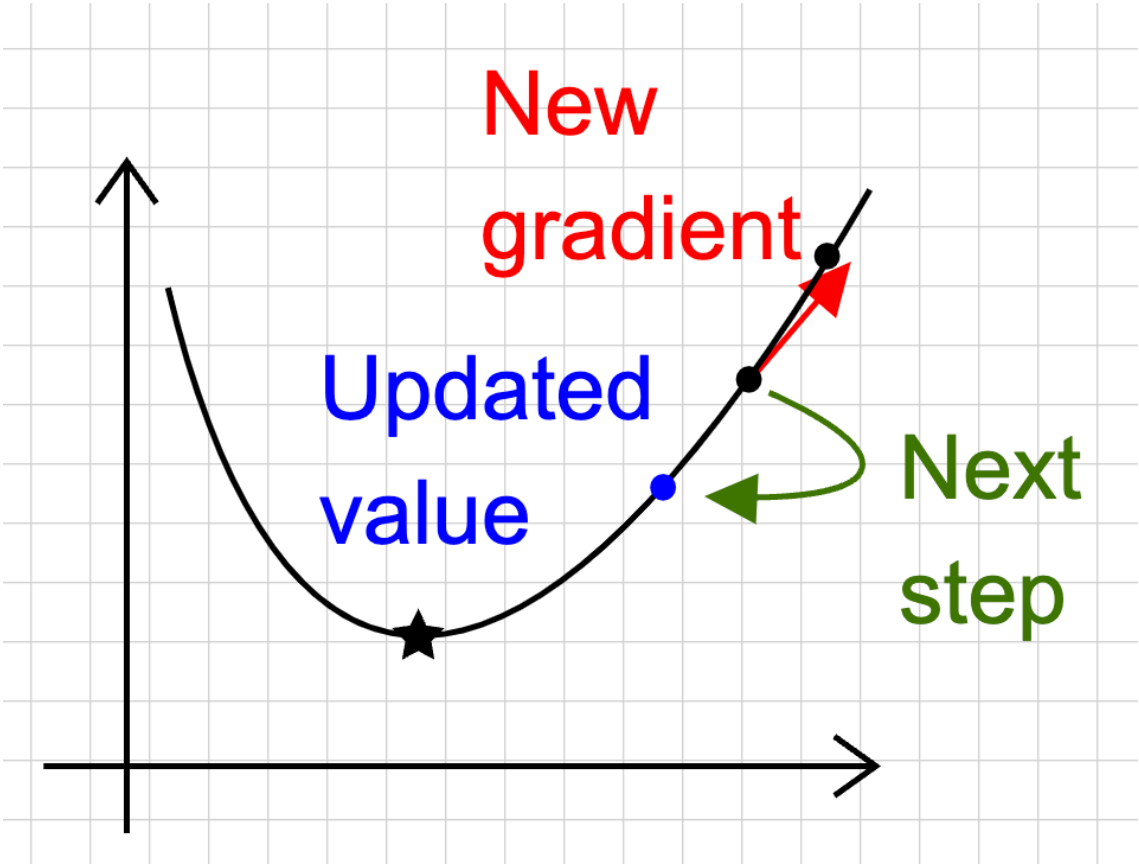
Gradient Descent

- Step 2, obtain gradient. Note that gradient always points to the direction that the function value increases.
- Step 3, update



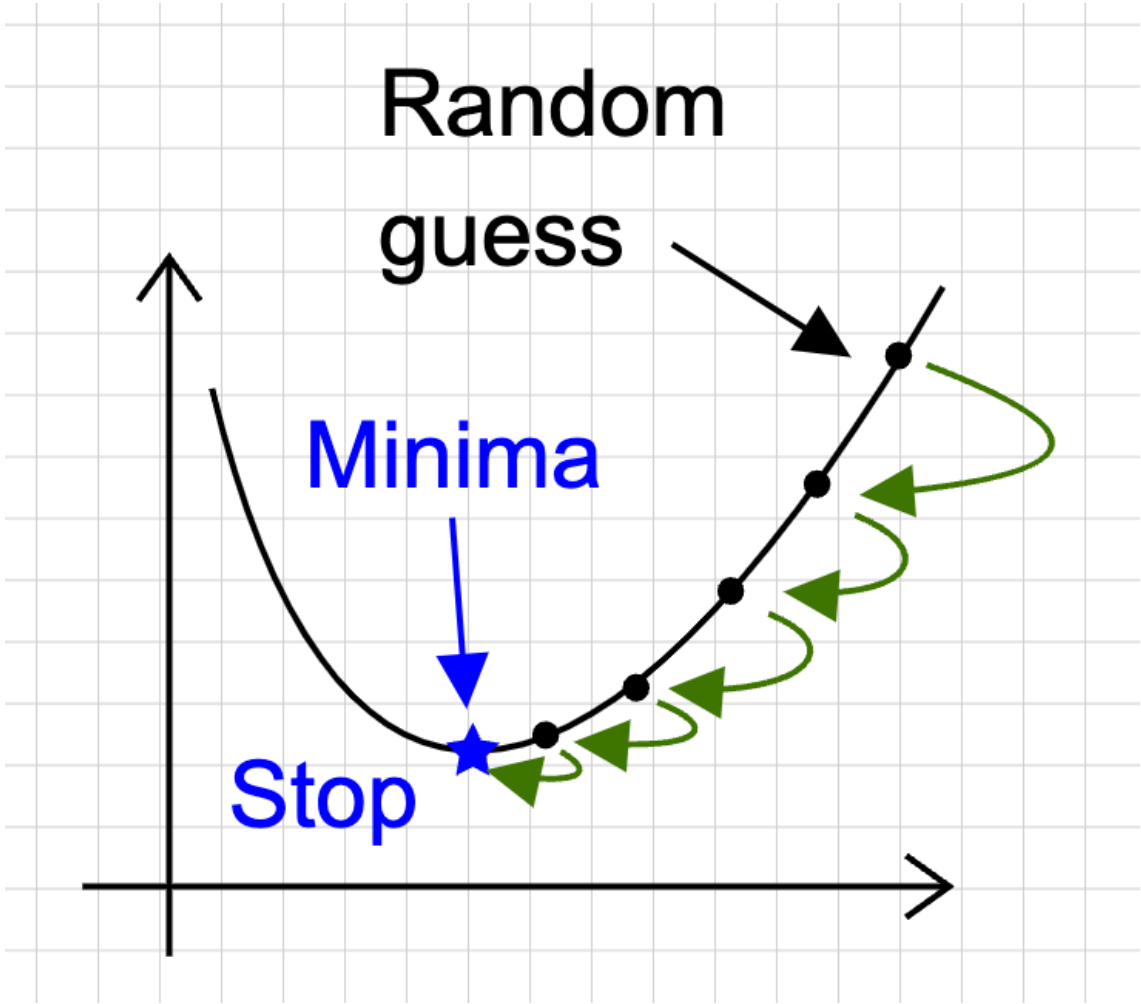
Gradient Descent

- obtain new gradient, update



Gradient Descent

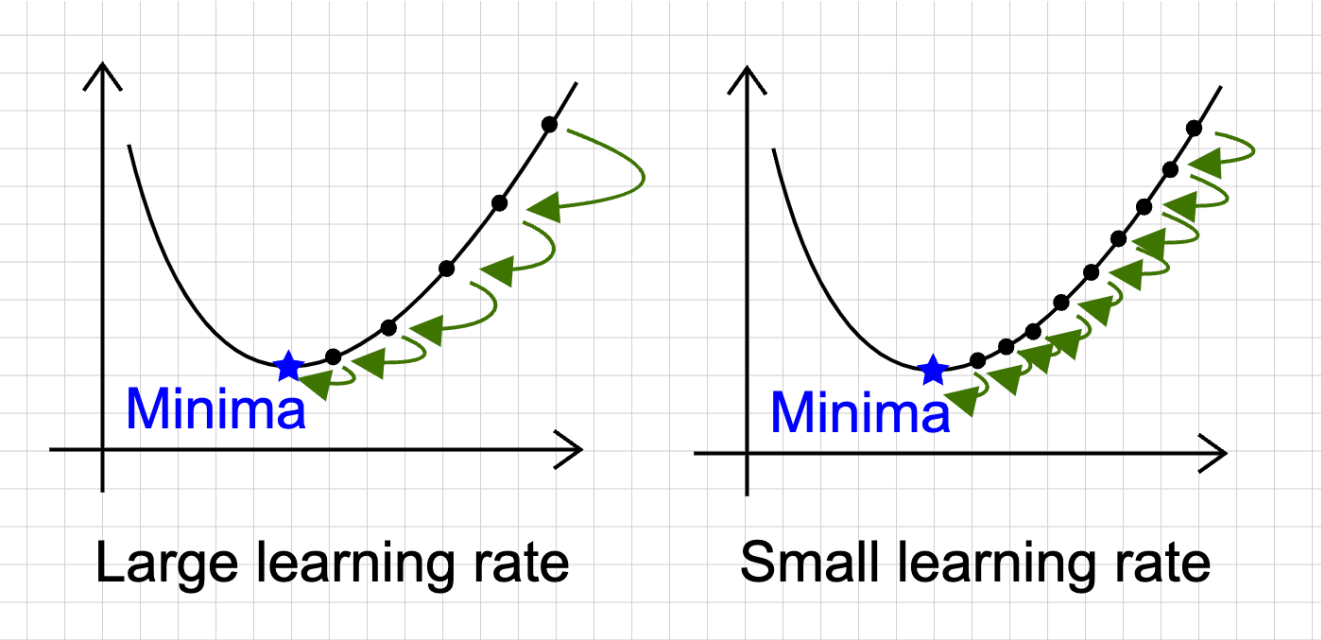
- repeat until "close" to minimum



Gradient Descent

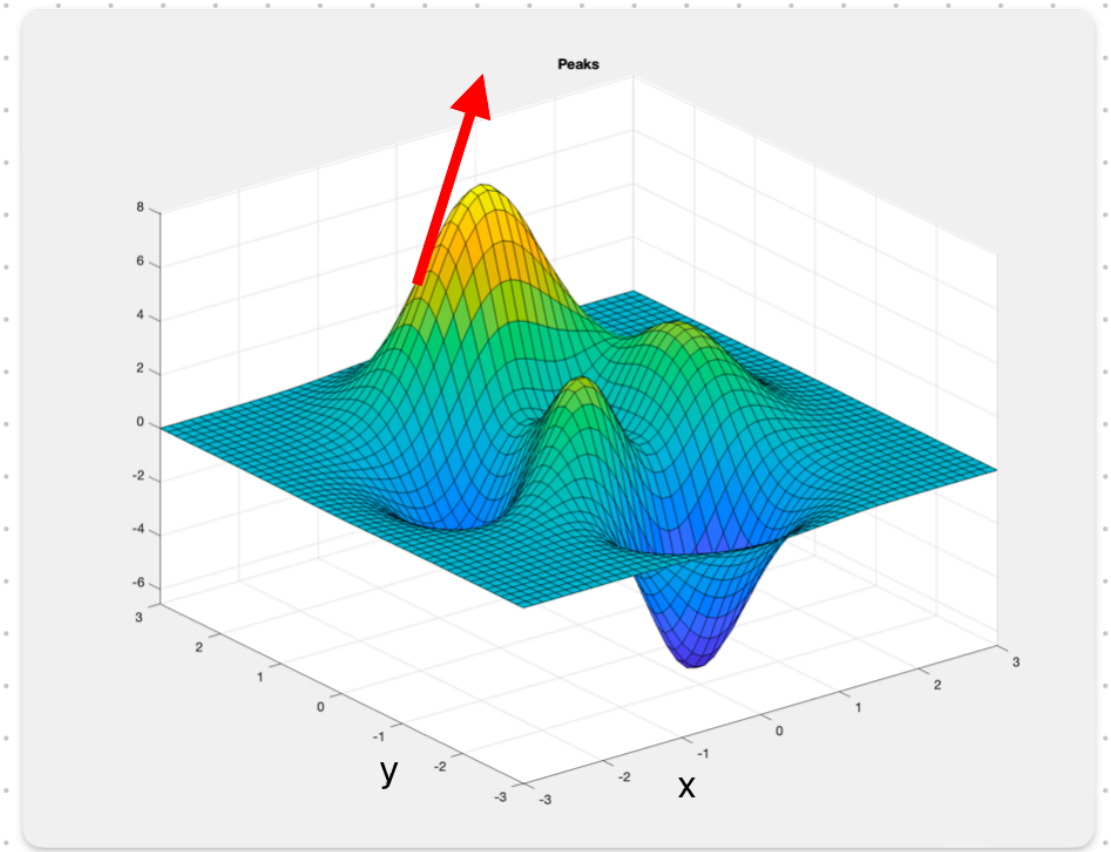
Summary

- Iterative. 1) random initialization 2) obtain gradient 3) update value according to gradient and learning rate 4) repeat until close to minima.
- The update direction is opposite to the gradient. That's why it is called gradient DESCENT.
- Different learning rate may result different descent speed.



Gradient Descent - Multi-variable

Consider function $f(\omega_1, \omega_2)$.

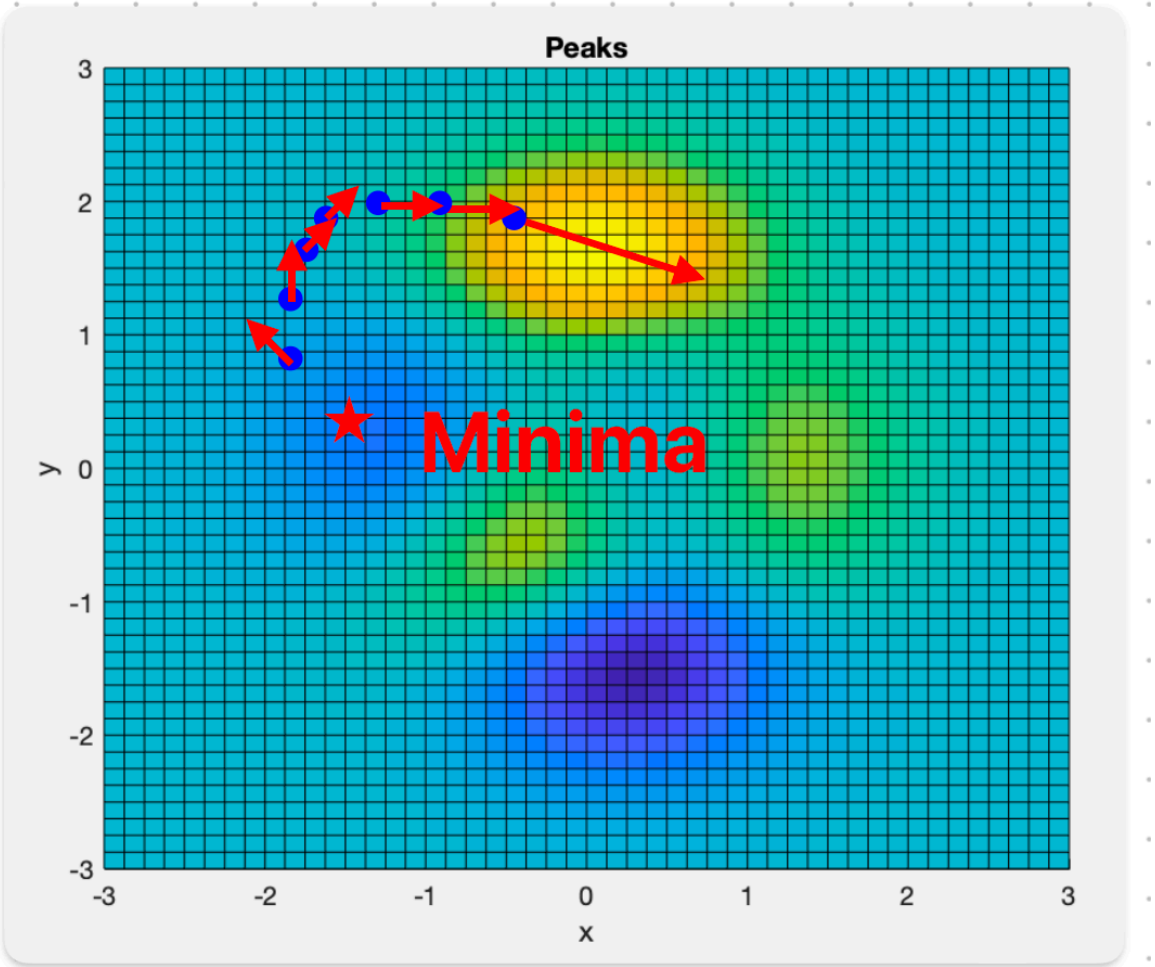


The gradient of a multi-variable function, $f(\cdot)$, is represented as $\nabla f(\cdot)$. In 2-variable case,

$$\nabla f(\omega_1, \omega_2) = \begin{bmatrix} \frac{\partial f(\omega_1, \omega_2)}{\partial \omega_1} \\ \frac{\partial f(\omega_1, \omega_2)}{\partial \omega_2} \end{bmatrix} \quad (17)$$

Gradient Descent - Multi-variable

Consider function $f(\omega_1, \omega_2)$.



Gradient Descent - Multi-variable - Example

Consider function $f(\omega_1, \omega_2) = (\omega_1 - 2)^2 + (\omega_2 - 2)^2, \omega_1, \omega_2 \in \mathbb{R}$. Denote $\omega_1^{(i)}$ and $\omega_2^{(i)}$ the i^{th} iteration result.

Random Initialization

$\omega_1^{(0)} = 1, \omega_2^{(0)} = 1$. Learning rate $\eta = 0.1$

Obtain gradient

$$\nabla f(\omega_1, \omega_2) = \begin{bmatrix} \frac{\partial f(\omega_1, \omega_2)}{\partial \omega_1} \\ \frac{\partial f(\omega_1, \omega_2)}{\partial \omega_2} \end{bmatrix} = \begin{bmatrix} 2\omega_1^{(0)} - 4 \\ 2\omega_2^{(0)} - 4 \end{bmatrix}. \text{ Then } \nabla f(\omega_1^{(0)}, \omega_2^{(0)}) = \begin{bmatrix} -2 \\ -2 \end{bmatrix}.$$

Update value according to gradient and learning rate

$$\begin{bmatrix} \omega_1^{(1)} \\ \omega_2^{(1)} \end{bmatrix} = \begin{bmatrix} \omega_1^{(0)} \\ \omega_2^{(0)} \end{bmatrix} - \eta \times \nabla f(\omega_1^{(0)}, \omega_2^{(0)}) = \begin{bmatrix} 1 \\ 1 \end{bmatrix} - 0.1 \times \begin{bmatrix} -2 \\ -2 \end{bmatrix} = \begin{bmatrix} 1.2 \\ 1.2 \end{bmatrix}.$$

Repeat the above steps.

Stopping Criterions

Given the number of iterations

Given a number of iterations N, stop when N iterations have been performed.

Given error limit

Given an error limit $0 < \epsilon \ll 1$, stop the iteration when $|g(x_n) - g(x_{n-1})| < \epsilon$ (one dimension case) or $|f(\omega_1^{(n)}, \omega_2^{(n)}) - f(\omega_1^{(n-1)}, \omega_2^{(n-1)})| < \epsilon$ (two dimensions case) is satisfied. The same applies for three or more parameters.

Newton's Method

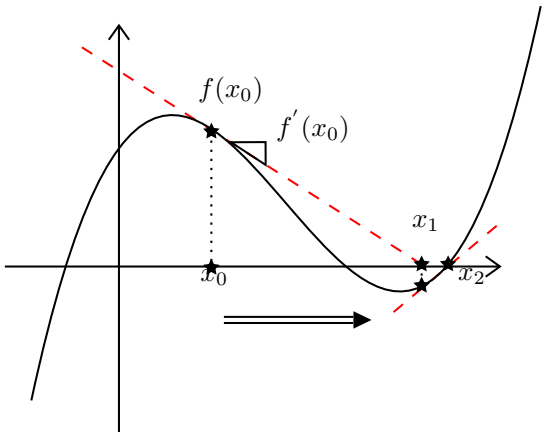
- Suppose we are solving an optimization problem

$$\min g(x) \tag{18}$$

- The optimal solution x^* should satisfy $g'(x^*) = 0$.
- Then the question becomes: how to find x^* such that $g'(x^*) = 0$?
- In other words, how to obtain one zero point of a differentiable function numerically?
- *Newton's method* could be useful.

Newton's Method

- Let $f(x) = g'(x)$, then we try using Newton's method to find the point x^* that satisfies $f(x^*) = g'(x^*) = 0$



$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)} = x_0 - \frac{g'(x_0)}{g''(x_0)}$$

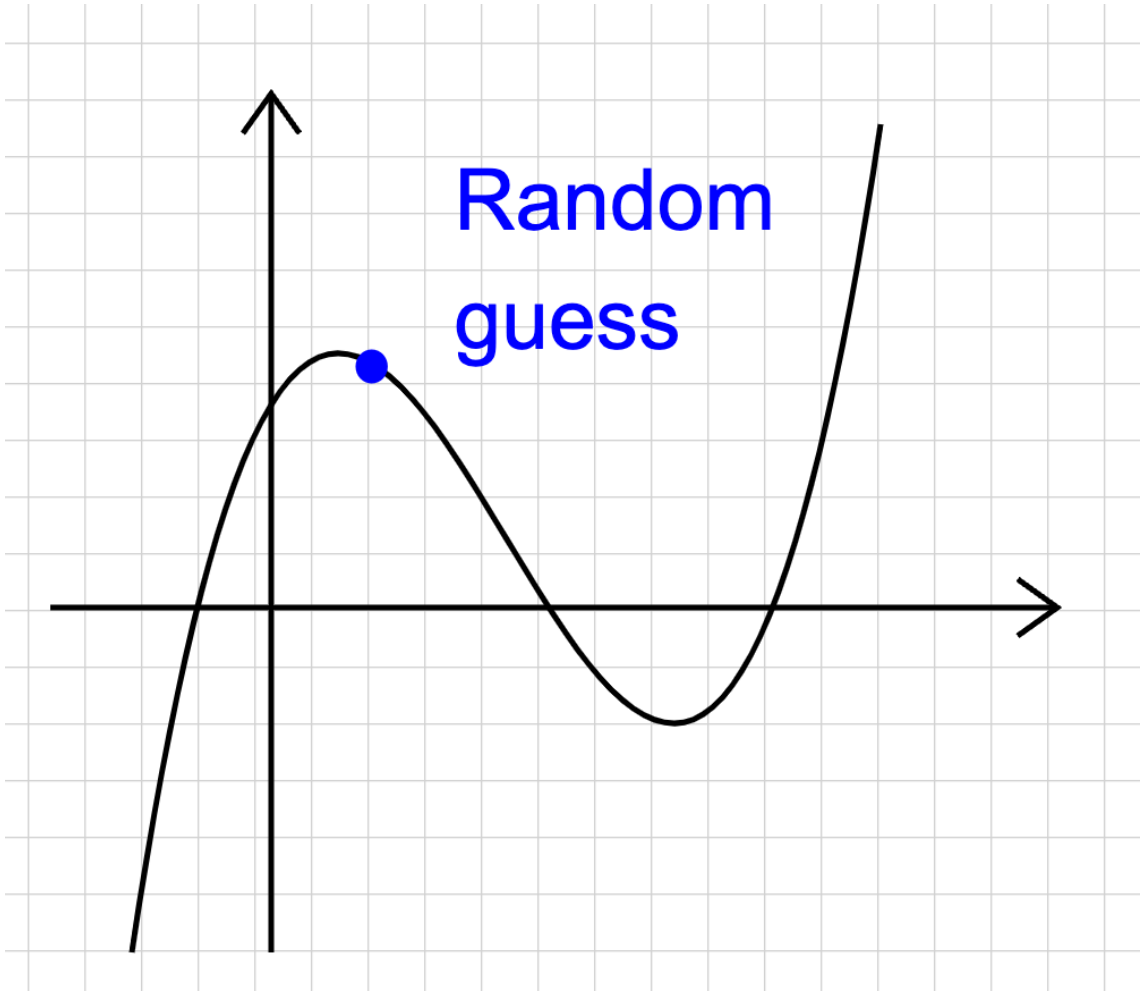
$$x_2 = x_1 - \frac{f(x_1)}{f'(x_1)} = x_1 - \frac{g'(x_1)}{g''(x_1)}$$

$$\vdots$$

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} = x_n - \frac{g'(x_n)}{g''(x_n)}$$

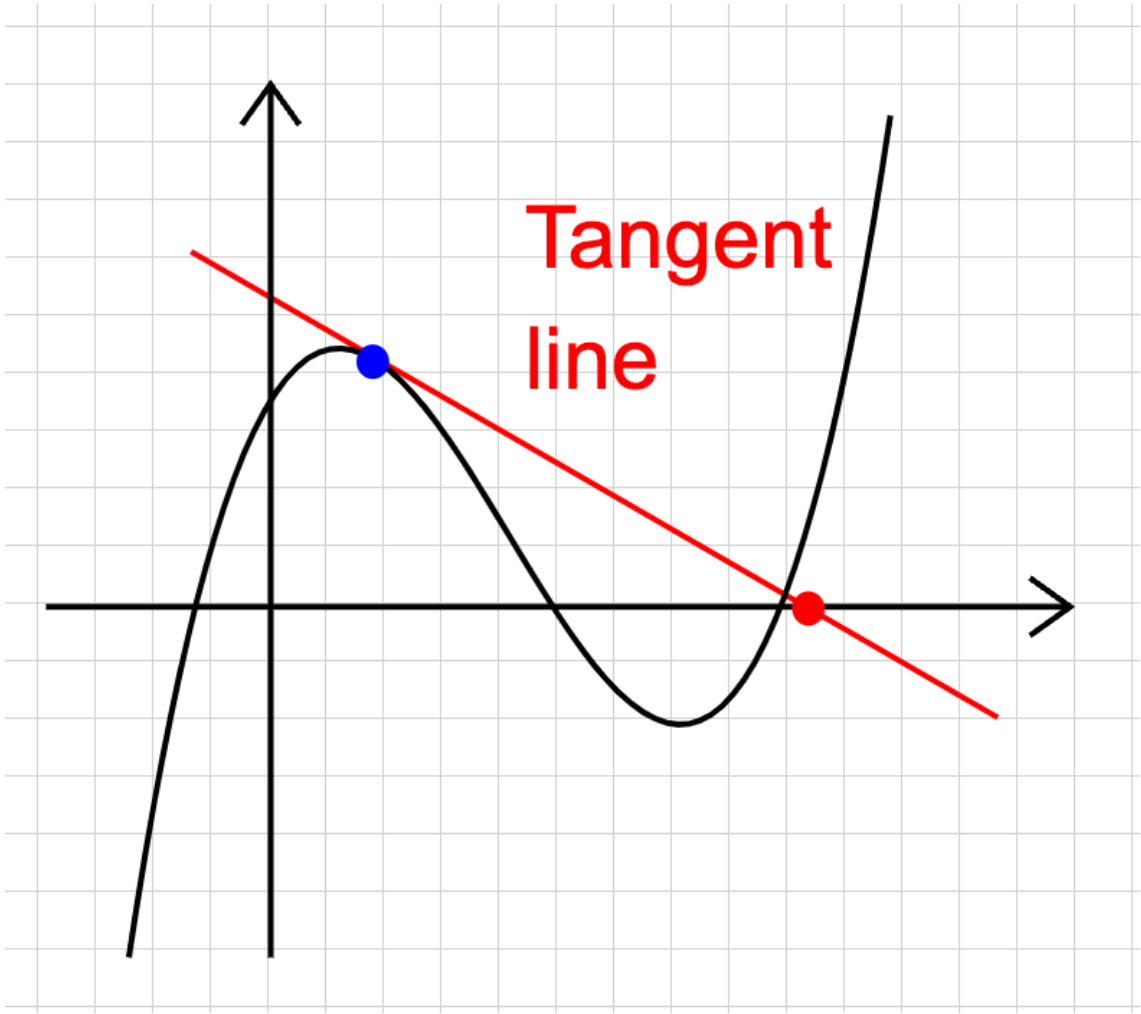
Newton's Method

- Step 1, random initialization



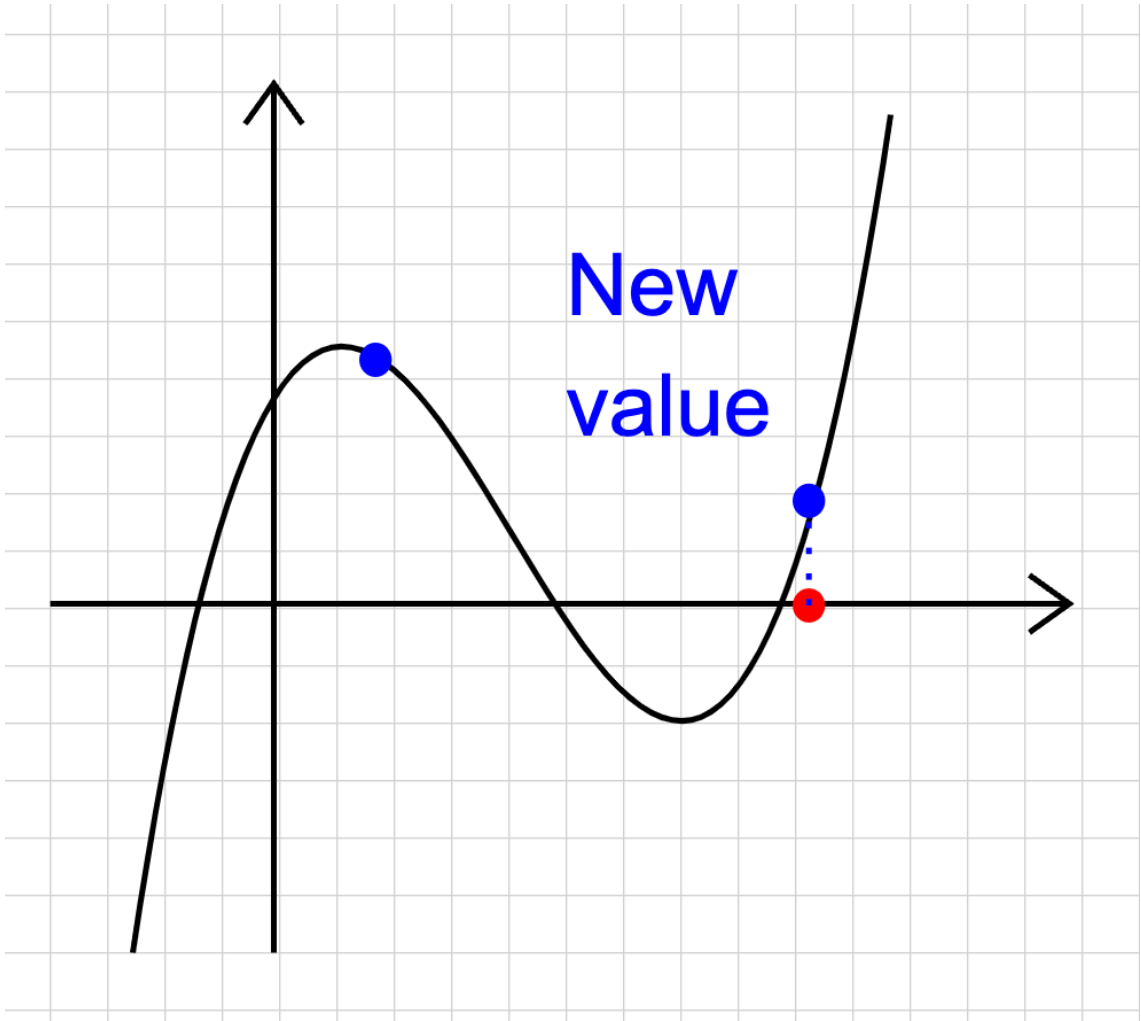
Newton's Method

- Step 2, take the derivative, obtain the zero point of the tangent line



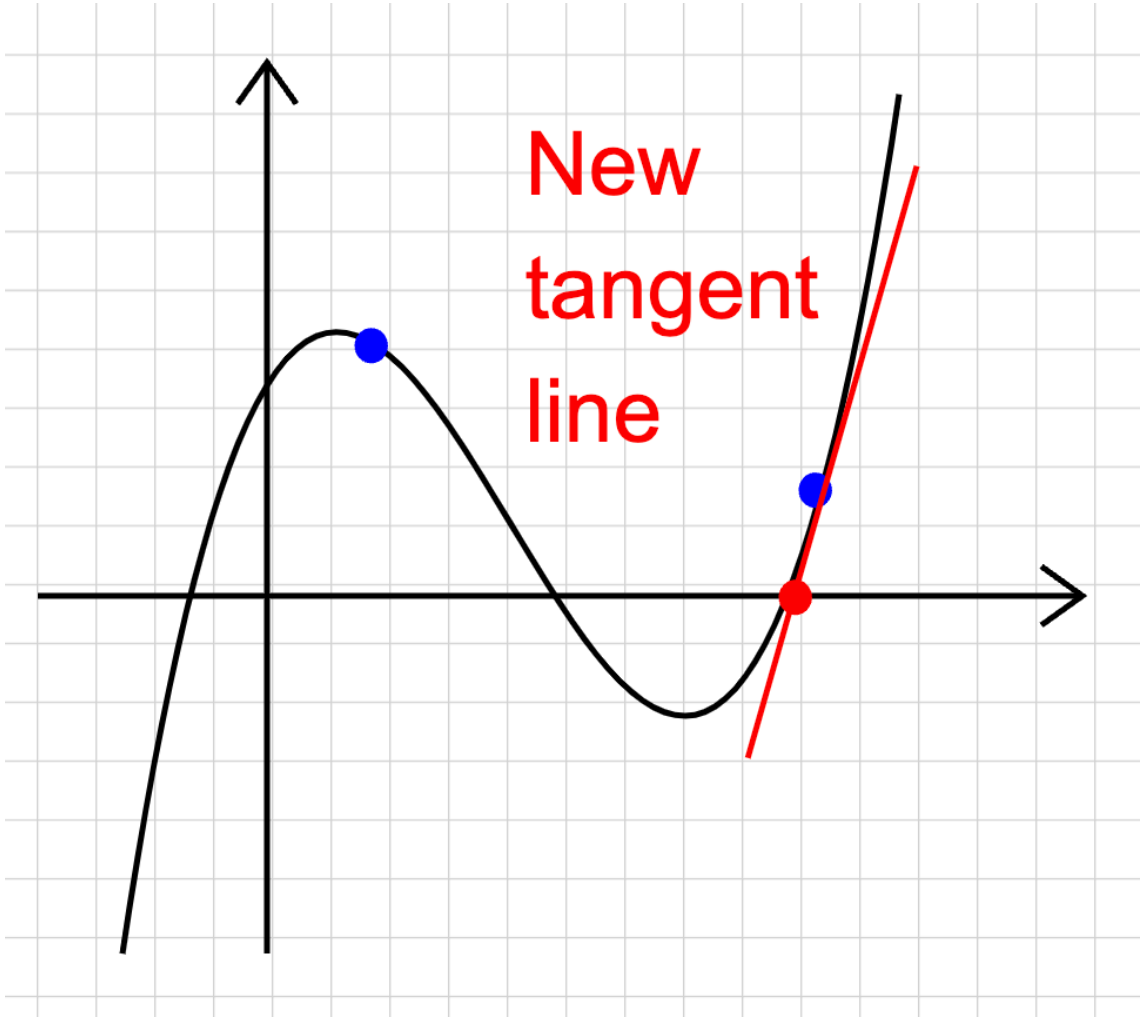
Newton's Method

- Step 3, get the new value, which is the function value of the corresponding zero point



Newton's Method

- Step 4, repeat: take the derivative again and obtain the new tangent line and zero point.



Newton's Method

Summary

- Iterative. 1) random initialization 2) derivative, get zero point of the tangent line 3) get new value 4) repeat until "close" to the zero.
- Fast. And no pre-specified learning rate needed.
- Can only solve for one zero. Different initials may result to different zeros.

References and Further Reading

- Russell and Norvig, *Artificial Intelligence: A Modern Approach*, 4th edition.
- Bishop and Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024.
- Goodfellow, Bengio, and Courville, *Deep Learning*, MIT Press.
- Hastie, Tibshirani, and Friedman, *The Elements of Statistical Learning*, Springer.
- Sutton and Barto, *Reinforcement Learning: An Introduction*, 2nd edition.
- Stanford HAI, *AI Index Report 2026*.
  `https://hai.stanford.edu/ai-index/2026-ai-index-report`